MEMBANGUN ADDON ERP

Odoo dari Nol

SupplyFlow Office

Tutorial distributor perlengkapan kantor: dari Docker dan model pertama sampai persetujuan, RFQ, retur pemasok, antarmuka, pengujian, serta backup/restore.

Muhammad Rafik | Odoo Community 19 | 28 Oktober 2026

Cara memakai: ikuti bab 1-7, lalu gunakan indeks file/fungsi pada bab 8 ketika membuka source. Blok kode panjang dapat terbungkus pada PDF; tanda > di awal baris lanjutan hanya penanda tata letak. Salin kode dari Markdown/source yang ditautkan untuk mempertahankan baris aslinya.

Daftar isi

Membuat ERP Odoo dari nol: SupplyFlow Office 6

Mulai dari sini . . . 6

Pilih cara mengikuti . . . 6

Versi PDF . . . 7

Cara membaca contoh kode . . . 7

Pemeriksaan panduan . . . 7

Peta hubungan paling awal . . . 7

Versi dan rujukan . . . 8

1. Persiapan dan Docker 9

1.1 Yang perlu dipasang . . . 9

1.2 Arti image, container, dan volume . . . 9

1.3 Membaca compose.yaml . . . 10

1.4 Siapa membuat password dan konfigurasi? . . . 11

1.5 Role database dan startup . . . 12

1.6 Folder kosong untuk jalur membangun ulang . . . 12

Checkpoint . . . 13

2. Kerangka addon dan hak akses 14

2.1 Susun folder addon . . . 14

2.2 Daftarkan addon melalui manifest . . . 14

2.3 Hubungkan import Python . . . 15

2.4 Tulis model pertama . . . 15

2.5 Buat kelompok pengguna . . . 16

2.6 ACL: boleh melakukan operasi apa? . . . 16

2.7 Record rule: record mana yang boleh terlihat? . . . 17

2.8 Buat daftar, form, dan action . . . 17

2.9 Hubungkan menu ke action . . . 18

2.10 Jalankan checkpoint kerangka . . . 18

2.11 Mengganti starter dengan kode lengkap . . . 19

03. Membuat model permintaan pembelian dan RFQ 20

1. Tentukan dokumen yang hendak dibuat . . . 20

2. Buat identitas kelas dan field dokumen . . . 21

3. Buat bentuk baris dan perhitungan estimasi . . . 22

4. Siapkan pemeriksaan bersama sebelum menyimpan dan mengganti status . . . 25

5. Tulis create() dan write() dokumen induk . . . 26

6. Bangun perpindahan status satu per satu . . . 28

7. Ubah permintaan yang disetujui menjadi RFQ native . . . 30

8. Jaga setiap jalur edit baris barang . . . 32

9. Hubungkan fungsi ke form XML . . . 33

10. Uji satu alur melalui akun latihan . . . 34

Peta fungsi yang belum dikutip utuh . . . 35

Kesalahan yang sering membuat implementasi berbeda dari tujuan . . . 36

04. Membuat kasus penerimaan barang dan retur pemasok 37

1. Siapkan file yang akan saling terhubung . . . 37

2. Tentukan receipt, move, dan kasus sebagai objek berbeda . . . 38

3. Susun status sebelum menulis tombol . . . 39

4. Lindungi perubahan sebelum menambahkan aksi review . . . 39

5. Validasi receipt dan baris di server . . . 41

6. Tambahkan aksi pengajuan, review, dan pembatalan . . . 41

7. Buat kelas baris di bawah kelas parent . . . 42

8. Hitung sisa retur dari sumber yang sama . . . 43

9. Buat picking melalui wizard Odoo . . . 44

10. Tambahkan revisi sumber untuk transaksi bersamaan . . . 46

11. Hubungkan tombol, ACL, sequence, dan manifest . . . 46

12. Periksa alur melalui UI dan tes . . . 47

Jika hasilnya berbeda . . . 48

05. Membuat form, workspace, dan PDF **50**

1. Kenali tiga jenis tampilan . . . 50

2. Daftarkan XML dan asset dalam manifest . . . 50

3. Buat daftar permintaan pembelian . . . 51

4. Susun form dan hubungkan tombol ke metode Python . . . 51

5. Tambahkan pencarian dan window action . . . 53

6. Baca form kasus pemasok dengan pola yang sama . . . 53

7. Hubungkan menu dengan action . . . 54

8. Siapkan endpoint model untuk workspace . . . 55

9. Buat komponen Owl dan muat data . . . 55

10. Buka daftar/form dari workspace . . . 56

11. Pasang template dan tata letak . . . 57

12. Buat binding report dan isi QWeb . . . 58

13. Pasang paket final pada folder latihan . . . 58

14. Periksa hasil di browser . . . 60

06. Menguji aturan ERP dan mencari penyebab error **61**

1. Pisahkan tiga lapisan pemeriksaan . . . 62

2. Pahami struktur tes ORM . . . 62

3. Uji alur permintaan dan penolakannya . . . 63

4. Uji kasus receipt dan perubahan stok sebenarnya . . . 64

5. Uji integrasi seluruh distribusi dan angka workspace . . . 65

6. Jalankan ORM pada database terisolasi . . . 66

7. Uji validator arsip tanpa PostgreSQL . . . 67

8. Siapkan browser test . . . 67

9. Baca helper login, RPC, dan tombol . . . 68

10. Telusuri sembilan workflow . . . 69

11. Ambil screenshot dari aplikasi yang berjalan . . . 70

12. Cari error dari lapisan yang paling dekat . . . 71

13. Susun urutan CI yang dapat diulang . . . 72

07. Mengisi data latihan, menjaga data, dan mencoba satu alur lengkap 73

1. Mengapa seed dijalankan melalui shell Odoo? . . . 73

2. Ikuti isi training_data.py . . . 74

3. Buat satu transaksi baru dari awal sampai selesai . . . 75

4. Mengapa backup berisi database dan filestore? . . . 76

5. Baca restore sebelum menjalankannya . . . 77

6. Mengapa sisa tes dibersihkan dengan guard? . . . 78

7. Latihan kecil: tambah referensi internal permintaan . . . 78

8. Simpan pekerjaan dan hentikan layanan . . . 79

Periksa pemahaman . . . 79

08. Indeks file, class, dan fungsi 81

Peta semua berkas teks proyek . . . 81

Class Python: tempat menaruh field dan method . . . 86

Semua fungsi Python aplikasi dan script . . . 86

Semua fungsi PowerShell . . . 91

Semua method komponen Owl . . . 92

Semua ekspor helper browser . . . 92

Jika mencari tes atau fungsi bawaan Odoo . . . 93

Membuat ERP Odoo dari nol: SupplyFlow Office

Panduan ini mengikuti kode SupplyFlow Office untuk distributor perlengkapan kantor. Kamu akan menyiapkan Odoo, membuat addon Python, menghubungkannya ke tampilan, kemudian mengikuti permintaan pembelian sampai RFQ dan kasus receipt sampai retur pemasok.

Odoo sudah menyediakan login, pengguna, perusahaan, Purchase, Inventory, Sales, ORM, dan web client. Pekerjaan kita adalah membuat addon yang memperluas bagian tersebut. Karena itu, tutorial tidak meminta kamu membuat mesin stok atau login baru.

Mulai dari sini

Urutan	Bab	Hasil belajar
1	Persiapan dan Docker	Memahami server, database, container, volume, secret, dan proses startup
2	Kerangka addon dan hak akses	Membuat addon minimal yang dapat dipasang, model pertama, menu, ACL, dan record rule
3	Model permintaan dan RFQ	Menulis field, perhitungan, validasi, persetujuan, dan pembuatan RFQ native
4	Receipt, masalah, dan retur	Menghubungkan kasus dengan barang yang diterima dan membuat retur yang jumlahnya dibatasi
5	Tampilan, workspace, dan PDF	Menghubungkan XML, tombol Python, Owl, dan QWeb serta melengkapi addon akhir
6	Pengujian dan debugging	Membaca tes nyata, menjalankan database uji terpisah, dan mencari sumber kesalahan
7	Operasi dan latihan	Memahami seed, backup/restore, menjalankan satu kasus lengkap, dan mengembangkan perubahan kecil
8	Indeks file dan fungsi	Mencari file, class, fungsi, serta lokasi pembahasannya

Istilah baru dijelaskan saat pertama dipakai. Tidak perlu memahami semua isi `purchase_request.py` sebelum mencoba bab pertama.

Pilih cara mengikuti

Membaca sambil membuka proyek yang sudah jadi. Gunakan folder `D:\kerjaan\done push repo\supplyflow-odoo`. Buka file yang ditautkan, cocokkan fungsi dengan penjelasannya, dan gunakan akun latihan untuk checkpoint UI. Jangan mengganti source proyek utama dengan contoh kerangka minimal bab 2.

Membangun ulang sebagai latihan. Bab 1 menyiapkan folder baru `D:\Latihan-Odoo\supplyflow-office` dan proyek Compose bernama `supplyflow-belajar`. Bab 2 merupakan checkpoint addon minimal. Bab 3 dan 4 menjelaskan penggantian model dengan implementasi lengkap. Bab 5 menyatukan seluruh file final sebelum instalasi/update berikutnya. Tidak perlu menyalin `node_modules`, database, arsip, atau secret proyek utama.

Folder latihan bersifat pilihan. Jika tujuanmu memahami kode, cukup gunakan proyek utama agar tidak menambah salinan yang membingungkan.

Versi PDF

Unduh buku tutorial PDF. PDF memiliki daftar isi bernomor halaman dan bookmark bab. Jika baris kode panjang terbungkus pada PDF, gunakan berkas Markdown atau source yang ditautkan untuk menyalin kode dengan susunan baris asli.

PDF dibuat dari sembilan berkas Markdown pada folder ini menggunakan build_tutorial_pdf.py. Untuk membuat ulang, pasang paket Python `reportlab` pada environment tooling yang kamu gunakan, lalu jalankan `python scripts/build_tutorial_pdf.py --publish-copy` dari root repo. Odoo tidak memerlukan generator PDF tutorial ini untuk berjalan.

Cara membaca contoh kode

- **Isi lengkap file:** boleh ditulis sebagai isi file yang disebutkan. Jika file sudah berisi versi contoh sebelumnya, ganti isinya; jangan menambahkan class kedua dengan nama yang sama.
- **Cuplikan class/fungsi:** menunjukkan bagian dari file final. Pertahankan import, decorator, field, dan fungsi lain dari source lengkap. Jangan menimpa satu file dengan satu cuplikan.
- **Perintah PowerShell:** dijalankan di terminal Windows pada folder yang disebutkan.
- **Kode ORM:** dijalankan pada shell/test Odoo yang menyediakan env, bukan pada Python biasa di Windows.
- **Hasil yang diharapkan:** checkpoint untuk praktikmu. Ini tidak berarti contoh perantara telah diuji sebagai paket aplikasi final.

File final yang ditautkan adalah rujukan utuh. Proyek final telah lulus 42 tes ORM, 25 tes arsip, dan sembilan workflow browser pada CI yang dicatat. Kerangka bab 2 tidak memiliki seluruh fitur atau pengamanan aplikasi akhir.

Pemeriksaan panduan

Pada 28 Oktober 2026, sembilan file starter diekstrak dari blok kode bab 2 dan dipasang memakai image proyek ke database uji kosong yang terpisah. Instalasi berhasil; konfigurasi pertama, model minimal, daftar kosong, grup admin, action, menu, dan view diperiksa. Database itu kemudian di-update dengan addon final sesuai tahap bab 5: modul berstatus installed, keempat model bisnis tersedia, permintaan/kasus tetap kosong, dan menu root tidak menjadi duplikat.

Pemeriksaan tersebut mencakup instalasi/konfigurasi model serta metadata view melalui Odoo, bukan click-through browser baru untuk kerangka minimal. Hasil browser aplikasi lengkap mengacu ke verifikasi proyek yang sudah dicatat. Database dan filestore uji dibersihkan sesudahnya; source dan database proyek utama tidak digunakan sebagai tempat menulis transaksi percobaan.

Peta hubungan paling awal

text

```
Browser di localhost:8070
  -> web client Odoo: menampilkan form dan mengirim aksi
  -> method model Python: memeriksa izin dan aturan bisnis
  -> ORM: membaca/menulis record PostgreSQL
  -> respons: form, status, atau dokumen native yang dibuka
```

```
Docker Compose
  -> container Odoo: menjalankan server dan addon
  -> container PostgreSQL: menyimpan data relasional
  -> volume Odoo: menyimpan filestore/lampiran
```

Saat menekan **Create RFQ**, browser memanggil method Python. Method itulah yang membuat `purchase.order`. XML hanya menghubungkan tombol dengan method dan mengatur tampilan.

Versi dan rujukan

Panduan memakai Odoo Community 19 dan source implementasi commit `ad7be04`, dengan pembaruan dokumentasi setelahnya. Nama API mengikuti versi itu, misalnya `stock.move.quantity`, `models.Constraint`, dan `<list>`. Jangan menggantinya dengan contoh versi lain tanpa memeriksa perbedaan API.

Rujukan utama tetap source proyek. Dokumentasi resmi untuk penjelasan platform: arsitektur Odoo, ORM, hak akses, dan pengujian.

1. Persiapan dan Docker

Hasil bab ini: kamu dapat menunjukkan tempat browser, server Odoo, database, addon, dan lampiran berjalan. Kamu juga mengetahui fungsi file yang menyalakan proyek.

1.1 Yang perlu dipasang

Gunakan Docker Desktop dengan Linux containers dan PowerShell. Git diperlukan untuk mengambil source dari GitHub jika proyek belum ada. Node.js hanya diperlukan untuk tes Playwright; aplikasi ERP tidak memakai server Node terpisah. Python dan PostgreSQL untuk server ERP sudah tersedia di image container, sehingga tidak perlu memasangnya lagi di Windows untuk menjalankan aplikasi.

Periksa dari PowerShell:

powershell

```
docker version
docker compose version
```

`docker version` harus dapat menampilkan bagian Client dan Server. Jika Server tidak tersedia, jalankan Docker Desktop terlebih dahulu.

Untuk membaca proyek yang sudah jadi, gunakan:

powershell

```
cd 'D:\kerjaan\done push repo\supplyflow-odoo'
Remove-Item Env:COMPOSE_PROJECT_NAME -ErrorAction SilentlyContinue
.\scripts\Start.ps1 -WithTrainingData -SkipPull
```

`-SkipPull` sesuai untuk komputer ini karena image proyek sudah pernah ditarik. Pada komputer baru, hilangkan opsi tersebut. Buka `http://localhost:8070`; kredensial ada di `.secrets/LOCAL-CREDENTIALS.txt`. Hentikan dengan `Stop.ps1` setelah selesai supaya layanan tidak terus memakai RAM.

1.2 Arti image, container, dan volume

Istilah	Pada proyek ini
Image	Paket Odoo 19 atau PostgreSQL 17 yang digunakan untuk membuat container
Container	Proses terisolasi yang menjalankan Odoo atau PostgreSQL dari image
Addon	Folder Python/XML/JavaScript `supplyflow_office` yang dibaca Odoo
Bind mount	Folder source di Windows terlihat di container; `:ro` berarti hanya dibaca dari container
Named volume	Penyimpanan Docker untuk database dan filestore, terpisah dari umur container
Secret	Berkas password yang dipasang ke `/run/secrets/...`
Filestore	Berkas lampiran Odoo; tabel database menyimpan metadata dan hubungannya

Mengganti container tidak sama dengan menghapus volume. Menyalin source repo juga tidak menyalin isi database dan filestore. Karena itu, proyek mempunyai backup yang mengambil keduanya.

1.3 Membaca `compose.yaml`

Buka file lengkap. Dua service bernama `postgres` dan `odoo` terhubung melalui jaringan internal Compose.

Cuplikan asli service database:

yaml

```yaml
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: supplyflow
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
    secrets:
      - postgres_password
      - db_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts/container/init-database.sh:/docker-entrypoint-initdb.d/10-supplyflow.sh:ro
```

`POSTGRES_DB` menentukan database awal. Password administrator PostgreSQL dibaca dari berkas, bukan ditulis di YAML. `init-database.sh` dimuat pada inisialisasi volume PostgreSQL baru. Ia tidak dijalankan ulang setiap restart.

Cuplikan asli service Odoo, bukan isi seluruh file:

yaml

```yaml
    environment:
      HOST: postgres
      USER: supplyflow
      PASSWORD_FILE: /run/secrets/db_password
      ODOO_RC: /run/odoo/odoo.conf
```

`HOST: postgres` berarti nama service di jaringan Docker. Jika Odoo memakai `localhost` sebagai host database di dalam containernya sendiri, ia akan mencari PostgreSQL di container Odoo, padahal database berada di container lain.

Bagian penyimpanan dan port:

yaml

```yaml
    volumes:
      - odoo_data:/var/lib/odoo
      - ./.runtime/odoo.conf:/run/odoo/odoo.conf:ro
      - ./addons:/mnt/extra-addons:ro
      - ./scripts/container:/opt/supplyflow:ro
    ports:
      - "127.0.0.1:8070:8069"
```

Browser Windows mengakses port `8070`; server Odoo di container mendengarkan `8069`. Binding `127.0.0.1` membatasi akses ke komputer ini. PostgreSQL tidak mempunyai mapping port host pada Compose proyek.

Healthcheck PostgreSQL memakai TCP `127.0.0.1`, sehingga tidak menganggap server socket sementara saat bootstrap sebagai server final yang sudah siap. `depends_on` menunggu status sehat sebelum memulai service yang bergantung padanya. Healthcheck Odoo memanggil `/web/health`; keberhasilan endpoint ini bukan pengganti tes alur bisnis.

Image Odoo di source dipatok dengan tag tanggal serta digest. Proyek memakai image resmi secara langsung; tidak ada Dockerfile custom. Batas memori Odoo pada Compose adalah 1536 MiB, sedangkan PostgreSQL dan aplikasi Windows tetap membutuhkan memori sendiri.

1.4 Siapa membuat password dan konfigurasi?

Buka Common.ps1. Tiga fungsi utamanya:

Fungsi	Masukan dan hasil
`Invoke-Compose`	Menerima argumen CLI, menjalankan Compose pada root proyek, dan menghentikan script bila exit code gagal
`New-LocalPassword`	Membuat 30 byte acak kriptografis, lalu mengembalikannya sebagai teks Base64
`Initialize-LocalConfiguration`	Memeriksa volume lama, mempertahankan secret yang ada, membuat secret baru jika diperlukan, lalu menulis config dan petunjuk login lokal

Cuplikan fungsi lengkap asli di `Common.ps1`:

powershell

```
function Invoke-Compose {
    # Keep native flags such as -d and -v out of PowerShell common-parameter binding.
    & docker compose --project-directory $script:ProjectRoot @args
    if ($LASTEXITCODE -ne 0) { throw "Docker Compose failed (exit $LASTEXITCODE)." }
}
```

`$PSScriptRoot` menunjuk folder script. Root proyek dihitung dari situ agar perintah tidak bergantung pada folder terminal yang kebetulan sedang aktif. `@args` meneruskan argumen, misalnya `up -d --wait postgres`. Fungsi ini sengaja bukan advanced function agar flag native seperti `-d` tidak tertangkap sebagai common parameter PowerShell.

Folder yang dihasilkan:

text

```
.secrets/
  postgres_password          administrator PostgreSQL
  db_password                role koneksi aplikasi supplyflow
  admin_password             login admin Odoo saat setup pertama
  master_password            password pengelolaan database Odoo
  training_credentials.json  password tiap akun latihan
  LOCAL-CREDENTIALS.txt       petunjuk login lokal
.runtime/
  odoo.conf                  konfigurasi Odoo untuk instance ini
```

Password database, password admin aplikasi, dan master password memiliki pekerjaan berbeda. Mengubah isi berkas secret tidak otomatis mengubah password yang sudah tersimpan pada database atau akun Odoo.

Jika volume data sudah ada tetapi secret aslinya hilang, script berhenti. Membuat password baru diam-diam justru dapat memutus hubungan dengan database lama. `.gitignore` mengecualikan folder-foder tersebut dari Git.

1.5 Role database dan startup

init-database.sh membuat role aplikasi:

sql

```
CREATE ROLE supplyflow LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION PASSWORD :'app_password';
ALTER DATABASE supplyflow OWNER TO supplyflow;
GRANT ALL ON SCHEMA public TO supplyflow;
```

Ini cuplikan SQL dari script shell. `:'app_password'` adalah substitusi variabel `psql` dengan quoting literal, bukan password yang ditulis manual di source. Role aplikasi mempunyai database lokal tetapi tidak diberi hak superuser, membuat role, atau membuat database lain.

Start.ps1 menjalankan urutan berikut:

1. Memanggil `Initialize-LocalConfiguration`.
2. Menarik image kecuali `-SkipPull` dipilih.
3. Menyalakan PostgreSQL dan menunggu healthcheck.
4. Menghentikan Odoo proyek ini sebelum proses instalasi satu kali.
5. Memanggil `manage.py initialize` untuk memasang addon bila belum terpasang.
6. Memanggil seed hanya bila `-WithTrainingData` diberikan.
7. Menyalakan service Odoo dan menunggu sehat.

manage.py adalah pengatur proses, bukan server HTTP baru. `main()` membaca mode `initialize`, `update`, atau `seed`. Mode initialize memeriksa tabel modul melalui koneksi PostgreSQL, kemudian memanggil Odoo CLI jika addon belum installed. `run_shell(filename)` membaca script container dan mengirimkannya ke `odoo shell` melalui stdin.

configure_instance.py berjalan pada shell itu, sehingga variabel `env` sudah tersedia. Pada setup pertama, ia membaca secret admin, memberi kelompok Approver, menetapkan mata uang IDR dan URL lokal, lalu mencatat `supplyflow.initialized`. Pada startup berikutnya, penanda itu mencegah password dan data bisnis ditulis ulang.

`Stop.ps1` hanya menghentikan dua service proyek ini. Ia tidak menjalankan penghapusan volume.

1.6 Folder kosong untuk jalur membangun ulang

Lewati bagian ini jika hanya membaca proyek utama. Untuk praktik dari folder kosong, buka satu terminal PowerShell khusus latihan dan jalankan:

powershell

```
$referensi = 'D:\kerjaan\done push repo\supplyflow-odoo'
$latihan = 'D:\Latihan-Odoo\supplyflow-office'
if (Test-Path -LiteralPath $latihan) {
    throw 'Folder latihan sudah ada. Gunakan folder kosong lain; jangan timpa pekerjaan lama.'
}
New-Item -ItemType Directory -Path $latihan | Out-Null
Copy-Item -LiteralPath "$referensi\compose.yaml" -Destination "$latihan\compose.yaml"
Copy-Item -LiteralPath "$referensi\compose.test.yaml" -Destination "$latihan\compose.test.yaml"
Copy-Item -LiteralPath "$referensi\.gitignore" -Destination "$latihan\.gitignore"
Copy-Item -LiteralPath "$referensi\.gitattributes" -Destination "$latihan\.gitattributes"
New-Item -ItemType Directory -Path "$latihan\scripts\container" -Force | Out-Null
foreach ($sourceFile in (Get-ChildItem -LiteralPath "$referensi\scripts" -Filter '*.ps1' -File)) {
    Copy-Item -LiteralPath $sourceFile.FullName -Destination "$latihan\scripts"
}
foreach ($sourceFile in (Get-ChildItem -LiteralPath "$referensi\scripts\container" -File)) {
```

```
    if ($sourceFile.Extension -in @('.py', '.sh')) {
        Copy-Item -LiteralPath $sourceFile.FullName -Destination "$latihan\scripts\container"
    }
}
Set-Location -LiteralPath $latihan
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
```

Yang disalin hanya infrastruktur source agar kamu dapat berfokus membuat addon. Pemilihan file tidak menyalin `__pycache__`. Tidak ada data, secret, atau dependency terpasang yang disalin. Seluruh helper tersebut dijelaskan pada bab ini dan bab 7; kamu dapat membuka file lengkapnya jika ingin mengetik ulang.

`COMPOSE_PROJECT_NAME` wajib tetap bernilai `supplyflow-belajar` pada terminal latihan. Ini memisahkan nama container dan volume dari proyek utama. Port tetap 8070, jadi hentikan proyek utama lebih dahulu jika masih berjalan. Pada terminal baru untuk latihan, set variabel itu lagi sebelum perintah Compose atau script proyek.

Belum jalankan `Start.ps1` di folder latihan sampai kerangka addon bab 2 selesai. Jangan memuat data latihan sebelum addon lengkap pada bab 5.

Checkpoint

Kamu dapat menjawab: browser mengakses port mana, Odoo menghubungi host database apa, source Python dipasang ke folder container mana, dan data apa yang tidak termasuk saat repo di-push. Jika salah satunya belum jelas, cocokkan kembali nilai pada `compose.yaml` dengan tabel istilah di atas.

2. Kerangka addon dan hak akses

Hasil bab ini: addon minimal terpasang, mempunyai model permintaan, dan membuka daftar kosong dari menu. Belum ada pengajuan, persetujuan, RFQ, baris produk, atau retur. Tujuan tahap ini adalah memahami cara Odoo menemukan Python dan XML sebelum menambahkan aturan bisnis.

Semua contoh berlabel **starter** pada bab ini hanya untuk folder latihan bab 1. Jangan menimpa addon lengkap di folder proyek utama. Kode starter berikut adalah isi lengkap setiap file yang disebutkan.

2.1 Susun folder addon

Di `D:\Latihan-Odoo\supplyflow-office`, buat struktur berikut:

text

```
addons/
  supplyflow_office/
    __init__.py
    __manifest__.py
    models/
      __init__.py
      purchase_request.py
    security/
      groups.xml
      ir.model.access.csv
      company_rules.xml
    views/
      purchase_request_views.xml
      menus.xml
```

Nama teknis folder `supplyflow_office` harus sama dengan nama yang dipasang oleh `manage.py`. Nama yang tampil di layar dapat berbeda, tetapi namespace XML dan nama modul harus konsisten.

2.2 Daftarkan addon melalui manifest

Isi starter `addons/supplyflow_office/__manifest__.py`:

python

```
{
    "name": "SupplyFlow Office",
    "summary": "Kerangka latihan permintaan perlengkapan kantor",
    "version": "19.0.1.0.0",
    "license": "LGPL-3",
    "depends": ["purchase_stock", "sale_stock", "mail"],
    "data": [
        "security/groups.xml",
        "security/ir.model.access.csv",
        "security/company_rules.xml",
        "views/purchase_request_views.xml",
        "views/menus.xml",
    ],
    "application": True,
    "installable": True,
}
```

Manifest adalah dictionary Python yang dibaca loader Odoo. `depends` meminta modul dasar dipasang lebih dahulu. `data` menentukan urutan XML/CSV: grup sebelum ACL yang menunjuk grup, view sebelum menu yang menunjuk action. Pada tahap ini belum ada `assets`, report, atau file kasus pemasok karena file tersebut belum dibuat.

Bandingkan dengan manifest final setelah seluruh bab selesai. Jangan menggunakan daftar file final sementara file yang dirujuk belum ada.

2.3 Hubungkan import Python

Isi lengkap starter `addons/supplyflow_office/__init__.py`:

python

```
from . import models
```

Isi lengkap starter `addons/supplyflow_office/models/__init__.py`:

python

```
from . import purchase_request
```

Titik pada `from .` berarti import relatif di dalam package. File Python yang tidak diimpor tidak otomatis menjadi model aktif hanya karena disimpan di folder `models`.

Rantai pemuatannya:

text

```
Odoo membaca __manifest__.py
  -> mengimpor package supplyflow_office
  -> __init__.py mengimpor models
  -> models/__init__.py mengimpor purchase_request
  -> class model didaftarkan
  -> loader membaca data XML/CSV sesuai manifest
```

2.4 Tulis model pertama

Isi lengkap starter `addons/supplyflow_office/models/purchase_request.py`:

python

```
from odoo import fields, models


class SupplyFlowRequest(models.Model):
    _name = "supplyflow.request"
    _description = "SupplyFlow learning request"
    _check_company_auto = True

    name = fields.Char(required=True, default="New")
    company_id = fields.Many2one(
        "res.company", required=True,
        default=lambda self: self.env.company,
    )
    purpose = fields.Text(string="Purpose")
```

`class` mengelompokkan definisi model. `models.Model` memberi perilaku ORM Odoo. `_name` adalah nama teknis model yang dipakai pada XML, ACL, dan `env["supplyflow.request"]`. Secara bawaan, ORM memetakan model ini ke tabel `supplyflow_request`.

`fields.Char` menampung teks pendek. `fields.Text` menampung uraian. `fields.Many2one("res.company")` menghubungkan satu permintaan ke satu perusahaan. Database menyimpan referensi record, bukan menyalin seluruh perusahaan ke setiap permintaan.

`self` adalah recordset tempat method berjalan. `self.env` menyediakan model lain, pengguna, context, perusahaan, dan cursor transaksi. `lambda self: self.env.company` dievaluasi sebagai default ketika record dibuat, sehingga tidak mengunci perusahaan tertentu pada waktu file Python diimpor.

`_check_company_auto` membantu pemeriksaan relasi yang memakai `check_company=True`. Ia bukan pengganti record rule atau pemeriksaan perusahaan pada aturan bisnis.

Pada checkpoint kerangka ini, kita belum membuat transaksi. Ini mencegah record percobaan yang belum mempunyai field wajib final mengganggu upgrade tahap berikutnya.

2.5 Buat kelompok pengguna

Isi lengkap starter `security/groups.xml`, sama dengan source final:

xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id="privilege_supplyflow" model="res.groups.privilege">
        <field name="name">SupplyFlow Office</field>
        <field name="category_id" ref="base.module_category_supply_chain"/>
        <field name="sequence">25</field>
    </record>
    <record id="group_user" model="res.groups">
        <field name="name">Requester</field>
        <field name="privilege_id" ref="privilege_supplyflow"/>
        <field name="sequence">10</field>
        <field name="implied_ids" eval="[(4, ref('base.group_user'))]"/>
    </record>
    <record id="group_approver" model="res.groups">
        <field name="name">Approver</field>
        <field name="privilege_id" ref="privilege_supplyflow"/>
        <field name="sequence">20</field>
        <field name="implied_ids" eval="[(4, ref('group_user'))]"/>
        <field name="user_ids" eval="[(4, ref('base.user_admin'))]"/>
    </record>
</odoo>
```

`record` membuat atau memperbarui record konfigurasi. `id="group_user"` membentuk external ID `supplyflow_office.group_user`, yang stabil untuk rujukan antarfile. `ref` mencari external ID lain, bukan angka ID database yang dapat berbeda antarinstalasi.

`implied_ids` berarti anggota Approver juga mendapat Requester. Command `(4, id)` menambahkan relasi tanpa mengganti semua relasi yang sudah ada. Ini sama keluarga konsepnya dengan `Command.link(id)` pada kode Python.

`group_approver` sudah disediakan pada starter karena script konfigurasi pertama mencari external ID tersebut. Izin grup native Purchase dan Inventory tetap merupakan pengaturan terpisah.

2.6 ACL: boleh melakukan operasi apa?

Isi lengkap starter `security/ir.model.access.csv`:

csv

```csv
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_request_user,SupplyFlow request,model_supplyflow_request,group_user,1,0,0,0
```

Baris ini memberikan **read saja** untuk anggota Requester. Hak create/write/unlink sengaja belum dibuka pada kerangka. Administrator tetap memiliki kemampuan administratif lebih luas, sehingga view juga menyembunyikan

pembuatan record pada checkpoint ini.

`model_supplyflow_request` adalah external ID model yang dibentuk Odoo dari `_name`. Empat kolom terakhir berarti baca, ubah, buat, dan hapus. Nilai ACL bersifat pemberian hak; angka nol pada satu baris tidak mencabut hak yang diberikan grup lain.

ACL final mempunyai empat model dan memberikan operasi kepada anggota addon. Pembatasan status/pemilik dilakukan lagi di method Python. Jangan menyalin ACL final empat baris sebelum model baris dan kasus terkait sudah terdaftar.

2.7 Record rule: record mana yang boleh terlihat?

Isi lengkap starter `security/company_rules.xml`:

xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id="request_company_rule" model="ir.rule">
        <field name="name">SupplyFlow request company isolation</field>
        <field name="model_id" ref="model_supplyflow_request"/>
        <field name="domain_force">[('company_id', 'in', company_ids)]</field>
    </record>
</odoo>
```

Domain adalah daftar kondisi pencarian. `company_ids` pada evaluasi rule mengikuti perusahaan aktif yang diizinkan pada environment pengguna. Rule ini global karena tidak dibatasi ke grup tertentu. Pada aplikasi final, rule setara ditambahkan untuk request line, issue, dan issue line.

ACL memperbolehkan membaca model. Rule membatasi record yang memenuhi domain. Method bisnis kemudian memeriksa pemilik, status, dan peran. Contohnya, boleh membaca permintaan satu perusahaan tidak berarti boleh menyetujui permintaan sendiri.

2.8 Buat daftar, form, dan action

Isi lengkap starter `views/purchase_request_views.xml`:

xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id="view_request_list" model="ir.ui.view">
        <field name="name">supplyflow.request.list</field>
        <field name="model">supplyflow.request</field>
        <field name="arch" type="xml">
            <list create="false" edit="false" delete="false">
                <field name="name"/>
                <field name="company_id"/>
                <field name="purpose"/>
            </list>
        </field>
    </record>
    <record id="view_request_form" model="ir.ui.view">
        <field name="name">supplyflow.request.form</field>
        <field name="model">supplyflow.request</field>
        <field name="arch" type="xml">
            <form create="false" edit="false" delete="false">
                <sheet>
                    <group>
                        <field name="name"/>
                        <field name="company_id"/>
                        <field name="purpose"/>
                    </group>
```

```
                </sheet>
            </form>
        </field>
    </record>
    <record id="action_requests" model="ir.actions.act_window">
        <field name="name">Purchase Requisitions</field>
        <field name="res_model">supplyflow.request</field>
        <field name="view_mode">list,form</field>
    </record>
</odoo>
```

View menjelaskan cara menampilkan model. `<list>` menampilkan banyak record dan `<form>` satu record. `field name="purpose"` harus cocok dengan field Python. Action menentukan model dan jenis view yang dibuka; ia belum menjadi menu sampai dihubungkan pada file berikutnya.

`create="false"` membantu menjaga checkpoint tetap kosong. Pembatasan UI ini bukan mekanisme keamanan server; karena itu ACL starter juga belum memberikan create.

2.9 Hubungkan menu ke action

Isi lengkap starter `views/menus.xml`:

xml

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <menuitem id="menu_supplyflow" name="SupplyFlow Office"
              sequence="10" groups="supplyflow_office.group_user"/>
    <menuitem id="menu_requests" name="Purchase requests"
              parent="menu_supplyflow" action="action_requests"
              sequence="20"/>
</odoo>
```

`parent` menempatkan submenu di bawah aplikasi. `action` menghubungkan klik ke window action yang sudah dibuat. ID starter mengikuti ID final untuk menu root dan permintaan, sehingga pembaruan tahap final tidak menciptakan menu duplikat dengan ID berbeda.

2.10 Jalankan checkpoint kerangka

Di terminal khusus latihan, pastikan proyek utama tidak sedang memakai port 8070. Jalankan tanpa data latihan:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Start.ps1
```

Hasil yang diharapkan: Odoo memasang modul, login admin dari secret lokal bekerja, dan aplikasi SupplyFlow Office membuka daftar permintaan yang masih kosong. Belum ada tombol workflow karena view starter tidak mendefinisikannya.

Setelah mengecek menu, hentikan layanan:

powershell

```
Set-Location -LiteralPath 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Stop.ps1
```

Jangan gunakan `-WithTrainingData` pada starter. Dataset final mencari field dan model yang baru dijelaskan pada bab berikutnya.

2.11 Mengganti starter dengan kode lengkap

Bab 3 dan 4 membahas model lengkap fungsi demi fungsi. Di folder latihan, ganti seluruh `purchase_request.py` starter dengan source final; tambahkan `supplier_issue.py` dan `workspace.py`. Jangan menambahkan class kedua di bawah class starter.

Sesudah mengganti model, tunggu bab 5 untuk memasang seluruh file XML, ACL, report, asset, import, dan manifest final secara bersama. Jangan menyalakan server di tengah kombinasi model final dan view/ACL starter yang belum lengkap.

Tiga hal yang perlu kamu kuasai sebelum lanjut: mengapa file perlu diimpor, bagaimana menu menemukan action, serta perbedaan ACL, record rule, dan pemeriksaan method. Gunakan nama `supplyflow.request` untuk mengikuti rujukannya di setiap file.

03. Membuat model permintaan pembelian dan RFQ

Bab ini mengikuti file models/purchase_request.py pada SupplyFlow Office. Hasilnya adalah dokumen permintaan pembelian yang diajukan oleh satu pengguna, ditinjau pengguna lain, lalu menghasilkan RFQ standar Odoo.

Kamu akan membaca kode dalam urutan pembuatannya: bentuk data, aturan angka, aturan pemilik, penyimpanan, perpindahan status, dan hubungan ke Purchase. Semua cuplikan Python di bab ini diambil dari source proyek. Cuplikan berlabel fungsi lengkap tetap merupakan bagian kelas, bukan file mandiri yang bisa langsung dijalankan dengan `python purchase_request.py`.

Untuk latihan yang dapat dijalankan, gunakan file lengkap dari repo beserta manifest, import, ACL, aturan perusahaan, sequence, dan XML view. Jangan mengganti file lengkap dengan gabungan beberapa cuplikan pilihan. Penjagaan akses pada implementasi ini saling bergantung.

Jika mengikuti pembangunan di folder kosong dari bab 02, **ganti seluruh isi file model minimal** dengan file final yang dijelaskan di bab ini. Jangan menambahkan kelas `SupplyFlowRequest` kedua di bawah kelas percobaan. Bab 03 dan 04 menyusun model final; instalasi lengkap pertama pada folder latihan dilakukan di bab 05, setelah manifest, keamanan, view, report, dan aset final tersedia bersama. Latihan UI pada akhir bab ini dijalankan di salinan proyek referensi yang sudah lengkap.

1. Tentukan dokumen yang hendak dibuat

Misalnya staf distributor ingin membeli kertas untuk memenuhi pesanan kantor pelanggan. Pengaju mencatat pemasok, gudang tujuan, tanggal kebutuhan, alasan, dan daftar barang. Approver menilai kebutuhan tersebut sebelum membuat RFQ.

Terdapat dua model buatan sendiri dalam satu file:

Kelas Python	Nama model Odoo	Isi
`SupplyFlowRequest`	`supplyflow.request`	Satu permintaan: siapa yang mengajukan, pemasok, gudang, status, keputusan, RFQ terkait
`SupplyFlowRequestLine`	`supplyflow.request.line`	Satu jenis barang pada permintaan, jumlah, satuan, estimasi harga, subtotal

Permintaan berisi banyak baris barang. Setiap baris menunjuk satu permintaan. Ketika RFQ dibuat, addon membuat record `purchase.order` dan `purchase.order.line` milik Odoo. Kita memakai proses pembelian yang sudah tersedia.

Istilah yang dipakai di bawah:

Istilah	Arti pada kode ini
Model	Definisi jenis data beserta aturan dan perilakunya
Record	Satu dokumen atau satu baris barang yang sudah dibuat
Recordset	Kumpulan record; `self` bisa berisi satu atau beberapa record
ORM	Cara model Python membaca dan menyimpan data melalui Odoo

Istilah	Arti pada kode ini
`self.env`	Lingkungan akses model, pengguna, perusahaan, dan context saat ini
`self.env.uid` / `self.env.user`	ID pengguna / record pengguna yang sedang melakukan operasi
`values`	Dictionary nilai field yang akan disimpan
Context	Nilai tambahan untuk operasi, misalnya perusahaan aktif atau default field; masukan context perlu diperiksa
`super()`	Melanjutkan operasi ke implementasi induk Odoo setelah pemeriksaan addon selesai

2. Buat identitas kelas dan field dokumen

Lokasi: `addons/supplyflow_office/models/purchase_request.py`, kelas `SupplyFlowRequest`. Letakkan definisi field setelah identitas kelas dan sebelum kumpulan method. Berikut bagian awal kelas yang lengkap sampai constraint referensinya; ini cuplikan dari file, bukan seluruh kelas.

python

```
class SupplyFlowRequest(models.Model):
    _name = "supplyflow.request"
    _description = "Purchase Requisition"
    _inherit = ["mail.thread", "mail.activity.mixin"]
    _order = "id desc"
    _check_company_auto = True

    name = fields.Char(required=True, readonly=True, copy=False, default="New", index=True)
    company_id = fields.Many2one(
        "res.company", required=True, default=lambda self: self.env.company, index=True
    )
    currency_id = fields.Many2one(related="company_id.currency_id", store=True)
    requester_id = fields.Many2one(
        "res.users", required=True, readonly=True, copy=False,
        default=lambda self: self.env.user, check_company=True, tracking=True,
    )
    vendor_id = fields.Many2one("res.partner", required=True, check_company=True, tracking=True)
    warehouse_id = fields.Many2one(
        "stock.warehouse", required=True, check_company=True, tracking=True,
        default=lambda self: self.env["stock.warehouse"].search(
            [("company_id", "=", self.env.company.id)], limit=1
        ),
    )
    requested_date = fields.Date(required=True, default=fields.Date.context_today, tracking=True)
    purpose = fields.Text(required=True, tracking=True)
    line_ids = fields.One2many("supplyflow.request.line", "request_id", string="Items", copy=True)
    amount_estimated = fields.Monetary(
        string="Estimated Total", compute="_compute_amount_estimated", store=True,
        help="Estimated amount before taxes, in the company's currency.",
    )
    state = fields.Selection(
        [
            ("draft", "Draft"), ("submitted", "Awaiting Approval"),
            ("approved", "Approved"), ("rejected", "Rejected"),
            ("rfq_created", "RFQ Created"), ("cancelled", "Cancelled"),
        ], default="draft", required=True, readonly=True, copy=False, index=True, tracking=True,
    )
    decision_reason = fields.Text(string="Decision Note", copy=False, tracking=True)
    submitted_by_id = fields.Many2one("res.users", readonly=True, copy=False, check_company=True)
    submitted_at = fields.Datetime(readonly=True, copy=False)
    approved_by_id = fields.Many2one("res.users", readonly=True, copy=False, check_company=True)
    approved_at = fields.Datetime(readonly=True, copy=False)
    rejected_by_id = fields.Many2one("res.users", readonly=True, copy=False, check_company=True)
    rejected_at = fields.Datetime(readonly=True, copy=False)
```

```
    purchase_order_id = fields.Many2one(
        "purchase.order", string="Request for Quotation", readonly=True, copy=False,
        check_company=True, ondelete="restrict", index=True,
    )
    _unique_rfq = models.Constraint(
        "UNIQUE(purchase_order_id)", "An RFQ can only belong to one requisition."
    )
    _unique_reference = models.Constraint(
        "UNIQUE(company_id, name)", "The requisition reference must be unique within its company."
    )
```

`_name` memberi nama model yang dipakai XML, ACL, dan `self.env["supplyflow.request"]`. `_inherit` menambahkan chatter dan aktivitas. Karena mewarisi `models.Model`, kelas ini memiliki operasi seperti `create`, `write`, `search`, dan `unlink`.

Beberapa pilihan field menentukan hubungan data:

- `vendor_id` adalah `Many2one` ke `res.partner`: satu permintaan memilih satu pemasok.
- `warehouse_id` menunjuk `stock.warehouse`. Default mencari gudang pada perusahaan aktif, bukan mengambil gudang sembarang perusahaan.
- `line_ids` adalah `One2many` dengan pasangan `request_id` pada model baris.
- `currency_id` mengambil mata uang dari perusahaan. Estimasi di bab ini menggunakan mata uang perusahaan dan belum termasuk pajak.
- `purchase_order_id` menyimpan RFQ hasil konversi. `ondelete="restrict"` menjaga agar RFQ yang dirujuk tidak mudah dihapus sehingga tautannya putus.
- `copy=False` mencegah nilai seperti nomor, status, dan jejak persetujuan ikut tersalin melalui mekanisme salin field Odoo.

`readonly=True` membantu tampilan form, tetapi server tetap memeriksa field yang dilindungi di `create()` dan `write()`. Pengguna tidak cukup dibatasi dengan menyembunyikan input atau tombol.

Dua `models.Constraint` menambah penjagaan database: satu RFQ tidak boleh dipakai dua permintaan dan nomor permintaan harus unik dalam perusahaan. Constraint tersebut melengkapi pemeriksaan Python.

Siapkan identitas model di paket addon

models/__init__.py memuat `purchase_request`. File import itu membuat Odoo membaca kelas Python saat addon dimuat. Definisi sequence berada di data/sequences.xml. Nama kode sequence yang dicari nanti adalah `supplyflow.request`.

Jangan menamai file sembarang lalu lupa mengimpornya. XML yang merujuk `supplyflow.request` memerlukan model tersebut sudah terdaftar.

3. Buat bentuk baris dan perhitungan estimasi

Lokasi: file yang sama, kelas `SupplyFlowRequestLine`, setelah kelas permintaan. Bagian berikut memuat identitas, field, dan constraint baris; ini cuplikan kelas.

python

```
class SupplyFlowRequestLine(models.Model):
    _name = "supplyflow.request.line"
    _description = "Purchase Requisition Item"
    _order = "id"
    _check_company_auto = True

    request_id = fields.Many2one("supplyflow.request", required=True, ondelete="cascade", index=True, check_company=
  >True)
```

```
    company_id = fields.Many2one(related="request_id.company_id", store=True, index=True)
    currency_id = fields.Many2one(related="request_id.currency_id")
    product_id = fields.Many2one("product.product", required=True, check_company=True, ondelete="restrict")
    name = fields.Char(string="Description", required=True, compute="_compute_name", store=True, readonly=False, pre
>compute=True)
    product_uom_id = fields.Many2one("uom.uom", string="Unit", compute="_compute_product_uom_id", store=True)
    product_qty = fields.Float(string="Quantity", default=1, required=True, digits="Product Unit")
    price_estimated = fields.Monetary(string="Estimated Unit Price", required=True, default=0)
    subtotal = fields.Monetary(compute="_compute_subtotal", store=True)

    _positive_quantity = models.Constraint("CHECK(product_qty > 0)", "Item quantity must be greater than zero.")
    _nonnegative_price = models.Constraint("CHECK(price_estimated >= 0)", "Estimated price cannot be negative.")
```

`request_id` adalah pasangan dari `line_ids`. Dengan `ondelete="cascade"`, baris ikut dihapus ketika permintaan induk boleh dihapus. Izin menghapus induk tetap diperiksa di `unlink()` milik permintaan.

`name` mengambil nama produk sebagai awal dan dapat diedit sebagai deskripsi. `product_uom_id` dihitung dari satuan produk, sehingga pengguna tidak memasukkan satuan berbeda melalui field manual. Perusahaan dan mata uang baris mengikuti induknya.

Lokasi: dalam `SupplyFlowRequestLine`, setelah deklarasi field. Tiga fungsi berikut lengkap sebagaimana source.

python

```
    @api.depends("product_id")
    def _compute_name(self):
        for line in self:
            line.name = line.product_id.display_name
```

python

```
    @api.depends("product_id")
    def _compute_product_uom_id(self):
        for line in self:
            line.product_uom_id = line.product_id.uom_id
```

python

```
    @api.depends("product_qty", "price_estimated")
    def _compute_subtotal(self):
        for line in self:
            line.subtotal = line.product_qty * line.price_estimated
```

`@api.depends` menyatakan field pemicu perhitungan ulang. Ketika jumlah atau estimasi harga berubah, subtotal dihitung ulang. `store=True` pada field hasil membuat nilai hasilnya tersimpan.

Contoh angka latihan, bukan data bisnis nyata: 12 unit dengan estimasi harga 8,5 menghasilkan subtotal 102. Harga nol diperbolehkan karena pengguna mungkin belum mengetahui penawaran pemasok. Jumlah nol dan harga negatif ditolak.

Lokasi: dalam `SupplyFlowRequest`, setelah deklarasi field dan konstanta. Fungsi berikut menjumlahkan subtotal semua baris.

python

```
    @api.depends("line_ids.subtotal")
    def _compute_amount_estimated(self):
        for request in self:
            total = sum(request.line_ids.mapped("subtotal"))
            try:
                rounded = request.currency_id.round(total)
            except (ValueError, OverflowError) as error:
                raise ValidationError(_("The estimated total is outside the currency's supported numeric range.")) f
>rom error
            if not math.isfinite(rounded):
```

```
                raise ValidationError(_("The estimated total must be finite."))
            request.amount_estimated = total
```

`mapped("subtotal")` mengambil nilai subtotal dari kumpulan baris. `currency_id.round(total)` dipakai untuk memastikan nilai dapat ditangani aturan mata uang. Bila penjumlahan terlalu besar atau menghasilkan angka tidak terhingga, model mengembalikan `ValidationError`.

Nilai `amount_estimated` yang disimpan tetap `total`, sesuai source. Jangan menyebutnya saldo anggaran, total faktur, atau realisasi pembayaran. Nilai ini hanya estimasi permintaan sebelum pajak.

Validasi angka sebelum Odoo mengonversi field

Lokasi: `SupplyFlowRequestLine`, fungsi lengkap `_validate_numeric_input()`. Fungsi ini dipanggil oleh `create()` dan `write()` baris sebelum penyimpanan.

python

```
    @api.model
    def _validate_numeric_input(self, quantity, price, unit, currency):
        try:
            quantity, price = float(quantity), float(price)
        except (TypeError, ValueError, OverflowError) as error:
            raise ValidationError(_("Enter numeric quantities and estimated prices.")) from error
        if not all(math.isfinite(value) for value in (quantity, price, quantity * price)):
            raise ValidationError(_("Quantity and estimated price must be finite numbers."))
        if quantity <= 0:
            raise ValidationError(_("Item quantity must be greater than zero."))
        if price < 0:
            raise ValidationError(_("Estimated price cannot be negative."))
        if unit:
            try:
                is_positive = unit.compare(quantity, 0) > 0
            except (ValueError, OverflowError) as error:
                raise ValidationError(_("The quantity is outside the supported numeric range.")) from error
            if not is_positive:
                raise ValidationError(_("Item quantity must be greater than zero at the product's unit precision."))
        try:
            monetary_values = [currency.round(value) for value in (price, quantity * price)]
        except (ValueError, OverflowError) as error:
            raise ValidationError(_("The estimated price or subtotal is outside the currency's supported numeric ran
>ge.")) from error
        if not all(math.isfinite(value) for value in monetary_values):
            raise ValidationError(_("Estimated price and subtotal must be finite."))
        return quantity, price
```

Urutan pemeriksaannya mempunyai tujuan berbeda:

1. Ubah masukan menjadi angka. Teks yang tidak bisa dikonversi mendapat pesan validasi.
2. Tolak `NaN`, tak terhingga, atau hasil perkalian yang tidak terhingga.
3. Pastikan jumlah positif dan harga tidak negatif.
4. Periksa jumlah memakai ketelitian satuan produk. Angka positif yang terlalu kecil hingga terbaca nol pada satuan produk tetap ditolak.
5. Pastikan harga dan subtotal dapat dibulatkan oleh mata uang. Angka yang masih finite dapat terlalu besar untuk operasi pembulatan tersebut.

Pemeriksaan sebelum `super().create()` atau `super().write()` mencegah nilai bermasalah mencapai konversi field atau SQL lebih dahulu.

Fungsi `_check_line_details()` kemudian memeriksa kembali angka, satuan, kelayakan produk, dan perusahaan setelah nilai berada pada record. Produk harus bisa dibeli (`purchase_ok`) dan dilacak stoknya (`is_storable`). Barang milik perusahaan lain tidak dapat dicampur. Constraint database untuk jumlah dan harga memberi lapisan tambahan, tetapi tidak menggantikan pesan validasi Python yang lebih spesifik.

4. Siapkan pemeriksaan bersama sebelum menyimpan dan mengganti status

Lokasi: kelas `SupplyFlowRequest`, sebelum `create()`. Bagian ini merupakan kumpulan fungsi lengkap yang dipakai bersama oleh action dan operasi penyimpanan.

python

```
@api.model
def _require_group(self, group="supplyflow_office.group_user"):
    if not self.env.user.has_group(group):
        raise AccessError(_("You do not have the required SupplyFlow role."))
```

python

```
def _lock_for_change(self):
    self._require_group()
    self.check_access("write")
    self.flush_recordset()
    self.lock_for_update()
    self.invalidate_recordset()
    self.check_access("write")
```

python

```
def _require_requester(self):
    if any(request.requester_id != self.env.user for request in self):
        raise AccessError(_("Only the requester can edit or withdraw this requisition."))
```

python

```
def _require_independent_approver(self):
    self._require_group("supplyflow_office.group_approver")
    if any(
        self.env.user in (request.requester_id | request.create_uid)
        for request in self
    ):
        raise AccessError(_("Another approver must review your own requisition."))
```

python

```
def _require_draft(self):
    self._require_requester()
    if any(request.state != "draft" for request in self):
        raise UserError(_("Items and requisition details can only be edited in Draft."))
```

`_require_group()` memastikan pengguna memiliki peran SupplyFlow yang diperlukan. Hak membaca atau menulis record tetap diperiksa oleh Odoo. Pembatasan perusahaan pada security/company_rules.xml bekerja bersama guard ini.

`_lock_for_change()` memeriksa akses, melakukan flush, mengunci record untuk perubahan, menghapus cache record, lalu memeriksa akses lagi. Action yang akan memutuskan status membaca data yang diperiksa setelah langkah ini. Dua permintaan HTTP yang berdekatan tidak cukup dilindungi dengan membuat tombol terlihat nonaktif di browser.

Penguncian dapat menghasilkan konflik transaksi ketika dua operasi bersaing. Jangan menganggap kedua klik selalu berhasil. Hasil yang hendak dijaga adalah satu konversi yang sah dan tidak ada RFQ kedua untuk permintaan yang sama. Uji HTTP konkurensi dicatat pada VERIFICATION.md.

`_require_independent_approver()` menolak pengguna yang sama dengan `requester_id` maupun `create_uid`. Jadi, pengguna yang punya peran Approver juga tetap perlu orang lain untuk meninjau permintaan buatannya sendiri.

`_require_draft()` menggabungkan dua syarat: pengguna adalah pengaju dan dokumen masih Draft. Pengguna SupplyFlow lain dalam perusahaan aktif dapat membaca sesuai ACL/rule, tetapi tidak mendapat hak mengubah draft milik orang lain dari guard ini.

Bedakan validasi tersimpan dan bantuan pada form

`_check_request_details()` menolak alasan kosong, gudang dari perusahaan berbeda, serta pemasok yang secara khusus dimiliki perusahaan berbeda. Pemasok bersama, yang `company_id`-nya kosong, tetap dapat dipakai.

`_onchange_company_id()` memilih ulang default gudang ketika perusahaan di form berubah. Onchange membantu pengisian form; aturan server tetap diperlukan untuk operasi lewat API. Model ini tidak mengizinkan mengganti perusahaan permintaan tersimpan ke perusahaan lain. Buat permintaan baru jika perusahaan tujuannya berbeda.

5. Tulis `create()` dan `write()` dokumen induk

Lokasi: `SupplyFlowRequest`, setelah kumpulan guard. Sebelum fungsi tersebut, source menentukan field yang dilindungi dan field yang hanya bisa diedit saat Draft. Cuplikan konstanta berikut lengkap.

python

```
_AUDIT_FIELDS = frozenset({
    "name", "requester_id", "state", "purchase_order_id", "currency_id", "amount_estimated",
    "submitted_by_id", "submitted_at", "approved_by_id", "approved_at",
    "rejected_by_id", "rejected_at", "create_uid", "create_date", "write_uid", "write_date",
})
_DRAFT_FIELDS = frozenset({
    "company_id", "vendor_id", "warehouse_id", "requested_date", "purpose", "line_ids",
})
```

`_AUDIT_FIELDS` mencakup status, identitas pengaju, total, jejak keputusan, dan tautan RFQ. `_DRAFT_FIELDS` mencakup isi permintaan yang ditinjau approver. Jika isinya berubah setelah diajukan, keputusan approver bisa mengacu pada data yang berbeda; karena itu field tersebut dibekukan sampai revisi Draft.

Fungsi lengkap `create()` berikut menghasilkan referensi otomatis dan memastikan dokumen baru selalu dimulai sebagai draft milik pengguna saat ini.

Masukan fungsi	Keluaran jika berhasil	Contoh penolakan
Daftar dictionary pemasok, gudang, tanggal, tujuan, dan baris barang	Recordset permintaan baru dengan nomor sequence, pengguna saat ini, dan status Draft	Pengaju palsu, perusahaan tidak aktif, atau status awal Approved

python

```
@api.model_create_multi
def create(self, vals_list):
    self._require_group()
    prepared = []
    for original in vals_list:
        values = dict(original)
        forbidden = self._AUDIT_FIELDS.intersection(values) - {"state", "requester_id", "name"}
        if forbidden or values.get("state", "draft") != "draft":
            raise AccessError(_("Approval, audit and RFQ fields are set by workflow actions."))
        if values.get("requester_id", self.env.uid) != self.env.uid:
            raise AccessError(_("Create requisitions under your own user account."))
        if values.get("name", "New") not in (False, "New"):
            raise AccessError(_("The requisition reference is assigned automatically."))
        company_id = values.get("company_id", self.env.company.id)
```

```
            if company_id not in self.env.companies.ids:
                raise AccessError(_("This company is not in your active companies."))
            company = self.env["res.company"].browse(company_id)
            reference = self.env["ir.sequence"].with_company(company).next_by_code("supplyflow.request")
            if not reference:
                raise UserError(_("The purchase requisition sequence is missing. Update SupplyFlow Office."))
            values.update({
                "name": reference, "company_id": company_id, "requester_id": self.env.uid,
                "state": "draft", "purchase_order_id": False, "decision_reason": False,
                "submitted_by_id": False, "submitted_at": False,
                "approved_by_id": False, "approved_at": False,
                "rejected_by_id": False, "rejected_at": False,
            })
            prepared.append(values)
        safe_context = {
            key: value for key, value in self.env.context.items()
            if key not in {f"default_{field}" for field in self._AUDIT_FIELDS}
        }
        return super(SupplyFlowRequest, self.with_context(safe_context)).create(prepared)
```

`@api.model_create_multi` menerima daftar dictionary agar operasi membuat beberapa record tetap mendapat pemeriksaan yang sama. `dict(original)` membuat salinan nilai masukan sebelum kode menambah atau menormalkannya.

Perhatikan beberapa keputusan di dalamnya:

- Nilai seperti `approved_by_id` ditolak jika dikirim langsung. Status awal selain `draft` juga ditolak.
- Pengaju tidak boleh mengatasnamakan pengguna lain dan nomor tidak boleh ditentukan manual.
- Perusahaan yang dipilih harus berada dalam perusahaan aktif pengguna.
- Nomor didapat dari `ir.sequence` dengan perusahaan yang sesuai. Sequence yang tidak tersedia menghasilkan pesan kesalahan, bukan nomor acak pengganti.
- Jejak persetujuan dan hubungan RFQ dikosongkan secara eksplisit.
- Key context seperti `default_state` atau `default_approved_by_id` dibuang untuk field yang dilindungi sebelum meneruskan create ke Odoo.

Default context juga merupakan masukan. Memeriksa `values` saja belum cukup apabila Odoo masih boleh mengambil default dari context setelahnya.

Fungsi lengkap `write()` berikut mengatur perubahan pada dokumen yang sudah ada.

Masukan fungsi	Keluaran jika berhasil	Contoh penolakan
Recordset dan dictionary perubahan	Nilai tersimpan setelah guard; hasil dari `super().write()`	Pengguna lain mengedit draft atau pengaju mengubah jumlah setelah Submit

python

```
    def write(self, values):
        self._lock_for_change()
        if self._AUDIT_FIELDS.intersection(values):
            raise AccessError(_("Approval, audit and RFQ fields are set by workflow actions."))
        if self._DRAFT_FIELDS.intersection(values):
            self._require_draft()
        if "company_id" in values and any(request.company_id.id != values["company_id"] for request in self):
            raise UserError(_("Create a new requisition to use another company."))
        if "decision_reason" in values:
            self._require_independent_approver()
            if any(request.state != "submitted" for request in self):
                raise UserError(_("Decision notes can only be entered while awaiting approval."))
        return super().write(values)
```

Semua penulisan lewat fungsi ini diawali guard dan lock. Penulisan langsung status, pengaju, field audit, total, atau tautan RFQ ditolak. Perubahan isi memerlukan draft milik pengaju.

`decision_reason` mempunyai aturan tersendiri: hanya approver independen dan hanya selama status `submitted`. Inilah alasan catatan keputusan tidak dimasukkan begitu saja ke daftar field editable Draft.

Untuk `unlink()`, source memanggil `_lock_for_change()` lalu `_require_draft()` sebelum `super().unlink()`. Akibatnya pengguna hanya dapat menghapus draft miliknya. Dokumen yang sudah diajukan perlu diselesaikan lewat workflow.

6. Bangun perpindahan status satu per satu

Urutan utama:

text

```
Draft -> Awaiting Approval -> Approved -> RFQ Created
                         -> Rejected -> Draft (revisi lalu ajukan lagi)
```

`Cancelled` dapat dicapai melalui aturan pembatalan yang dijelaskan di bawah. Menyimpan `state="approved"` lewat `write()` bukan jalur perpindahan status yang didukung.

Lokasi: `SupplyFlowRequest`, setelah operasi CRUD. Fungsi lengkap `_transition()` menyediakan jalur internal yang dipakai action setelah guard dan validasi.

python

```
    def _transition(self, values):
        # Only private, validated actions use this path; RPC context flags cannot unlock it.
        return super().write(values)
```

`super().write(values)` di helper internal ini melewati override `write()` pada kelas yang sama, sehingga action dapat mengisi field audit yang memang ditolak pada penulisan biasa. Karena itu pemanggilnya wajib sudah memeriksa pengguna, status, dan data. Jangan menambahkan flag context publik seperti `skip_approval=True` untuk membuka jalur tersebut.

Ajukan permintaan

Fungsi lengkap `action_submit()` dipanggil tombol **Submit for Approval**.

python

```
    def action_submit(self):
        self._lock_for_change()
        self._require_draft()
        for request in self:
            if not request.line_ids:
                raise UserError(_("Add at least one item before submitting."))
            request._check_request_details()
            request.line_ids._check_line_details()
            request._check_company()
        self._transition({
            "state": "submitted", "submitted_by_id": self.env.uid,
            "submitted_at": fields.Datetime.now(), "decision_reason": False,
        })
        for request in self:
            request.message_post(body=_("Submitted for independent review."))
        return True
```

Kodenya menolak pengaju yang salah, status selain draft, dan permintaan tanpa barang. Detail induk, setiap baris, serta kecocokan perusahaan diperiksa lagi sebelum status berubah. `submitted_by_id` dan `submitted_at` berasal

dari lingkungan server. `message_post()` menambahkan catatan ke chatter.

Setujui oleh pengguna lain

Fungsi lengkap `action_approve()` berikut hanya menerima permintaan yang sedang menunggu persetujuan.

python

```
    def action_approve(self):
        self._lock_for_change()
        self._require_independent_approver()
        if any(request.state != "submitted" for request in self):
            raise UserError(_("Only submitted requisitions can be approved."))
        self._transition({
            "state": "approved", "approved_by_id": self.env.uid,
            "approved_at": fields.Datetime.now(),
        })
        for request in self:
            request.message_post(body=_("Approved. Decision note: %s", request.decision_reason or "No note supplied.
>"))
        return True
```

Setelah disetujui, barang belum dibeli dan stok belum bertambah. Status ini hanya membuat dokumen layak dikonversi menjadi RFQ.

Tolak dengan alasan

Fungsi lengkap `action_reject()` membutuhkan catatan keputusan yang sudah diisi approver.

python

```
    def action_reject(self):
        self._lock_for_change()
        self._require_independent_approver()
        if any(request.state != "submitted" for request in self):
            raise UserError(_("Only submitted requisitions can be rejected."))
        if any(not (request.decision_reason or "").strip() for request in self):
            raise UserError(_("Enter a decision note explaining the rejection."))
        self._transition({
            "state": "rejected", "rejected_by_id": self.env.uid,
            "rejected_at": fields.Datetime.now(),
        })
        for request in self:
            request.message_post(body=_("Rejected: %s", request.decision_reason))
        return True
```

Approver mengisi `decision_reason` melalui `write()` yang tadi dijaga, lalu menjalankan action ini. Spasi saja tetap dianggap kosong. Penyimpanan alasan serta pencatatan penolakannya memakai transaksi operasi Odoo yang sedang berjalan.

Kembalikan ke Draft untuk revisi

Fungsi lengkap `action_reset_draft()` dipakai oleh pengaju setelah penolakan.

python

```
    def action_reset_draft(self):
        self._lock_for_change()
        self._require_requester()
        if any(request.state != "rejected" for request in self):
            raise UserError(_("Only rejected requisitions can return to Draft."))
        self._transition({
            "state": "draft", "decision_reason": False,
            "submitted_by_id": False, "submitted_at": False,
            "approved_by_id": False, "approved_at": False,
```

```
            "rejected_by_id": False, "rejected_at": False,
        })
        for request in self:
            request.message_post(body=_("Returned to Draft for revision; previous decisions remain in the chatter.")
>)
        return True
```

Field keputusan aktif dikosongkan karena data berikutnya akan diajukan sebagai revisi. Pesan keputusan sebelumnya tetap ada pada chatter, sebagaimana diperiksa tes. Mengubah jumlah pada permintaan `rejected` langsung tetap ditolak; pengaju menekan **Revise Draft**, mengedit, lalu mengajukan ulang. Revisi tersebut memerlukan persetujuan baru sebelum RFQ.

Batalkan sesuai status dan pelakunya

`action_cancel()` memakai guard yang sama, kemudian menerapkan aturan berikut:

Status sebelum pembatalan	Pengguna yang boleh membatalkan	Tombol form
`draft`, `submitted`, `rejected`	Pengaju dokumen	Withdraw
`approved`	Approver independen	Cancel Approval
`rfq_created`, `cancelled`	Ditolak oleh action ini	Kelola RFQ yang sudah ada melalui Purchase

Pembatalan persetujuan tidak mensyaratkan alasan tambahan pada implementasi ini. Identitas pelaku dicatat ke chatter. Membatalkan dokumen SupplyFlow setelah RFQ tercipta tidak disediakan, karena RFQ sudah menjadi transaksi native dengan aturan pembatalannya sendiri.

7. Ubah permintaan yang disetujui menjadi RFQ native

Lokasi: `SupplyFlowRequest`, fungsi lengkap `action_create_rfq()`. Bagian ini menghubungkan addon ke aplikasi Purchase.

Masukan fungsi	Keluaran jika berhasil	Contoh penolakan
Satu permintaan yang disetujui dan pengguna dengan peran serta izin Purchase	Satu RFQ native tertaut dan dictionary pembuka form; pemanggilan berikutnya membuka RFQ yang sama	Draft belum disetujui atau pelaku tidak memiliki izin membuat pembelian

python

```
    def action_create_rfq(self):
        self.ensure_one()
        self._lock_for_change()
        self._require_group("supplyflow_office.group_approver")
        if self.purchase_order_id:
            self.purchase_order_id.check_access("read")
            return self._rfq_action()
        if self.state != "approved":
            raise UserError(_("Approve this requisition before creating an RFQ."))
        self._check_request_details()
        self.line_ids._check_line_details()
        self._check_company()
        self.vendor_id.check_access("read")
        self.warehouse_id.check_access("read")
        self.line_ids.product_id.check_access("read")
        order = self.env["purchase.order"].with_company(self.company_id).create({
```

```
            "partner_id": self.vendor_id.id,
            "company_id": self.company_id.id,
            "currency_id": self.currency_id.id,
            "picking_type_id": self.warehouse_id.in_type_id.id,
            "origin": self.name,
            "user_id": self.env.uid,
            "state": "draft",
            "locked": False,
            "order_line": [fields.Command.create({
                "product_id": line.product_id.id,
                "name": line.name,
                "product_uom_id": line.product_uom_id.id,
                "product_qty": line.product_qty,
                "price_unit": line.price_estimated,
                "date_planned": datetime.combine(self.requested_date, time(hour=12)),
            }) for line in self.line_ids],
        })
        order.order_line._compute_tax_id()
        # Native vendor computations can replace input prices, including an intentional zero estimate.
        for requested, purchased in zip(self.line_ids, order.order_line):
            purchased.write({
                "name": requested.name,
                "price_unit": requested.price_estimated,
                "technical_price_unit": requested.price_estimated,
                "date_planned": datetime.combine(self.requested_date, time(hour=12)),
            })
        self._transition({"state": "rfq_created", "purchase_order_id": order.id})
        self.message_post(body=_("RFQ %s created. Review vendor prices and taxes in Purchase before confirmation.",
>order.name))
        return self._rfq_action()
```

Baca fungsi tersebut menurut urutan operasinya:

1. `ensure_one()` menuntut satu permintaan. Konversi ini tidak memproses kumpulan dokumen sekaligus.
2. Perubahan dikunci dan pelaku memerlukan peran Approver. Hak native untuk membuat Purchase Order tetap diperlukan karena kode tidak menggunakan `sudo()` pada pembuatannya.
3. Bila sudah ada `purchase_order_id`, kode memeriksa akses baca RFQ dan membuka dokumen itu. Inilah perilaku idempotent: menjalankan konversi lagi tidak membuat pembelian kedua.
4. Tanpa RFQ yang ada, status wajib `approved`. Data dan hak baca pemasok, gudang, serta produk diperiksa ulang.
5. `purchase.order` dibuat dengan perusahaan, pemasok, mata uang, dan incoming picking type milik gudang permintaan. `origin` menyimpan nomor permintaan agar operator dapat menelusuri sumbernya.
6. Setiap baris menjadi `fields.Command.create(...)` untuk `purchase.order.line`. Pada Odoo 19 yang dipakai proyek, field satuan baris PO adalah `product_uom_id`.
7. Kode meminta kalkulasi pajak native, lalu mengembalikan deskripsi, estimasi harga, technical price, dan tanggal yang bersumber dari permintaan. Perhitungan native pemasok dapat mengganti harga masukan, termasuk estimasi nol, sehingga bagian penulisan ulang ini diuji khusus.
8. Setelah berhasil, permintaan menyimpan tautan RFQ, berganti `rfq_created`, dan menambahkan pesan. Fungsi mengembalikan action untuk membuka form Purchase.

Tanggal rencana pada source dibentuk dari `requested_date` dengan jam 12. Kebijakan ini ada pada contoh proyek; sesuaikan bila kebutuhan bisnis kelak memerlukan waktu penerimaan yang lebih rinci.

Lokasi: setelah `action_create_rfq()`, fungsi lengkap `_rfq_action()` menghasilkan instruksi tampilan ke web client.

python

```
    def _rfq_action(self):
        return {
            "type": "ir.actions.act_window", "name": _("Request for Quotation"),
            "res_model": "purchase.order", "view_mode": "form",
            "res_id": self.purchase_order_id.id, "target": "current",
```

```
        }
```

Dictionary tersebut meminta Odoo membuka form `purchase.order` dengan ID RFQ terkait pada jendela saat ini. Dictionary ini tidak memindahkan barang.

RFQ yang dibuat masih `draft`. Operator harus memeriksa harga, pajak, dan syarat pemasok dalam Purchase, mengonfirmasi pembelian, lalu memproses receipt. Stok bertambah melalui validasi penerimaan barang oleh proses Inventory. Membuat permintaan, menyetujuinya, atau membuat RFQ belum menambah stok.

Batas kebijakan juga perlu dipahami: persetujuan ini mengatur dokumen SupplyFlow. Pengguna yang memiliki hak native Purchase dapat tetap membuat RFQ langsung melalui aplikasi Purchase. Addon ini belum menjadikan persetujuan SupplyFlow sebagai gerbang wajib untuk semua pembelian native.

8. Jaga setiap jalur edit baris barang

Form induk yang readonly belum melindungi model baris dari pemanggilan langsung. Karena itu `SupplyFlowRequestLine` juga memiliki `create()`, `write()`, dan `unlink()` sendiri.

Lokasi: kelas `SupplyFlowRequestLine`, setelah validator angka. Fungsi lengkap `create()` berikut memeriksa induk sebelum menambahkan baris.

python

```
    @api.model_create_multi
    def create(self, vals_list):
        self.env["supplyflow.request"]._require_group()
        defaults = self.default_get(["product_id", "product_qty", "price_estimated"])
        prepared = []
        for original in vals_list:
            values = dict(original)
            if {"company_id", "currency_id", "subtotal", "product_uom_id", "create_uid", "create_date", "write_uid",
> "write_date"}.intersection(values):
                raise AccessError(_("Item company, unit and totals are assigned automatically."))
            request_id = values.get("request_id", self.env.context.get("default_request_id"))
            if not request_id:
                raise ValidationError(_("An item must belong to a requisition."))
            request = self.env["supplyflow.request"].browse(request_id)
            request._lock_for_change()
            request._require_draft()
            values["request_id"] = request_id
            product = self.env["product.product"].browse(values.get("product_id", defaults.get("product_id")))
            values["product_qty"], values["price_estimated"] = self._validate_numeric_input(
                values.get("product_qty", defaults.get("product_qty", 1)),
                values.get("price_estimated", defaults.get("price_estimated", 0)),
                product.uom_id, request.currency_id,
            )
            prepared.append(values)
        protected_defaults = {
            f"default_{field}" for field in (
                "company_id", "currency_id", "subtotal", "product_uom_id",
                "create_uid", "create_date", "write_uid", "write_date",
            )
        }
        safe_context = {key: value for key, value in self.env.context.items() if key not in protected_defaults}
        return super(SupplyFlowRequestLine, self.with_context(safe_context)).create(prepared)
```

Kode mengambil `request_id` dari nilai eksplisit atau default context, memastikan ada induknya, lalu mengunci induk dan memeriksa draft milik pengaju. Satuan, total, perusahaan, serta metadata audit tidak boleh ditentukan langsung oleh pemanggil. Default context untuk field tersebut juga disaring.

`default_get()` mengambil default produk, jumlah, dan harga yang memang dipakai untuk input barang. Default angka tetap melewati `_validate_numeric_input()`; keberadaan kata default tidak membuatnya otomatis dipercaya.

Fungsi lengkap `write()` berikut mencegah baris dipindahkan ke permintaan lain dan memeriksa angka baru terhadap produk serta mata uang yang benar.

python

```
    def write(self, values):
        self.check_access("write")
        requests = self.mapped("request_id")
        requests._lock_for_change()
        requests._require_draft()
        if {"request_id", "company_id", "currency_id", "subtotal", "product_uom_id", "create_uid", "create_date", "w
>rite_uid", "write_date"}.intersection(values):
            raise AccessError(_("Items cannot be moved between requisitions or assigned manual totals or units."))
        for line in self:
            unit = self.env["product.product"].browse(values["product_id"]).uom_id if "product_id" in values else li
>ne.product_uom_id
            line._validate_numeric_input(
                values.get("product_qty", line.product_qty),
                values.get("price_estimated", line.price_estimated), unit, line.currency_id,
            )
        return super().write(values)
```

Jika `product_id` ikut berubah, pemeriksaan menggunakan satuan produk baru. Bila tidak, satuan dari baris saat ini dipakai. Pemeriksaan jumlah dan harga melihat kombinasi nilai yang akan terbentuk, bukan hanya angka yang kebetulan dikirim pada panggilan tersebut.

Fungsi lengkap `unlink()` memeriksa hak hapus baris dan kondisi induknya.

python

```
    def unlink(self):
        self.check_access("unlink")
        requests = self.mapped("request_id")
        requests._lock_for_change()
        requests._require_draft()
        return super().unlink()
```

Ketiga operasi tersebut menutup jalur terpisah: menambah barang ke permintaan yang sudah diajukan, mengubah jumlah melalui model baris langsung, menghapus barang sesudah review, serta memindahkan baris agar lolos aturan induk.

9. Hubungkan fungsi ke form XML

Sumber tampilan: views/purchase_request_views.xml. Cuplikan berikut adalah dua elemen tombol utuh dari bagian `<header>` form; ini bukan seluruh file XML.

xml

```
                    <button name="action_submit" string="Submit for Approval" type="object" class="oe_highlight" inv
>isible="state != 'draft' or requester_id != uid"/>
                    <button name="action_approve" string="Approve" type="object" class="oe_highlight" groups="supply
>flow_office.group_approver" invisible="state != 'submitted' or requester_id == uid"/>
```

`type="object"` menghubungkan klik tombol dengan method model pada field `name`. `groups` membatasi penampilan berdasarkan grup. `invisible` mengatur kapan tombol tampil sesuai status dan pengguna.

Tombol Approve disembunyikan dari pengaju, sedangkan `_require_independent_approver()` di server juga memeriksa pembuat record. Pemeriksaan server tetap menjadi batas penolakan ketika fungsi dipanggil tanpa melalui tombol.

Di bagian Requested Items, XML menampilkan `line_ids` sebagai daftar yang dapat diedit saat draft. Domain produk menyaring produk yang dapat dibeli dan dilacak stoknya. Validasi model tetap diperlukan karena domain

hanyalah pilihan yang disediakan UI.

`action_requests` pada akhir file adalah `ir.actions.act_window` dengan `res_model` yang sama dan `view_mode="list,form"`. Menu mengarahkan pengguna ke action tersebut. Penjelasan view, menu, ACL, dan report secara menyeluruh dilanjutkan pada bab antarmuka dan keamanan tutorial ini.

10. Uji satu alur melalui akun latihan

Gunakan repo lengkap dan database latihan lokal. Hindari memakai data perusahaan nyata saat mencoba penolakan, pembatalan, atau perubahan jumlah.

1. Dari folder proyek, jalankan `./scripts/Start.ps1 -WithTrainingData -SkipPull` bila image sudah tersedia. Untuk mesin yang belum mengunduh image, gunakan `./scripts/Start.ps1 -WithTrainingData`. Buka `http://localhost:8070` dan baca kredensial lokal dari `.secrets/LOCAL-CREDENTIALS.txt`.
2. Masuk sebagai `buyer`. Buka **SupplyFlow Office > Purchase requests**, buat permintaan baru, pilih pemasok dan gudang dengan awalan `[Training]`, lalu isi tujuan pembelian.
3. Tambah produk latihan yang dilacak stok, isi jumlah 12 dan estimasi harga 8,5. Pastikan subtotal dan total menjadi 102 dalam mata uang perusahaan. Angka ini contoh hitungan, bukan rekomendasi harga barang.
4. Simpan, lalu pilih **Submit for Approval**. Pastikan status menjadi Awaiting Approval dan isi barang tidak dapat diedit lagi.
5. Gunakan browser atau profil terpisah untuk masuk sebagai `approver`. Buka permintaan dari buyer. Isi catatan keputusan jika diperlukan, lalu pilih **Approve**.
6. Pilih **Create RFQ**. Pastikan form Purchase terbuka dan pemasok, jumlah, satuan, gudang tujuan, serta harga awal sesuai permintaan. Tinjau harga dan pajak sebelum mengonfirmasi pembelian.
7. Kembali ke permintaan, pilih **Open RFQ**. ID atau nomor RFQ yang dibuka harus tetap sama. Pada tahap ini pembuatan RFQ saja belum mengubah stok.
8. Buat permintaan latihan kedua untuk jalur penolakan. Ajukan sebagai buyer, isi alasan lalu Reject sebagai approver. Buyer menekan **Revise Draft**, mengubah jumlah, dan mengajukan ulang. Periksa bahwa persetujuan baru tetap diperlukan serta alasan sebelumnya terlihat di chatter.

Jangan menggunakan satu akun untuk seluruh latihan persetujuan. Server akan menolak approver yang mencoba menyetujui permintaannya sendiri.

Cocokkan perilaku dengan tes yang sudah dibuat

Sumber: tests/test_purchase_request.py. File ini berisi 17 method tes. Tabel berikut memberi titik baca untuk keputusan utama, bukan hasil menjalankan ulang tes saat membaca tutorial.

Yang hendak dipahami	Method tes
Data permintaan masuk ke RFQ pada perusahaan/gudang yang benar	`test_request_to_native_rfq_uses_company_warehouse_and_estimate`
Klik konversi kedua membuka RFQ yang sama	`test_rfq_conversion_is_idempotent`
Estimasi nol tetap dipertahankan saat vendor mempunyai harga	`test_rfq_preserves_estimate_even_when_vendor_prices_are_configured`
Pengguna tidak boleh menyetujui permintaan sendiri	`test_approval_needs_an_independent_reviewer`

Yang hendak dipahami	Method tes
Penolakan, revisi, chatter, dan persetujuan ulang	`test_reject_and_revision_preserve_chatter_then_require_new_approval`
Manipulasi status/audit/link melalui `write` ditolak	`test_direct_state_audit_and_link_changes_are_blocked`
Default context tidak dapat memalsukan persetujuan	`test_context_defaults_cannot_forge_approval_or_link`
Semua jalur mutasi baris dibekukan setelah submit	`test_submitted_parent_and_every_line_mutation_path_are_frozen`
Peran SupplyFlow tidak menggantikan izin native Purchase	`test_standard_purchase_rights_are_still_required`
Nilai angka bermasalah ditolak sebelum konversi field	`test_numeric_edits_are_validated_before_database_conversion`

Tes lain dalam file itu mencakup perusahaan, produk non-stok, edit draft pengguna lain, langkah status yang tidak sah, pembatalan persetujuan, dan pengguna tanpa peran. Alur penerimaan dan perubahan stok yang sesungguhnya terdapat dalam tests/test_distribution_flow.py.

Untuk menjalankan paket tes ORM proyek dari PowerShell, gunakan `./scripts/Test.ps1 -SkipPull`. Script ini menghentikan layanan aplikasi proyek untuk pengujian, menggunakan database tes terpisah, lalu membersihkan database tes sesuai opsi script. Setelah selesai, nyalakan kembali dengan `./scripts/Start.ps1 -SkipPull` bila perlu. Ikuti OPERATIONS.md untuk detail operasional.

Peta fungsi yang belum dikutip utuh

Seluruh implementasi tetap berada pada purchase_request.py. Lima fungsi berikut dijelaskan di bab ini tanpa menyalin seluruh isinya:

Kelas	Fungsi	Letak saat menyusun file	Tujuan
`SupplyFlowRequest`	`_onchange_company_id`	Setelah perhitungan total	Mengisi ulang pilihan default gudang pada form
`SupplyFlowRequest`	`_check_request_details`	Setelah onchange	Menolak alasan kosong dan relasi lintas perusahaan yang tidak sesuai
`SupplyFlowRequest`	`unlink`	Setelah `write`	Menghapus hanya draft milik pengaju setelah guard dan lock
`SupplyFlowRequest`	`action_cancel`	Setelah aksi revisi dan sebelum konversi RFQ	Menentukan pelaku pembatalan menurut status dan mencatatnya ke chatter
`SupplyFlowRequestLine`	`_check_line_details`	Setelah perhitungan subtotal	Memeriksa angka, ketelitian satuan, kelayakan produk, dan perusahaan pada record

Kesalahan yang sering membuat implementasi berbeda dari tujuan

Kesalahan	Dampak	Periksa bagian ini
Hanya mengunci input atau menyembunyikan tombol di XML	Operasi langsung ke model dapat melewati aturan tampilan	Guard Python serta override `create`, `write`, `unlink`
Memberi semua pengguna `sudo()` saat membuat PO	Izin native Purchase tidak lagi menjadi batas operasi tersebut	`action_create_rfq` dan tes hak native
Menjaga induk tetapi membiarkan model baris bebas diedit	Jumlah barang bisa berubah setelah approver meninjau	Ketiga operasi CRUD pada `SupplyFlowRequestLine`
Memercayai `default_state` atau `default_approved_by_id` dari context	Record baru dapat menerima default yang tidak semestinya	Penyaringan default dan pengisian audit pada `create`
Memanggil `write({"state": "approved"})` dari tombol	Override `write` menolak perubahan audit langsung	Action tervalidasi dan `_transition` internal
Membuat PO tanpa mengecek `purchase_order_id` yang sudah ada	Percobaan berulang dapat menggandakan transaksi	Guard, lock, dan jalur membuka RFQ yang sama
Menganggap harga estimasi sudah merupakan harga pemasok dan pajak final	Operator melewati peninjauan Purchase	Catatan RFQ dan pemeriksaan sebelum konfirmasi
Menganggap RFQ yang tercipta otomatis menambah barang	Jumlah stok yang diharapkan tidak sesuai proses Inventory	Receipt dan validasi transfer native
Menggunakan nama field dari versi Odoo berbeda	Pembuatan PO bisa gagal dengan field tidak dikenal	Source proyek Odoo 19, khususnya `product_uom_id` pada baris PO

Pada akhir bab ini, telusuri satu klik **Create RFQ** dari tombol XML ke `action_create_rfq()`, pembuatan `purchase.order`, penyimpanan `purchase_order_id`, lalu action pembuka form. Jika urutan tersebut sudah jelas, kamu dapat menjelaskan hubungan kode addon dengan proses Purchase tanpa menganggap setiap langkah ERP ditulis ulang sendiri.

04. Membuat kasus penerimaan barang dan retur pemasok

Bab ini menambahkan penanganan barang rusak, barang salah, dan kekurangan jumlah pada receipt yang sudah selesai. Hasilnya adalah dua model baru, `supplyflow.issue` dan `supplyflow.issue.line`, serta satu perluasan kecil pada `stock.move`.

Contoh bisnisnya: gudang telah mencatat penerimaan 10 karton kertas, tetapi 3 karton rusak. Pelapor mencatat kasus, orang lain meninjaunya, lalu addon membuat dokumen retur Odoo sebanyak 3 karton. Petugas gudang memeriksa dan memvalidasi retur itu. Stok berkurang pada validasi gudang, bukan pada persetujuan kasus.

Bab ini mengikuti kode proyek yang sudah ada. Setiap blok berlabel **potongan kode** mengambil bagian dari file tersebut; blok itu bukan isi file lengkap. Saat menulis ulang, masukkan setiap bagian ke kelas yang disebutkan, tanpa menggandakan definisi metode. File sumber lengkap menjadi pembanding terakhir sebelum memperbarui addon.

1. Siapkan file yang akan saling terhubung

Lanjutkan setelah kerangka addon dan rancangan permintaan pembelian sudah terbentuk. Anda memakai Odoo Community 19 yang sama dengan proyek ini. Jika menulis ulang dari folder kosong, kumpulkan seluruh model, security, view, dan asset terlebih dahulu; instalasi lengkap serta data latihan dilakukan pada bab 05. Untuk mencoba UI sambil membaca bab ini, gunakan salinan proyek referensi yang sudah lengkap. Urutan susunan file pada bab ini:

File	Yang ditambahkan	Cara Odoo memakainya
models/supplier_issue.py	`SupplyflowIssue`, `SupplyflowIssueLine`, dan perluasan `StockMove`	ORM mendaftarkan model dan menjalankan aturan bisnis Python
models/__init__.py	Import `supplier_issue`	Membuat file Python tersebut ikut dimuat
data/sequences.xml	Sequence `supplyflow.issue`	Memberikan nomor kasus `ISS/tahun/nomor`
security/ir.model.access.csv	ACL untuk kedua model baru	Memberi anggota addon izin operasi model
security/company_rules.xml	Rule perusahaan untuk kasus dan baris	Membatasi record pada perusahaan aktif yang diizinkan
views/supplier_issue_views.xml	List, form, search, dan window action	Menghubungkan tombol layar dengan metode Python
__manifest__.py	Memuat file security/data/view dalam urutan yang sesuai	Memasang data dan antarmuka addon
tests/test_supplier_issue.py	Tes transaksi kasus dan retur	Memeriksa aturan di server dan hubungan dengan stok native

Path pada tabel relatif terhadap `addons/supplyflow_office`. Seluruh bagian harus terpasang bersama sebelum alur digunakan. Menambahkan tombol lebih dahulu sementara metode/ACL belum tersedia akan menghasilkan error saat instalasi atau saat tombol dipakai.

Pada `models/__init__.py`, baris import yang diperlukan sudah berbentuk berikut. Ini **potongan kode**; pertahankan import model lain yang sudah ada:

python

```
from . import supplier_issue
```

2. Tentukan receipt, move, dan kasus sebagai objek berbeda

`stock.picking` adalah satu dokumen perpindahan, misalnya satu receipt. `stock.move` adalah baris perpindahan suatu produk pada dokumen itu. `stock.quant` menyimpan kuantitas per produk/lokasi dan dimutakhirkan oleh mesin Inventory. Model kasus menunjuk receipt serta move sumber; model kasus tidak mengubah `stock.quant` secara langsung.

Satu receipt dapat berisi beberapa produk. Karena itu, baris kasus memakai `move_id`, bukan hanya `product_id`. Hubungan ke move membuktikan barang mana yang benar-benar diterima, satuannya apa, dan retur mana yang terkait sumber yang sama.

Di bagian atas `models/supplier_issue.py`, masukkan import lalu kelas parent berikut. Ini **potongan kode dari awal file**, sebelum deklarasi field:

python

```
from datetime import timedelta
from math import isfinite

from odoo import _, api, fields, models
from odoo.exceptions import AccessError, UserError, ValidationError
from odoo.tools.float_utils import float_compare, float_round


class SupplyflowIssue(models.Model):
    _name = "supplyflow.issue"
    _description = "Supplier Receipt Issue"
    _inherit = ["mail.thread", "mail.activity.mixin"]
    _order = "create_date desc, id desc"
    _check_company_auto = True
```

`_name` membuat model custom baru. `_inherit` di sini menambahkan chatter dan aktivitas. `_check_company_auto` membantu pemeriksaan hubungan beratribut `check_company=True`. Ketiganya mempunyai tugas berbeda; keamanan metode tetap harus ditulis.

Tambahkan field di dalam kelas tersebut dengan memakai definisi lengkap dari file sumber. Kelompokkan saat membaca atau menulisnya:

Kelompok	Field	Keterangan
Identitas	`name`, `company_id`, `requester_id`	Nomor sequence, perusahaan, dan pelapor dari pengguna yang sedang login
Sumber	`picking_id`	Receipt incoming berstatus done, dengan hubungan perusahaan yang diperiksa
Masalah	`reason`, `description`, `line_ids`	Jenis masalah, uraian wajib, dan barang yang terdampak
Penanggung jawab	`resolver_id`, `resolution_due`	Pengguna internal yang memiliki akses perusahaan dan tenggat, default tujuh hari
Keputusan	`state`, `approved_by_id`, `approved_at`, `resolution_note`	Status kasus, pelaksana/waktu review, dan catatan penyelesaian
Retur tertaut	`return_picking_id`, `return_state`	Dokumen retur native dan status terkaitnya

`resolver_id` adalah penugasan tanggung jawab. Field ini tidak memberi hak Approver dan tidak membuat hanya resolver yang dapat meninjau kasus. Pemeriksaan review memakai kelompok serta identitas pelapor.

Potongan kode untuk field sumber, diletakkan di dalam `SupplyflowIssue`:

python

```
    picking_id = fields.Many2one(
        "stock.picking", string="Received shipment", required=True,
        check_company=True, ondelete="restrict", tracking=True,
        domain="[('state', '=', 'done'), ('picking_type_code', '=', 'incoming'), ('company_id', '=', company_id)]",
    )
```

`domain` mempersempit pilihan pada form. Domain bukan pemeriksaan akhir: pemanggil RPC dapat mengirim ID tanpa memakai pilihan form. Metode `_check_receipt()` di langkah 5 memeriksa sumber lagi pada server.

3. Susun status sebelum menulis tombol

Gunakan nama teknis status berikut, persis seperti nilai `Selection` pada model:

Alur proses (tabel padanan diagram pada Markdown)

Dari	Aksi / kondisi	Ke
draft	action_submit	submitted
submitted	action_approve, barang fisik	approved
approved	action_create_return	return_created
submitted	action_accept_resolution dan catatan	accepted
draft	action_cancel	cancelled
submitted	action_cancel	cancelled
approved	action_cancel oleh Approver	cancelled
return_created	Langkah berikutnya	Petugas gudang memproses picking retur
Petugas gudang memproses picking retur	Langkah berikutnya	return_state menjadi done saat picking divalidasi

Kasus tidak memiliki status `rejected` atau aksi reset ke draft. Setelah pengajuan, isi pokok dan baris dibekukan. Jika kasus keliru dan masih berada pada status yang mengizinkan pembatalan, batalkan lalu buat kasus baru yang benar.

`state` dan `return_state` harus dibedakan. Setelah retur divalidasi, `state` kasus tetap `return_created`, sedangkan `return_state` terkait berubah menjadi done. Jika picking retur dibatalkan dari Inventory, `return_state` menjadi `cancel`; kasus tetap menyimpan hubungan yang sama.

4. Lindungi perubahan sebelum menambahkan aksi review

Di bawah field, masukkan `_protected_fields` dan `_content_fields` dari sumber. Set pertama melindungi nomor, status, identitas pelapor, metadata review, tautan retur, dan field audit ORM. Set kedua menentukan isi yang hanya boleh berubah pada draft milik pelapor.

Lalu tambahkan helper akses berikut di dalam `SupplyflowIssue`. Tabel ini merupakan inventaris seluruh helper parent; baca implementasinya pada file sumber ketika menyusunnya.

Metode	Input	Hasil atau penolakan	Mengapa diperlukan
`_check_member()`	Pengguna environment saat ini	Selesai tanpa nilai khusus, atau `AccessError`	Anggota kelompok SupplyFlow saja yang boleh memakai aksi kasus
`_check_approver()`	Pengguna saat ini	Memeriksa keanggotaan addon lalu kelompok Approver	Memisahkan hak pelapor dan peninjau
`_check_draft_owner()`	Recordset kasus	Memeriksa akses tulis, `draft`, dan `requester_id`	Anggota lain tidak boleh mengubah draft pelapor
`_check_independent_reviewer()`	Kasus dan pengguna saat ini	Memeriksa Approver, menolak jika pengguna adalah pelapor	Pelapor yang juga Approver tetap perlu orang lain untuk review
`_lock()`	Recordset kasus	Memeriksa akses, row lock, membuang cache, lalu memeriksa akses kembali	Membaca status terbaru setelah persaingan transaksi
`_transition(values)`	Dictionary field workflow yang sudah diotorisasi	Hasil `super(...).write(values)`	Jalur internal perubahan status melewati override `write()` yang melarang perubahan langsung

Potongan kode helper lock dari kelas parent:

python

```
    def _lock(self):
        self.check_access("write")
        self.lock_for_update()
        self.invalidate_recordset()
        self.check_access("write")
```

`invalidate_recordset()` membuang nilai cache ORM agar pemeriksaan setelah lock tidak bergantung pada nilai yang dibaca sebelumnya. Lock tidak memberikan hak akses tambahan. Metode private berawalan _ dipakai dari jalur Python internal; browser memakai metode publik `action_*` yang mengulang pemeriksaan bisnis.

Tambahkan override `create`, `write`, dan `unlink` berikut **secara lengkap dari sumber**, masih pada kelas parent:

Metode	Langkah yang harus dipertahankan
`create(vals_list)`	Memeriksa anggota; menolak status/pelapor/metadata audit yang dipalsukan; memberi sequence, pelapor, status draft, dan perusahaan; membersihkan `default_*` untuk field terlindungi; meneruskan pembuatan ke ORM
`write(vals)`	Menolak field terlindungi; mengunci saat isi/catatan berubah; membatasi isi pada draft pelapor; mengizinkan catatan draft oleh pelapor atau catatan submitted oleh reviewer independen; menolak perubahan catatan setelah review selesai
`unlink()`	Mengunci, memeriksa draft milik pelapor, lalu menghapus melalui ORM

`readonly=True` di deklarasi field dan XML membantu form, tetapi tidak cukup untuk menolak `write({"state": "approved"})` lewat RPC. Override tersebut yang menolak perubahan itu. Penghapusan `default_state`,

`default_approved_by_id`, dan default terlindungi lain juga mencegah pemanggil mengisi metadata review melalui context.

ACL tetap memberi anggota addon izin CRUD pada model; metode Python mempersempit operasi menurut pemilik dan status. Record rule perusahaan membatasi record yang boleh diakses. Pengguna dapat membaca kasus perusahaan aktif yang diizinkan untuk koordinasi; membaca kasus bukan berarti dapat mengeditnya.

5. Validasi receipt dan baris di server

Tambahkan `_check_receipt()` dan `_check_lines()` setelah override parent. Keduanya dipanggil ulang oleh aksi workflow.

`_check_receipt()` memiliki decorator `@api.constrains("picking_id", "company_id", "resolver_id", "description")`. Metode memeriksa akses baca receipt, status `done`, jenis `incoming`, perusahaan yang sama, resolver internal yang memiliki perusahaan itu, serta uraian yang tidak kosong/berisi spasi saja. Bila baris sudah ada, semua move harus berasal dari receipt terpilih.

`_check_lines(returning=False)` memanggil pemeriksaan receipt, mewajibkan sedikitnya satu baris, lalu memanggil validasi move pada baris. Jika kasus akan diretur, atau alasannya bukan shortage, jumlah juga dibandingkan dengan sisa yang dapat diretur. `float_compare` memakai pembulatan satuan sumber; jangan menggantinya dengan pembandingan float tanpa ketelitian satuan.

Potongan kode dari `_check_lines()` yang membedakan shortage:

python

```
if returning and issue.reason == "shortage":
    raise UserError(_("Missing goods cannot be returned. Resolve a shortage with a review note."))
if returning or issue.reason != "shortage":
    for line in issue.line_ids:
        remaining = line._remaining_returnable_quantity()
```

Untuk shortage, `affected_qty` mencatat jumlah yang kurang. Jumlah itu tetap harus positif dan memiliki move sumber yang sah, tetapi tidak dibatasi oleh sisa jumlah retur. Barang yang belum datang tidak mempunyai barang fisik untuk dikirim kembali. Contoh: pesanan 20, diterima 12, kekurangan 8. Penerimaan 12 tetap benar, backorder native dapat mengelola 8 yang ditunggu, dan kasus shortage mencatat tindak lanjut pemasok.

Keterbatasan model saat ini: kasus membutuhkan receipt selesai dan move sumber selesai. Kasus ini tidak mencakup keluhan sebelum ada receipt selesai atau SKU yang tidak memiliki move selesai pada receipt terpilih.

6. Tambahkan aksi pengajuan, review, dan pembatalan

Letakkan metode aksi pada `SupplyflowIssue`, setelah helper validasi. Form memanggilnya menggunakan tombol `type="object"`.

Pengajuan

Berikut **satu metode lengkap** dari file sumber; masukkan satu kali pada kelas parent:

python

```
def action_submit(self):
    self.ensure_one()
    self._check_draft_owner()
    self._lock()
    self._check_draft_owner()
    self._check_lines()
```

```
        self._transition({"state": "submitted"})
        self.message_post(body=_("Supplier issue submitted for independent review."))
        return True
```

`ensure_one()` mensyaratkan satu kasus. Pemeriksaan draft diulang setelah lock karena transaksi lain mungkin sudah mengubah status. Hasil `True` berarti aksi selesai; pesan chatter memberi catatan pengajuan, sedangkan pelarangan edit datang dari pemeriksaan Python.

Keputusan

Tambahkan metode berikut dari sumber. Tidak ada parameter formulir tambahan; metode membaca field record yang sudah disimpan dan identitas environment Odoo.

Metode	Syarat	Perubahan	Hasil
`action_approve()`	Reviewer independen, kasus `submitted`, baris layak retur, bukan shortage	`approved`, `approved_by_id`, `approved_at`, pesan chatter	`True`; belum ada picking baru
`action_accept_resolution()`	Reviewer independen, kasus `submitted`, baris valid, `resolution_note` tidak kosong	`accepted`, reviewer/waktu, pesan chatter	`True`; tanpa perpindahan stok
`action_cancel()`	Anggota addon, status draft/submitted/approved, pelapor atau Approver; status approved memerlukan Approver	`cancelled`	`True`; pemanggilan ulang pada cancelled juga `True`

Penyelesaian tanpa retur juga boleh digunakan untuk barang rusak/salah bila ada kesepakatan yang dicatat. Untuk shortage, inilah jalur keputusan yang tersedia. Catatan tidak otomatis membuat kiriman pengganti, refund, credit note, atau jurnal akuntansi.

Pembatalan kasus yang sudah disetujui memerlukan kelompok Approver; kode pembatalan tidak mengulang syarat reviewer berbeda dari pelapor. Pemeriksaan independen berlaku pada `action_approve()` dan `action_accept_resolution()`. Kasus `accepted` atau `return_created` tidak dapat dibatalkan melalui tombol kasus. Picking yang sudah ada dikelola melalui aturan Inventory native.

7. Buat kelas baris di bawah kelas parent

Setelah metode terakhir `SupplyflowIssue`, tulis kelas `SupplyflowIssueLine(models.Model)` dengan `_name = "supplyflow.issue.line"`. Kelas ini masih berada pada file `supplier_issue.py`, tanpa indentasi di bawah kelas parent.

Masukkan field lengkap dari sumber: `issue_id`, perusahaan/sumber terkait, `move_id`, produk/satuan terkait, `affected_qty`, dan `returnable_qty`. `product_id` serta `product_uom_id` berasal dari `move_id`; pengguna tidak boleh memasangkan move kertas dengan identitas produk lain.

Tambahkan constraint database berikut. Ini **potongan kode** pada kelas baris:

python

```
    _unique_issue_move = models.Constraint("UNIQUE(issue_id, move_id)", "Add each received product only once per iss
>ue.")
```

Constraint mencegah move yang sama muncul dua kali dalam satu kasus. Tanpanya, dua baris untuk sumber yang sama dapat membuat pemetaan jumlah pada wizard ambigu.

Tambahkan `_derived_fields` lalu metode berikut dari file lengkap:

Metode	Input dan hasil	Mengapa ada
`_validated_quantity_input(value)`	Mengubah masukan menjadi float positif, finite, dan dapat diproses menurut precision `Product Unit`; mengembalikan nilai atau `ValidationError`	Menolak teks, NaN, infinity, angka terlalu besar, nol, atau negatif sebelum konversi field ORM
`_check_source_move()`	Memeriksa move dapat dibaca, done, berasal dari receipt kasus, perusahaan sama, serta jumlah positif pada pembulatan satuan	Memvalidasi relasi dan jumlah di server, termasuk setelah perubahan baris
`create(vals_list)`	Mengambil parent dari nilai/context yang sah, memastikan parent ada, mengunci parent dan memeriksa draft pemilik, menolak derived field, memvalidasi jumlah, membersihkan default derived	Melindungi pembuatan baris langsung lewat RPC maupun One2many
`write(vals)`	Mengunci parent dan memeriksa draft pemilik; melarang pindah parent, melarang derived field, memvalidasi perubahan jumlah	Pengguna tidak dapat mengubah kasus submitted melalui model baris
`unlink()`	Mengunci dan memeriksa draft pemilik sebelum penghapusan	Membekukan penghapusan baris setelah pengajuan
`_compute_returnable_qty()`	Menghitung `returnable_qty` setiap dependency move/retur berubah	Memberi angka bantuan pada form; validasi aksi tetap menghitung ulang
`_remaining_returnable_quantity()`	Mengembalikan sisa dalam satuan move sumber	Membatasi jumlah yang dapat dipakai kasus retur baru

`_check_source_move()` menggunakan `float_compare(..., precision_rounding=...)` selain pemeriksaan awal `_validated_quantity_input()`. Angka yang secara Python lebih besar dari nol tetapi menjadi nol pada ketelitian satuan tetap ditolak.

8. Hitung sisa retur dari sumber yang sama

Masukkan **metode lengkap berikut** pada `SupplyflowIssueLine`, sesuai file sumber:

python

```
    def _remaining_returnable_quantity(self):
        self.ensure_one()
        source = self.move_id
        # Count all returns of an authorized source, even if a custom stock rule hides a sibling transfer.
        returned = self.env["stock.move"].sudo().search([
            ("origin_returned_move_id", "=", source.id), ("company_id", "=", source.company_id.id),
            ("state", "!=", "cancel"),
        ])
        committed = sum(
            move.product_uom._compute_quantity(
                move.quantity if move.state == "done" else move.product_uom_qty, source.product_uom,
                round=False,
            ) for move in returned
        )
        return max(0.0, source.quantity - committed)
```

Metode menghitung semua retur yang menunjuk `origin_returned_move_id` sumber tersebut: retur yang dibuat addon maupun retur manual melalui Inventory. Retur done memakai jumlah aktual `quantity`; retur yang belum selesai memakai jumlah rencana `product_uom_qty` sebagai komitmen. Retur berstatus `cancel` tidak mengurangi sisa. Konversi satuan menyamakan semua jumlah ke satuan move sumber sebelum penjumlahan.

`sudo()` di sini dibatasi pada pembacaan retur dari sumber/perusahaan yang sudah diotorisasi oleh pemeriksaan pemanggil. Tujuannya agar aturan stock tambahan yang menyembunyikan retur lain tidak menyebabkan batas jumlah terhitung terlalu besar. Kode tidak mengirim daftar retur tersembunyi ke pengguna, tidak memberi `sudo()` pada seluruh kasus, dan tidak membuat picking dengan izin yang ditingkatkan.

Contoh hitungan, semuanya dalam karton:

Kejadian	Sisa untuk kasus retur baru	Stok gudang jika tidak ada transaksi lain
Receipt selesai menerima 10	10	10
Kasus A disetujui untuk 7, belum membuat picking	10	10
Kasus B disetujui untuk 4, belum membuat picking	10	10
A membuat picking retur 7, belum divalidasi	3	10
B mencoba membuat picking 4	Ditolak karena hanya 3 tersisa	10
Gudang memvalidasi retur A sebanyak 7	3	3

Persetujuan kasus belum mereservasi batas kuantitas pada metode ini. Pembuatan picking non-cancel yang menambah komitmen. Pemeriksaan diulang saat membuat retur, sehingga dua kasus yang sebelumnya sama-sama disetujui belum tentu sama-sama dapat membuat picking.

`returnable_qty` mengukur sisa hak retur terhadap receipt sumber, bukan stok fisik yang tersedia di lokasi sekarang. Barang mungkin sudah dijual atau dipindahkan. Reservasi, lokasi, lot/serial, dan validasi stok tetap mengikuti Inventory native.

9. Buat picking melalui wizard Odoo

Kembali ke kelas parent `SupplyflowIssue`. Tambahkan `_return_action()` dan `action_open_return()` sebelum `action_create_return()`.

`_return_action()` memeriksa akses baca picking lalu mengembalikan dictionary window action dengan `res_model="stock.picking"` dan `res_id` retur tertaut. `action_open_return()` memeriksa anggota/akses baca kasus, menolak jika belum ada retur, lalu memakai helper tersebut. Keduanya membuka dokumen yang sudah ada; tidak membuat stok baru.

Berikut **metode lengkap** pembuat retur dari sumber. Tempatkan satu kali di dalam `SupplyflowIssue`:

python

```
    def action_create_return(self):
        self.ensure_one()
        self._check_approver()
        self._lock()
        if self.return_picking_id:
            return self._return_action()
        if self.state != "approved":
            raise UserError(_("Approve the supplier issue before creating a return transfer."))
        moves = self.line_ids.move_id.sorted("id")
        moves.check_access("write")
        moves.lock_for_update()
        moves.invalidate_recordset()
        self._check_lines(returning=True)
        # Updating a source revision makes stale repeatable-read transactions retry safely.
        moves._supplyflow_advance_return_revision()
        wizard = self.env["stock.return.picking"].with_context(
            {}, allowed_company_ids=self.env.companies.ids, lang=self.env.lang, tz=self.env.user.tz,
            active_model="stock.picking", active_id=self.picking_id.id, active_ids=self.picking_id.ids,
        ).create({"picking_id": self.picking_id.id})
        quantities = {line.move_id.id: line.affected_qty for line in self.line_ids}
        for return_line in wizard.product_return_moves:
            return_line.quantity = quantities.get(return_line.move_id.id, 0.0)
        action = wizard.action_create_returns()
        picking = self.env["stock.picking"].browse(action["res_id"])
        self._transition({"return_picking_id": picking.id, "state": "return_created"})
        self.message_post(body=_("Supplier return transfer created. Stock changes only when the warehouse validates
>it."))
        return self._return_action()
```

Baca metode itu dalam urutan pelaksanaannya:

1. Metode mensyaratkan Approver, mengunci kasus, dan membuka retur yang sama jika sudah tertaut. Ini membuat klik ulang tidak menambah retur kedua untuk kasus yang sama.
2. Untuk pembuatan baru, kasus harus approved. Metode mengurutkan dan mengunci move sumber, memeriksa akses tulis, serta menghitung ulang sisa. Dua kasus berbeda dapat memakai sumber yang sama, sehingga lock parent kasus saja belum cukup.
3. Metode menaikkan revisi move sumber. Penjelasan transaksi ada di langkah berikut.
4. `with_context({}...)` memulai context wizard yang bersih, kemudian meneruskan perusahaan yang diizinkan, bahasa, zona waktu, dan receipt aktif yang benar. Default `picking_id` atau baris wizard yang disisipkan pemanggil tidak boleh mengganti barang yang telah disetujui.
5. `stock.return.picking` adalah wizard sementara native. Addon mengisi `product_return_moves`: jumlah baris yang terdampak mengikuti kasus, semua baris lain menjadi nol.
6. `wizard.action_create_returns()` membuat dokumen native. Hasilnya window action; addon mengambil `action["res_id"]`, menautkannya ke kasus, lalu mengembalikan action untuk membuka form retur.

Pada sumber wizard Odoo 19, `action_create_returns()` menghasilkan action `stock.picking`; proses pembuatannya mengonfirmasi dan mencoba mereservasi transfer. Addon memakai API versi ini. Jangan mengganti nama metode dengan contoh versi Odoo lain tanpa memeriksa sumber yang sesuai.

Tidak ada pemanggilan `button_validate()` di metode addon tersebut. Pembuatan dokumen dan upaya reservasi native belum mengurangi on-hand. Petugas gudang tetap harus memeriksa operasi dan jumlah sebelum memvalidasi picking. Approver juga memerlukan hak Inventory native yang sesuai; kelompok SupplyFlow saja tidak memberikan seluruh hak stok.

10. Tambahkan revisi sumber untuk transaksi bersamaan

Di bagian paling bawah file, setelah kelas baris, tambahkan perluasan `stock.move` berikut. Ini **kelas lengkap dari sumber**:

python

```
class StockMove(models.Model):
    _inherit = "stock.move"

    supplyflow_return_revision = fields.Integer(default=0, readonly=True, copy=False)

    def write(self, vals):
        if "supplyflow_return_revision" in vals:
            raise AccessError(_("Supplier return revisions are managed by the issue workflow."))
        return super().write(vals)

    def _supplyflow_advance_return_revision(self):
        for move in self:
            super(StockMove, move).write({"supplyflow_return_revision": move.supplyflow_return_revision + 1})
```

`_inherit = "stock.move"` memperluas model native yang sudah ada; tidak menamai model baru. Field integer tambahan ini tidak menyimpan jumlah barang. Nilainya menandai perubahan sumber ketika alur addon menciptakan komitmen retur baru.

Lock mengendalikan akses serentak pada baris. Namun transaksi Odoo/PostgreSQL dapat memiliki snapshot baca yang lebih lama. Perubahan revisi memaksa perubahan nyata pada row sumber, sehingga konflik transaksi yang basi dapat dideteksi dan ditangani oleh mekanisme transaksi/RPC Odoo. Setelah percobaan ulang yang berhasil memperoleh keadaan terbaru, pemeriksaan sisa kuantitas tetap wajib berjalan. Lock/retry tidak menggantikan validasi jumlah.

Override `write()` menolak masukan revisi dari pemanggilan biasa. Helper private menaikkan angka melalui `super(StockMove, move).write(...)` pada jalur internal yang telah memeriksa kasus dan sumber. Flag context buatan pengguna tidak membuka field tersebut.

Jaminan ini mempunyai batas: alur addon saling mengoordinasikan kasus yang membuat retur melalui `action_create_return()`. Kode tidak mengganti semua jalur retur manual Odoo menjadi wajib persetujuan SupplyFlow. Perhitungan membaca retur manual yang sudah ada, tetapi jangan mengklaim addon mengendalikan seluruh modifikasi Inventory atau semua persaingan dengan kode modul lain.

11. Hubungkan tombol, ACL, sequence, dan manifest

Salin definisi lengkap dari supplier_issue_views.xml ke file view yang sama pada proyek latihan. Di dalam form, tombol berikut merupakan **potongan XML** yang memanggil metode pengajuan:

xml

```
<button name="action_submit" type="object" string="Submit for review" class="oe_highlight" invisible="state != 'draf
  >t'"/>
```

`name` harus sama dengan nama metode Python. `type="object"` memanggil model record aktif. `invisible` menyesuaikan layar; server tetap memeriksa status/pemilik. Tombol review juga memakai `groups` dan kondisi `requester_id != uid` untuk membantu pengguna menemukan aksi yang boleh dipakai.

Form mengatur isi/baris readonly di luar draft, menampilkan pemberitahuan shortage, menampilkan peringatan bahwa picking baru belum mengubah stok, serta menyediakan smart button `Supplier return`. `duplicate="false"` mencegah alur salin form yang tidak disediakan untuk data sumber/audit ini.

Search view memberi filter pelapor, menunggu review, approved, unresolved, tenggat lewat, dan menunggu gudang. Filter menunggu gudang memerlukan `state="return_created"` dan status picking bukan `done`/`cancel`. Filter overdue melihat kasus `submitted`/`approved` yang melewati `resolution_due`.

Pada sequences.xml, record `sequence_issue` memakai code `supplyflow.issue`, prefix `ISS/%(year)s/`, dan padding 5. Code itu harus sama dengan argumen `next_by_code("supplyflow.issue")` pada `create()`.

Pastikan dua ACL model kasus/baris serta dua record rule perusahaan sudah ditambahkan. Manifest memuat security lebih dahulu, kemudian sequence, kemudian view; menus.xml menghubungkan menu kasus ke `action_issues`. Pada proyek yang dibangun ulang, tunggu sampai susunan lengkap pada bab 05 sebelum memasang addon. Pada proyek referensi yang sudah lengkap, pembaruan mengikuti prosedur OPERATIONS. Menyegarkan browser saja tidak memasang field atau XML yang baru.

12. Periksa alur melalui UI dan tes

Lakukan pemeriksaan ini pada proyek referensi lengkap yang sudah berjalan, atau setelah instalasi lengkap pada bab 05. Gunakan data latihan atau data sintetis sendiri. Siapkan dua akun berbeda: pelapor dengan akses SupplyFlow/Inventory yang sesuai, dan reviewer dengan kelompok Approver serta hak Inventory. Password latihan dibaca dari file kredensial lokal, bukan ditulis ke dokumentasi atau test.

1. Selesaikan satu receipt incoming, misalnya 10 karton. Catat on-hand awal pada produk/lokasi yang dipakai.
2. Sebagai pelapor, buka SupplyFlow > Supplier issues, buat kasus, pilih receipt tersebut, isi alasan/uraian/resolver/tenggat, lalu pilih move dan jumlah 3.
3. Simpan lalu pilih **Submit for review**. Pastikan kasus `submitted` dan isi/baris tidak dapat diedit.
4. Masuk sebagai reviewer lain, pilih **Approve supplier return**. Pastikan `approved`, reviewer/waktu terisi, dan stok belum berubah.
5. Pilih **Create return transfer**. Pastikan form picking native terbuka, jumlah rencana 3, dan sumber menunjuk receipt asli. On-hand masih sama sebelum validasi.
6. Buka kasus lagi dan gunakan **Supplier return**. Picking yang terbuka harus sama, bukan picking kedua.
7. Petugas gudang memeriksa barang/jumlah lalu memvalidasi picking. Untuk contoh tanpa transaksi lain, on-hand 10 menjadi 7; `return_state` menjadi `done`.
8. Ulangi dengan alasan **Quantity shortage**. Review harus melalui **Resolve without return** dengan catatan. Pastikan kasus `accepted` dan tidak memiliki picking retur.

Tambahkan test_supplier_issue.py ke paket tests/__init__.py. Tes memakai `TransactionCase` Odoo, bukan Python kosong. Fixture `_make_receipt()` membuat transfer native, menulis jumlah aktual pada move, lalu memvalidasi receipt sungguhan.

Tes yang dibaca	Perilaku yang dibuktikan
`test_independent_approval_creates_native_return_without_moving_stock`	Hubungan picking/move benar, pembuatan retur belum mengurangi on-hand, validasi menguranginya
`test_return_creation_is_idempotent`	Klik/panggil ulang mengembalikan retur yang sama dan revisi naik satu kali
`test_existing_pending_returns_reserve_their_quantity`	Dua kasus approved tetap diperiksa ulang ketika mulai membuat picking
`test_native_pending_return_is_included_in_returnability`	Retur manual native yang masih menunggu juga dihitung

Tes yang dibaca	Perilaku yang dibuktikan
`test_cancelled_return_does_not_consume_returnable_quantity`	Retur dibatalkan tidak menghabiskan sisa batas
`test_reporter_cannot_approve_or_create_return`, `test_approver_cannot_approve_own_issue`	Hak kelompok dan independensi review diperiksa di server
`test_direct_state_and_review_writes_cannot_bypass_workflow`, `test_context_defaults_cannot_seed_review_or_forge_audit`	`write`/context tidak dapat memalsukan keputusan
`test_line_defaults_cannot_replace_derived_receipt_details`, `test_return_wizard_context_cannot_replace_approved_goods`	Identitas barang dan sumber wizard tidak dapat diganti melalui default context
`test_submitted_issue_and_lines_are_frozen`, `test_another_reporter_cannot_edit_draft_or_reparent_line`	Pembekuan dan kepemilikan berlaku juga pada akses langsung model baris
`test_invalid_source_and_nonpositive_quantity_are_rejected`, `test_invalid_numeric_inputs_are_rejected_before_field_conversion`	Sumber salah dan angka tidak valid ditolak
`test_empty_or_excessive_issue_cannot_be_submitted`	Kasus kosong atau retur melampaui sisa tidak bisa diajukan
`test_shortage_requires_written_nonreturn_resolution`	Shortage memerlukan catatan, tanpa picking baru, dan catatan keputusan dibekukan
`test_issue_company_must_match_received_shipment`, `test_source_revision_cannot_be_forged`	Pembatas perusahaan dan revisi sumber tetap berlaku
`test_only_reporter_or_approver_can_cancel_open_issue`	Pembatalan mengikuti identitas/status dan tetap aman dipanggil ulang

test_distribution_flow.py melanjutkan pemeriksaan sampai PO, receipt parsial, backorder, retur, dan delivery pelanggan. Tes browser juga memakai sesi HTTP terpisah untuk memeriksa kasus retur bersaing. Tes transaksi satu cursor saja tidak mewakili dua permintaan HTTP bersamaan.

Jika hasilnya berbeda

Gejala	Pemeriksaan
Receipt tidak muncul pada pilihan	Pastikan done, jenis incoming, dan perusahaan sesuai; receipt yang masih menunggu bukan sumber yang diizinkan
Produk tidak muncul pada baris kasus	Pilih move selesai dari receipt yang sama; nama produk yang sama pada receipt lain belum tentu move yang benar
Review ditolak walau akun Approver	Bandingkan akun reviewer dengan `requester_id`; penugasan resolver tidak menghapus syarat reviewer berbeda

Gejala	Pemeriksaan
Create return ditolak karena akses	Periksa hak native Inventory serta ACL/record rule, bukan hanya kelompok SupplyFlow
Jumlah yang kemarin tersedia sekarang ditolak	Periksa retur manual/pending dari sumber yang sama; pemeriksaan saat membuat picking membaca komitmen terbaru
Shortage tidak mempunyai tombol approve retur	Ini jalur yang diharapkan; isi catatan penyelesaian lalu selesaikan tanpa retur
Stok belum berkurang setelah membuat retur	Buka picking dan periksa statusnya; stok berkurang setelah gudang memvalidasi jumlah aktual
Kasus tetap `return_created` setelah picking selesai	Periksa `return_state`; kedua field memang menyatakan tahap dokumen yang berbeda
Kasus tidak dapat diedit setelah diajukan	Isi/baris memang dibekukan; gunakan pembatalan pada status yang diizinkan bila perlu membuat laporan pengganti
Field baru atau tombol tidak ditemukan	Periksa import Python, manifest, log update addon, dan hasil pemuatan XML sebelum mencoba ulang di browser

Setelah bab ini, Anda dapat menjelaskan hubungan setiap lapisan: form memanggil `action_*`, metode memeriksa hak/status dan mengunci record, baris memvalidasi sumber serta batas jumlah, wizard native menghasilkan picking, dan Inventory memproses perpindahan stok saat validasi gudang.

05. Membuat form, workspace, dan PDF

Bab ini menghubungkan model Python yang sudah dibuat dengan layar yang dipakai pengguna. Hasilnya terdiri dari form/daftar native Odoo, satu workspace Owl, dan laporan PDF QWeb. Model tetap memegang aturan bisnis; layar memanggil model tersebut.

Kerjakan pada salinan repo untuk belajar. Potongan di bawah diambil dari sumber proyek dan menunjukkan letak kode yang dibahas. Potongan bukan pengganti seluruh isi berkas. Tautan sumber lengkap disediakan agar penutupan tag, import, dan bagian lain tetap utuh.

Bab 02 memakai kerangka minimal agar addon pertama dapat dipasang. Sesudah mempelajari model lengkap pada bab 03 dan 04, gunakan versi lengkap manifest, import model, security, view, report, dan asset yang dirujuk bab ini. Muat seluruh rangkaian tersebut bersama sebelum memilih data latihan. View final merujuk field model final; memasangkan view final dengan model kerangka akan menghasilkan error field tidak ditemukan.

1. Kenali tiga jenis tampilan

Jenis	Dipakai untuk	Sumber proyek
View XML native	Daftar, form, pencarian, tombol, dan menu dokumen	purchase_request_views.xml, supplier_issue_views.xml, menus.xml
Komponen Owl	Ringkasan operasi yang mengambil data lewat RPC	workspace.py, workspace.js, workspace.xml, workspace.scss
QWeb report	Dokumen ringkasan yang dapat dicetak menjadi PDF	purchase_request_report.xml

Satu berkas XML form dapat berisi beberapa record view dan action. Odoo menyimpan definisi tersebut dalam database saat addon dipasang atau diperbarui. Mengedit XML pada disk belum memperbarui definisi database yang sedang dipakai.

2. Daftarkan XML dan asset dalam manifest

Buka __manifest__.py. Bagian `data` memuat data XML/CSV secara berurutan. Pada proyek ini urutannya dimulai dari security, penomoran, kedua view, report, lalu menu. Menu diletakkan setelah action yang dirujuknya tersedia.

Bagian berikut berada setelah `data` di dictionary manifest. Ini satu entry dictionary, sehingga tidak dapat dijalankan sebagai file Python tersendiri. Pertahankan kurung kurawal pembuka/penutup dan entry lain pada manifest lengkap:

python

```
    "assets": {
        "web.assets_backend": [
            "supplyflow_office/static/src/workspace.js",
            "supplyflow_office/static/src/workspace.xml",
            "supplyflow_office/static/src/workspace.scss",
        ],
    },
```

`web.assets_backend` adalah kumpulan asset untuk web client ERP setelah login. JavaScript, template Owl, dan SCSS workspace dimuat melalui kumpulan ini. File XML pada `static/src` berbeda jalur pemuatannya dari XML view dalam `data`.

Checkpoint: cocokkan semua path dengan nama berkas pada disk. Salah satu nama yang keliru dapat membuat asset gagal dibangun sebelum komponen tampil.

3. Buat daftar permintaan pembelian

Di purchase_request_views.xml, record pertama adalah `view_request_list`. Bagian pembukanya:

xml

```
    <record id="view_request_list" model="ir.ui.view">
        <field name="name">supplyflow.request.list</field>
        <field name="model">supplyflow.request</field>
        <field name="arch" type="xml">
            <list decoration-muted="state == 'cancelled'" decoration-danger="state == 'rejected'">
                <field name="name"/>
                <field name="requester_id"/>
                <field name="vendor_id"/>
```

Letakkan record view di dalam tag `<odoo>`, seperti pada sumber lengkap. `model="ir.ui.view"` membuat definisi view; field `model` di dalamnya menentukan model bisnis yang ditampilkan. `arch` menampung susunan layar.

Setiap `<field>` pada `<list>` menjadi kolom. Nama field harus sama dengan model Python. Kolom lain pada sumber menampilkan gudang, tanggal kebutuhan, estimasi, perusahaan, dan status. `sum="Estimated total"` menjumlahkan kolom estimasi pada daftar. `currency_id` disertakan untuk format monetary meskipun kolomnya disembunyikan.

`decoration-muted` dan `decoration-danger` memberi petunjuk visual sesuai status. Keduanya tidak mengubah status atau izin record.

4. Susun form dan hubungkan tombol ke metode Python

Record `view_request_form` pada berkas yang sama memakai:

xml

```
            <form string="Purchase Requisition" class="sf_document" duplicate="false">
                <header>
                    <button name="action_submit" string="Submit for Approval" type="object" class="oe_highlight" inv
>isible="state != 'draft' or requester_id != uid"/>
                    <button name="action_approve" string="Approve" type="object" class="oe_highlight" groups="supply
>flow_office.group_approver" invisible="state != 'submitted' or requester_id == uid"/>
```

Tempatkan tombol transisi di `<header>`; identitas, field isian, dan tab berada di `<sheet>`. `duplicate="false"` menonaktifkan Duplicate pada form karena salinan langsung berisiko membawa snapshot/audit yang tidak sesuai dengan alur pembuatan baru.

Rantai tombol Approve adalah:

text

```
Klik Approve
  → web client mengirim call_button untuk supplyflow.request.action_approve
  → metode Python memeriksa akses, pemilik, dan status
  → ORM memperbarui status serta metadata keputusan
  → form membaca ulang hasilnya
```

Pada tombol `type="object"`, `name` adalah nama metode model, bukan URL. Metode tersebut berada di purchase_request.py. `string` adalah label yang terlihat pengguna. `groups` membatasi penampilan tombol, sedangkan `invisible` menentukan kapan tombol disembunyikan.

`uid` menunjuk ID pengguna aktif. Ekspresi XML membantu menampilkan tindakan yang relevan. Python tetap memeriksa izin karena pengguna juga dapat memanggil RPC tanpa menekan tombol.

Seluruh tombol permintaan pada sumber:

Label	Metode pada `supplyflow.request`	Kapan dipakai
Submit for Approval	`action_submit()`	Pengaju mengirim draft miliknya
Approve	`action_approve()`	Approver independen menyetujui submitted
Reject	`action_reject()`	Approver menolak submitted dengan alasan
Create RFQ	`action_create_rfq()`	Membuat RFQ dari approved
Open RFQ	`action_create_rfq()`	Membuka RFQ yang sudah tertaut; jalur ini memakai hasil yang sama
Revise Draft	`action_reset_draft()`	Pengaju merevisi dokumen rejected
Withdraw	`action_cancel()`	Pengaju menarik dokumen pada status yang diizinkan
Cancel Approval	`action_cancel()`	Approver independen membatalkan persetujuan sebelum RFQ terbentuk

`action_create_rfq()` mengembalikan action pembukaan `purchase.order`. Form RFQ setelah itu berasal dari modul Purchase native. Proyek tidak membuat salinan layar pembelian sendiri.

Field readonly, domain, tab, dan chatter

Di dalam `<sheet>`, kelompok field menggunakan nama yang sudah ada di model. Contohnya:

xml

```
                        <field name="vendor_id" readonly="state != 'draft' or requester_id != uid" domain="['|',
> ('company_id', '=', False), ('company_id', '=', company_id)]"/>
                        <field name="warehouse_id" readonly="state != 'draft' or requester_id != uid" domain="[(
>'company_id', '=', company_id)]"/>
```

`readonly` membantu menjaga dokumen yang sudah diajukan tetap terbaca sebagai hasil review. `domain` menyaring pilihan pemasok/gudang pada UI. Validasi kecocokan perusahaan juga berada di server.

`<notebook>` memuat tab Requested Items dan Approval. Field `line_ids` menampilkan `<list editable="bottom">` untuk menambah item di bagian bawah tabel. Field subtotal/total menampilkan hasil perhitungan model, bukan hasil JavaScript buatan form. UoM baris readonly karena merupakan snapshot satuan produk.

Tab Approval menampilkan catatan dan siapa/kapan pengajuan, persetujuan, atau penolakan dilakukan. Satu field `decision_reason` muncul dengan aturan kelompok berbeda: Approver dapat mengisinya pada submitted milik orang lain, anggota lain membacanya readonly.

`<chatter/>` berada setelah `<sheet>`. Model mengaktifkan fasilitas mail/chatter dan menulis riwayat keputusan. Menambahkan tag chatter saja tidak menciptakan seluruh aturan pencatatan tersebut.

Checkpoint: masuk sebagai buyer, buat permintaan, tambah item, lalu ajukan. Form berhenti menerima perubahan baris. Masuk sebagai approver lain untuk melihat Approve. Bandingkan hasilnya dengan pemeriksaan server pada model.

5. Tambahkan pencarian dan window action

Record `view_request_search` pada berkas yang sama mendefinisikan field pencarian, filter, serta group by. Salah satu filternya:

xml

```
<filter name="awaiting_approval" string="Awaiting Approval" domain="[('state', '=', 'submitted')]"/>
```

`name` menjadi pengenal filter untuk context JavaScript nanti. `string` adalah label. `domain` menentukan record hasilnya. Filter My Requisitions memakai `requester_id = uid`; filter tersebut tidak menjadi record rule yang menyembunyikan dokumen tim.

Record action ada setelah view pencarian:

xml

```
<record id="action_requests" model="ir.actions.act_window">
    <field name="name">Purchase Requisitions</field>
    <field name="res_model">supplyflow.request</field>
    <field name="view_mode">list,form</field>
    <field name="search_view_id" ref="view_request_search"/>
```

Action ini meminta web client membuka daftar/form model `supplyflow.request`. `search_view_id` memilih view pencariannya. Field `help` pada sumber lengkap menyediakan penjelasan ketika daftar belum memiliki record.

6. Baca form kasus pemasok dengan pola yang sama

supplier_issue_views.xml memiliki `view_issue_list`, `view_issue_form`, `view_issue_search`, serta `action_issues`. List menampilkan receipt, alasan, penanggung jawab, tenggat, status kasus, dan status retur. Form menghubungkan tindakan ke supplier_issue.py.

Label	Metode pada `supplyflow.issue`	Hasil utama
Submit for review	`action_submit()`	Mengajukan kasus draft
Approve supplier return	`action_approve()`	Menyetujui rencana retur oleh reviewer independen
Resolve without return	`action_accept_resolution()`	Menyelesaikan kasus dengan catatan, tanpa picking retur
Create return transfer	`action_create_return()`	Membuat picking retur native yang tertaut
Cancel issue	`action_cancel()`	Membatalkan pada status/aktor yang diizinkan
Supplier return	`action_open_return()`	Membuka picking retur tertaut

Tombol Cancel issue mempunyai dua definisi XML: satu untuk Approver dan satu untuk kelompok selain Approver. Pemeriksaan model menentukan izin akhirnya. Tombol Submit for review sendiri tidak menuliskan syarat pemilik pada `invisible`; server tetap memeriksa pelakunya.

Smart button Supplier return berada dalam `oe_button_box`. `return_state` menunjukkan status picking, sedangkan statusbar form menunjukkan status kasus. Saat kasus menjadi `return_created`, stok belum otomatis berkurang. Petugas gudang masih harus memvalidasi picking di Inventory.

Filter kasus yang dipakai workspace adalah:

Nama filter XML	Isi
`open_reviewed`	Kasus submitted atau approved
`overdue`	Kasus submitted/approved yang tenggatnya sudah lewat
`waiting_warehouse`	Kasus return_created dengan picking belum done/cancel

Sesudah description, form memasang editor barang terdampak dan catatan resolusi:

xml

```
                    <field name="line_ids" readonly="state != 'draft'">
                        <list editable="bottom">
                            <field name="picking_id" column_invisible="True"/>
                            <field name="company_id" column_invisible="True"/>
                            <field name="move_id" options="{'no_create': True}"/>
                            <field name="product_id" readonly="True"/>
                            <field name="affected_qty"/>
                            <field name="product_uom_id" readonly="True"/>
                            <field name="returnable_qty" readonly="True"/>
                        </list>
                    </field>
                    <separator string="Resolution"/>
                    <field name="resolution_note" nolabel="1" class="w-100" readonly="state not in ['draft', 'submit
>ted']" placeholder="Record the supplier agreement and explain why no physical return is required."/>
```

`move_id` memilih stock move dari receipt. Produk dan UoM diturunkan dari sumber tersebut; pengguna mengisi `affected_qty`, sedangkan `returnable_qty` memperlihatkan sisa jumlah menurut model. Tabel menjadi readonly setelah draft. Catatan resolusi dapat diisi pada draft/submitted menurut UI; metode server tetap memeriksa siapa yang boleh menulisnya. Skenario browser memakai kombinasi fixture ORM/RPC dan klik form, sehingga hasil pengujian perlu dibaca sesuai cakupannya pada bab berikut.

7. Hubungkan menu dengan action

menus.xml berisi:

xml

```
    <record id="action_workspace" model="ir.actions.client">
        <field name="name">Operations</field>
        <field name="tag">supplyflow_workspace</field>
    </record>
    <menuitem id="menu_supplyflow" name="SupplyFlow Office" sequence="1" groups="group_user"/>
    <menuitem id="menu_workspace" name="Operations" parent="menu_supplyflow" action="action_workspace" sequence="1"/
>>
    <menuitem id="menu_requests" name="Purchase requests" parent="menu_supplyflow" action="action_requests" sequence
>="10"/>
    <menuitem id="menu_issues" name="Supplier issues" parent="menu_supplyflow" action="action_issues" sequence="20"/
>>
```

Menu dokumen menunjuk window action. Operations menunjuk client action karena akan membuka komponen JavaScript. Nilai `supplyflow_workspace` harus sama dengan key registry pada `workspace.js`.

Kelompok `group_user` didefinisikan di groups.xml. ACL pada ir.model.access.csv memberi operasi model, dan company_rules.xml membatasi perusahaan. Menambahkan kelompok pada menu hanya mengatur visibilitas menu, sehingga dua lapisan security lain tetap diperlukan.

8. Siapkan endpoint model untuk workspace

workspace.py memakai `models.AbstractModel` dengan `_name = "supplyflow.workspace"`. Model ini menyediakan layanan ringkasan tanpa membuat tabel dokumen bisnis baru. Pastikan file diimpor dari models/__init__.py.

Awal fungsi `get_overview()`:

python

```
@api.model
def get_overview(self):
    if not self.env.user.has_group("supplyflow_office.group_user"):
        raise AccessError(self.env._("SupplyFlow access is required."))
    requests = self.env["supplyflow.request"]
    issues = self.env["supplyflow.issue"]
    open_issues = issues.search([("state", "in", ["submitted", "approved"])])
    waiting_returns = issues.search([("state", "=", "return_created")]).filtered(
        lambda issue: issue.return_picking_id.state not in ("done", "cancel")
    )
    recent = requests.search([], limit=8, order="create_date desc, id desc")
```

Letaknya setelah definisi nama/description class. `@api.model` sesuai untuk pemanggilan yang tidak membutuhkan ID satu dokumen. `has_group` menolak akun yang tidak mendapat akses SupplyFlow. Pencarian melalui ORM berjalan sebagai pengguna aktif, sehingga record rule perusahaan tetap berlaku. Fungsi ini tidak memanggil `sudo()` untuk menghitung seluruh perusahaan.

Dictionary hasil fungsi memuat:

Key	Cara sumber menghitung
`company`, `today`	Nama perusahaan aktif dan tanggal menurut context
`pending_requests`	`search_count` permintaan submitted
`open_issues`	Jumlah kasus submitted/approved
`overdue_issues`	Kasus terbuka dengan `resolution_due` sebelum hari ini
`pending_returns`	Kasus return_created dengan picking belum done/cancel
`recent_requests`	Maksimal delapan permintaan terbaru yang boleh dibaca

Setiap permintaan terbaru dikirim sebagai ID, nomor, pemasok, pengaju, label status, dan estimasi terformat mata uang. Browser menerima ringkasan yang diperlukan, bukan recordset Python mentah.

9. Buat komponen Owl dan muat data

Di awal workspace.js, proyek mengimpor `Component`, `onWillStart`, `useState`, registry, `useService`, serta `Layout` dari modul Odoo. Class `SupplyFlowWorkspace` memilih template `supplyflow_office.Workspace` dan mendaftarkan komponen `Layout`.

Masukkan `setup()` dan `load()` sebagai metode di dalam class, seperti sumber berikut:

javascript

```
setup() {
    this.orm = useService("orm");
    this.action = useService("action");
    this.state = useState({ loading: true, error: "", data: null });
```

```
        onWillStart(() => this.load());
    }

    async load() {
        this.state.loading = true;
        this.state.error = "";
        try {
            this.state.data = await this.orm.call("supplyflow.workspace", "get_overview", []);
        } catch {
            this.state.error = "The workspace could not be loaded. Check your connection and try again.";
        } finally {
            this.state.loading = false;
        }
    }
```

`orm` mengirim RPC melalui sesi login Odoo. `action` membuka layar Odoo. `useState` membuat perubahan pada `loading`, `error`, dan `data` memicu pembaruan template. `onWillStart` menjalankan pemuatan saat komponen mulai dibuka.

Alur lengkapnya:

Alur proses (tabel padanan diagram pada Markdown)

Urutan	Dari	Ke	Pesan / tindakan
1	Pengguna	Owl workspace.js	Buka Operations / Refresh
2	Owl workspace.js	Owl workspace.js	loading=true, error kosong
3	Owl workspace.js	workspace.py	orm.call(get_overview)
4	workspace.py	workspace.py	Periksa kelompok dan akses perusahaan
5	workspace.py	PostgreSQL melalui ORM	search / search_count
6	PostgreSQL melalui ORM	workspace.py	Record yang boleh diakses
7	workspace.py	Owl workspace.js	Dictionary ringkasan
8	Owl workspace.js	Owl workspace.js	Simpan data, loading=false
9	Owl workspace.js	Pengguna	Template menampilkan angka dan daftar

Jika RPC gagal, `catch` menyimpan pesan yang dapat dibaca pengguna. `finally` mematikan loading pada kedua hasil. Tombol Try again memanggil fungsi yang sama.

10. Buka daftar/form dari workspace

Metode navigasi juga berada di dalam class JavaScript:

javascript

```
    openRequests(pending = false) {
        return this.action.doAction("supplyflow_office.action_requests", {
            additionalContext: pending ? { search_default_awaiting_approval: 1 } : {},
        });
    }

    openIssues(filter = "") {
        return this.action.doAction("supplyflow_office.action_issues", {
            additionalContext: filter ? { [`search_default_${filter}`]: 1 } : {},
```

```
        });
    }

    openRequest(id) {
        return this.action.doAction({ type: "ir.actions.act_window", res_model: "supplyflow.request", res_id: id, vi
>ews: [[false, "form"]], target: "current" });
    }
```

`search_default_awaiting_approval` mengaktifkan filter `awaiting_approval` yang dibuat di XML. Nama setelah `search_default_` harus cocok. `openIssues()` melakukan pola yang sama untuk tiga filter kasus. `openRequest(id)` membuka satu record langsung dari baris terbaru.

Di luar class, baris akhir file menghubungkan komponen dengan client action menu:

javascript

```
registry.category("actions").add("supplyflow_workspace", SupplyFlowWorkspace);
```

Checkpoint: telusuri satu key dari ujung ke ujung, misalnya `overdue_issues` di Python, `state.data.overdue_issues` pada template, `openIssues('overdue')` di tombol, lalu filter `overdue` di XML kasus.

11. Pasang template dan tata letak

Template workspace.xml dibungkus `<templates>` dan memiliki `t-name="supplyflow_office.Workspace"`. Nama tersebut harus cocok dengan `static template` pada class. `Layout` mempertahankan struktur web client Odoo.

Contoh tombol metrik pada template:

xml

```
<button class="sf_metric" t-on-click="() => this.openRequests(true)">
    <span>Requests awaiting approval</span><strong t-esc="state.data.pending_requests"/>
</button>
```

`t-on-click` menghubungkan event ke metode. `t-esc` menampilkan nilai dengan escaping. Bagian `t-if`, `t-elif`, dan `t-else` memilih loading, error, data, atau daftar kosong. Daftar terbaru memakai `t-foreach` dan `t-key="request.id"` supaya setiap baris memiliki identitas stabil.

Data kosong tetap merupakan hasil yang sah: empat angka nol dan pesan No purchase requests yet. Error RPC memiliki pesan dan Try again. Loading menyembunyikan metrik sebelumnya sementara permintaan baru berjalan dan menonaktifkan Refresh. Ketiga kondisi itu diuji terpisah pada browser.

workspace.scss berisi class yang diawali `sf_` agar aturan workspace mudah dikenali. Grid metrik berubah dari empat kolom menjadi dua pada lebar 1000 px, kemudian satu pada 575 px. Tombol memiliki tinggi minimum 44 px dan `:focus-visible` memiliki outline. Tabel memakai wrapper `table-responsive` agar dapat bergulir di dalam bagian tabel.

Bagian `.sf_document .o_form_label.o_form_label_readonly` pada akhir SCSS mempertahankan keterbacaan label readonly form addon. Aturan ini juga menunjukkan bahwa file asset dapat memuat gaya form custom selain gaya workspace. Gaya native Sales/Purchase/Inventory tetap berasal dari Odoo.

12. Buat binding report dan isi QWeb

Di purchase_request_report.xml, record pertama mendaftarkan laporan:

xml

```
<record id="action_report_request" model="ir.actions.report">
    <field name="name">Purchase request summary</field>
    <field name="model">supplyflow.request</field>
    <field name="report_type">qweb-pdf</field>
    <field name="report_name">supplyflow_office.report_request</field>
    <field name="report_file">supplyflow_office.report_request</field>
    <field name="binding_model_id" ref="model_supplyflow_request"/>
    <field name="binding_type">report</field>
    <field name="print_report_name">'Purchase request - %s' % object.name.replace('/', '-')</field>
</record>
```

Letakkan record ini di dalam `<odoo>`, sebelum template laporan pada sumber. `model` menentukan dokumen asal. `report_name` menunjuk external ID template. `binding_model_id` dan `binding_type` menempatkan laporan pada menu Actions/Print dokumen terkait. `print_report_name` menentukan nama unduhan berdasarkan nomor permintaan.

Template `report_request` memanggil `web.html_container`, melakukan `t-foreach="docs"`, lalu memakai `web.external_layout` dan `<div class="page">`. Susun metadata pemasok/gudang/pengaju, purpose, tabel item, total, keputusan, serta RFQ tertaut di dalam page.

Baris tabel pada sumber memakai pola berikut:

xml

```
<tr t-foreach="o.line_ids" t-as="line">
    <td><span t-field="line.name"/></td>
    <td class="text-end"><span t-field="line.product_qty"/></td>
    <td><span t-field="line.product_uom_id.name"/></td>
    <td class="text-end"><span t-field="line.price_estimated"/></td>
    <td class="text-end"><span t-field="line.subtotal"/></td>
</tr>
```

`t-field` memakai informasi field Odoo untuk format angka, monetary, dan tanggal. Pada implementasi ini letakkan `t-field` di `<span>` di dalam `<td>`. Pemasangan langsung pada `<td>` sempat menyebabkan error render; sumber akhir sudah memakai span.

Laporan adalah ringkasan permintaan internal. Kalimat pada PDF mengarahkan pengguna meninjau RFQ di Purchase sebelum memesan ke pemasok. PDF tidak mengonfirmasi PO dan tidak memindahkan stok. Tampilan PDF juga tidak memakai SCSS Owl sebagai halaman ERP biasa: QWeb report mempunyai jalur render dan layout sendiri.

13. Pasang paket final pada folder latihan

Paket final menggantikan seluruh starter bab 02. Model final, security empat model, view, report, asset, import, manifest, dan tes harus tersedia bersama. Menambahkan hanya satu cuplikan fungsi ke model starter belum membentuk addon final.

Gunakan folder latihan `D:\Latihan-Odoo\supplyflow-office`. Pada **setiap terminal latihan yang baru**, tetapkan kembali `COMPOSE_PROJECT_NAME` sebelum menjalankan script atau Compose. Nilainya tidak tersimpan otomatis ke terminal lain. Nama `supplyflow-belajar` memisahkan container dan volume latihan dari proyek utama; port host tetap 8070, sehingga proyek utama perlu berhenti saat latihan dijalankan.

Jika starter sudah pernah dijalankan dan belum dihentikan, hentikan layanan latihan lebih dahulu:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Stop.ps1
```

Lewati perintah Stop tersebut jika belum pernah menyalakan starter dan konfigurasi lokal belum dibuat. Sesudah itu, salin **hanya folder addon** dari source referensi ke folder latihan:

powershell

```
$referensi = 'D:\kerjaan\done push repo\supplyflow-odoo'
$latihan = 'D:\Latihan-Odoo\supplyflow-office'
if (-not (Test-Path -LiteralPath $latihan -PathType Container)) {
    throw 'Folder latihan belum ada. Selesaikan persiapan bab 01 lebih dahulu.'
}
if (-not (Test-Path -LiteralPath "$referensi\addons\supplyflow_office\__manifest__.py" -PathType Leaf)) {
    throw 'Source addon referensi tidak ditemukan. Periksa lokasi repo utama.'
}
Set-Location -LiteralPath $latihan
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
New-Item -ItemType Directory -Path "$latihan\addons" -Force | Out-Null
Copy-Item -LiteralPath "$referensi\addons\supplyflow_office" -Destination "$latihan\addons" -Recurse -Force
```

Perintah itu sengaja menimpa berkas starter pada `addons\supplyflow_office` dengan versi final yang sudah dibahas. Tujuan salinan adalah parent `addons`, sehingga folder hasilnya tetap `addons\supplyflow_office`, bukan folder bernama sama yang bertumpuk. Tidak ada penghapusan direktori. Source utama tetap menjadi referensi; secret, konfigurasi lokal, backup, node_modules, database, dan filestore tidak ikut disalin.

Periksa paket hasil salinan:

Kelompok berkas final	Pemeriksaan sebelum update
__init__.py dan models/__init__.py	Root mengimpor models; models mengimpor purchase_request, supplier_issue, dan workspace
purchase_request.py, supplier_issue.py, workspace.py	Model final dari bab 03/04 dan layanan ringkasan terpasang
groups.xml, ir.model.access.csv, company_rules.xml	Seluruh model parent/baris mendapat security yang dirancang
sequences.xml	Nomor permintaan/kasus tersedia
Kedua view, menus, report, dan tiga asset yang dibahas di bab ini	Semua referensi field, action, template, dan registry sesuai
__manifest__.py	Dependency, daftar data, dan assets memakai versi lengkap yang cocok dengan paket di atas
tests/__init__.py dan empat file test Python pada folder yang sama	Paket tes final ikut tersalin; penjelasannya ada pada bab 06

Jalur A: starter bab 02 sudah pernah terpasang

Database latihan sudah mengenal addon starter, sehingga lakukan update modul sebelum memuat data latihan final. Dari terminal latihan:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
docker compose up -d --wait postgres
docker compose stop odoo
docker compose run --rm -T odoo python3 /opt/supplyflow/manage.py update
.\scripts\Start.ps1 -WithTrainingData -SkipPull
```

Update menambahkan model/field/view/security final ke database yang sebelumnya memuat starter. Karena checkpoint starter dijaga tanpa record transaksi, tidak ada permintaan percobaan yang kekurangan field wajib final untuk dipertahankan. Setelah update berhasil, Start memeriksa instalasi, memuat latihan, dan menyalakan Odoo. `-SkipPull` dipakai karena checkpoint sebelumnya sudah menyediakan image yang dipatok.

Jalur B: starter belum pernah dipasang

Jika baru membuat folder/file latihan dan langsung memasang paket final, belum ada modul starter pada database. Jalankan instalasi biasa sesudah salinan addon lengkap tersedia:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Start.ps1 -WithTrainingData
```

Start membuat secret/config lokal latihan, menyalakan PostgreSQL, memasang modul final melalui initialize, lalu menjalankan seed. Pada jalur ini tidak perlu memanggil update sebelum instalasi pertama. Pilih salah satu jalur sesuai keadaan database latihan, bukan menjalankan keduanya sebagai langkah wajib.

Kredensial latihan ada pada `D:\Latihan-Odoo\supplyflow-office\.secrets\LOCAL-CREDENTIALS.txt`; kredensial proyek utama tidak disalin ke sini. Jangan memuat latihan final saat model masih berupa kerangka minimal karena script latihan memanggil field dan metode workflow final. Rincian update/operasi tersedia pada OPERATIONS.md, dengan tambahan context proyek latihan yang ditetapkan di atas.

14. Periksa hasil di browser

Refresh browser setelah update. Untuk menelusuri asset saat belajar, buka `http://localhost:8070/odoo?debug=assets`, lalu lihat Console dan Network pada developer tools browser.

Periksa hasil secara berurutan:

1. Menu SupplyFlow muncul untuk akun yang mendapat kelompok addon.
2. Purchase requests membuka daftar, New membuka form, dan filter Awaiting Approval bekerja.
3. Pengaju dapat mengirim dokumennya, reviewer lain dapat mengambil keputusan, dan RFQ terbuka lewat tombol tertaut.
4. Operations menampilkan hasil `get_overview`, serta klik setiap metrik membuka filter yang sesuai.
5. Refresh, kondisi kosong, error/retry, keyboard, dan viewport sempit tetap dapat dipakai.
6. Actions/Print pada permintaan menawarkan Purchase request summary dan menghasilkan PDF.

Contoh layar nyata tersedia pada 01-operations.png, 02-purchase-requests.png, 03-approved-request.png, 04-native-purchase.png, 05-supplier-resolution.png, 06-native-receipt.png, 07-native-sale.png, dan 08-mobile-workspace.png. Data pada gambar merupakan latihan sintetis. Hasil pemeriksaan yang sudah dilakukan tercatat pada VERIFICATION.md.

Lanjutkan ke 06. Pengujian dan debugging untuk memeriksa aturan server, tombol, PDF, serta transaksi bersamaan.

06. Menguji aturan ERP dan mencari penyebab error

Tes perlu membuktikan bahwa dokumen dan stok berubah sesuai aturan, termasuk saat seseorang memanggil metode langsung tanpa tombol UI. SupplyFlow memeriksa hal itu melalui tes ORM Odoo, validator arsip backup, dan browser Playwright.

Bab ini menjelaskan berkas tes yang sudah ada, cara membacanya, cara menjalankannya, dan lokasi pemeriksaan saat terjadi error. Hasil aktual proyek dicatat terpisah pada VERIFICATION.md. Menyalin tes atau menjalankan perintah belum berarti tes tersebut lulus.

Semua perintah praktik di bab ini memakai folder **latihan** sesudah paket final bab 05 dipasang. Pada setiap terminal baru, masuk ke `D:\Latihan-Odoo\supplyflow-office` dan set `COMPOSE_PROJECT_NAME=supplyflow-belajar` lagi. Nama itu harus cocok dengan data serta secret yang dibuat terminal latihan sebelumnya.

Siapkan berkas verifikasi pada folder latihan

Bab 01 sudah menyalin scripts dan Compose; bab 05 menyalin addon lengkap beserta tes ORM-nya. Untuk tes arsip dan browser, salin sumber verifikasi berikut. Perintah hanya menyalin file source dan lockfile, tanpa `node_modules`, password, hasil tes, atau data instance:

powershell

```
$referensi = 'D:\kerjaan\done push repo\supplyflow-odoo'
$latihan = 'D:\Latihan-Odoo\supplyflow-office'
if (-not (Test-Path -LiteralPath "$latihan\addons\supplyflow_office\__manifest__.py" -PathType Leaf)) {
    throw 'Addon latihan belum tersedia. Selesaikan paket final bab 05 lebih dahulu.'
}
Set-Location -LiteralPath $latihan
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
Copy-Item -LiteralPath "$referensi\package.json" -Destination "$latihan\package.json" -Force
Copy-Item -LiteralPath "$referensi\package-lock.json" -Destination "$latihan\package-lock.json" -Force
Copy-Item -LiteralPath "$referensi\playwright.config.js" -Destination "$latihan\playwright.config.js" -Force
New-Item -ItemType Directory -Path "$latihan\tests\browser" -Force | Out-Null
Copy-Item -LiteralPath "$referensi\tests\test_backup_archive.py" -Destination "$latihan\tests\test_backup_archive.py
  >" -Force
Copy-Item -LiteralPath "$referensi\tests\browser\helpers.js" -Destination "$latihan\tests\browser\helpers.js" -Force
Copy-Item -LiteralPath "$referensi\tests\browser\workflows.spec.js" -Destination "$latihan\tests\browser\workflows.s
  >pec.js" -Force
Copy-Item -LiteralPath "$referensi\tests\browser\screenshots.spec.js" -Destination "$latihan\tests\browser\screensho
  >ts.spec.js" -Force
```

Jika hanya membaca dan sengaja menguji proyek utama yang sudah jadi, pilih terminal terpisah dan gunakan pengganti konteks berikut sebelum Docker/script proyek utama:

powershell

```
Remove-Item Env:COMPOSE_PROJECT_NAME -ErrorAction SilentlyContinue
Set-Location -LiteralPath 'D:\kerjaan\done push repo\supplyflow-odoo'
```

Pada jalur proyek utama, jangan menerapkan `COMPOSE_PROJECT_NAME=supplyflow-belajar`. Pada jalur latihan, jangan beralih ke folder utama sambil membawa variabel latihan. Setiap blok praktik berikut kembali menetapkan folder dan context latihan supaya password/config serta volume berasal dari instance yang sama. Port kedua instance tetap 8070; jalankan satu instance pada satu waktu.

1. Pisahkan tiga lapisan pemeriksaan

Lapisan	Jumlah pada versi terdokumentasi	Sumber	Yang dibuktikan
ORM addon	42 metode tes: 17 permintaan, 20 kasus, 3 workspace, 2 distribusi	tests addon	Model, izin, transisi, angka, hubungan native, perusahaan, dan perubahan stok
Validator arsip	25 metode unittest	test_backup_archive.py	Struktur/path/hash/versi arsip sebelum restore
Workflow browser	9 tes	workflows.spec.js	Login, tombol/form, navigasi, kondisi workspace, PDF, dan transaksi HTTP bersamaan
Capture	1 tes yang menghasilkan 8 PNG	screenshots.spec.js	Mengambil gambar dari aplikasi yang berjalan dengan data latihan

Capture dipisahkan dari sembilan workflow melalui tag `@capture`. Full run lokal yang mengikutsertakan capture berjumlah sepuluh tes. CI menjalankan sembilan workflow tanpa capture. Angka 42/25/9 tidak boleh dijumlahkan lalu disebut persentase coverage kode; proyek tidak mengukur line coverage pada catatan tersebut.

2. Pahami struktur tes ORM

addons/supplyflow_office/tests/__init__.py mengimpor keempat file tes. Berkas tes yang tidak diimpor dapat luput dari pemuatan test runner addon.

Pada test_purchase_request.py, pembuka class adalah:

python

```
@tagged("post_install", "-at_install", "supplyflow")
class TestPurchaseRequest(TransactionCase):
    @classmethod
    def setUpClass(cls):
        super().setUpClass()
```

`TransactionCase` memberikan environment Odoo beserta transaksi database. `setUpClass()` menyiapkan fixture bersama satu kali untuk class. Fixture adalah data yang sengaja dibuat agar kondisi awal tes jelas, seperti pemasok, produk, gudang, pengaju, dan reviewer.

Odoo menjalankan setiap metode `test_...` dalam savepoint, yaitu batas pemulihan di dalam transaksi. Perubahan suatu metode dikembalikan setelah selesai agar metode berikutnya tidak menerima hasil samping tes sebelumnya. Transaksi class ditutup tanpa commit. Jangan menambahkan `self.env.cr.commit()` sebagai cara membuat tes lolos. Perilaku ini berasal dari implementasi resmi TransactionCase Odoo 19.

Tag `post_install` meminta tes dijalankan setelah pemasangan modul, sedangkan `-at_install` mengeluarkannya dari fase tes saat pemasangan masing-masing modul. Tag `supplyflow` memberi penanda tambahan pada sebagian class; script proyek memilih addon melalui `--test-tags /supplyflow_office`.

Fixture harus mewakili pengguna sebenarnya

`setUpClass()` permintaan membuat requester, approver, peer, outsider, reviewer yang hanya punya sebagian hak native, dan akun multi-perusahaan. `new_test_user()` mengisi kelompok secara eksplisit. `with_user(...)` kemudian menjalankan metode sebagai akun yang diuji.

Helper `_values()` menyiapkan nilai permintaan, `_request()` membuatnya sebagai requester, dan `_approved_request()` menjalankan submit serta persetujuan akun lain. Helper tersebut mengurangi pengulangan persiapan. Isi assertion tetap menjelaskan aturan yang diuji.

Checkpoint: cari satu pemanggilan `with_user(self.approver)` dan bedakan dari environment awal tes. Tanpa pergantian pengguna yang benar, tes izin dapat berjalan sebagai akun yang terlalu berkuasa dan memberi rasa aman yang keliru.

3. Uji alur permintaan dan penolakannya

test_purchase_request.py mempunyai 17 metode tes. Baca `test_request_to_native_rfq_uses_company_warehouse_and_estimate()` terlebih dahulu. Tes ini memeriksa status RFQ draft, pemasok, perusahaan, mata uang, gudang incoming, origin, produk, satuan, kuantitas, estimasi, serta metadata persetujuan.

Tes pengulangan aksi menggunakan fungsi lengkap berikut di dalam class:

python

```
def test_rfq_conversion_is_idempotent(self):
    request = self._approved_request()
    first = request.with_user(self.approver).action_create_rfq()
    second = request.with_user(self.approver).action_create_rfq()
    self.assertEqual(first["res_id"], second["res_id"])
    orders = self.env["purchase.order"].search([("origin", "=", request.name)])
    self.assertEqual(orders, request.purchase_order_id)
```

Idempoten berarti pengulangan aksi menghasilkan dokumen hasil yang sama, bukan RFQ tambahan. Dua panggilan di atas berjalan berurutan. Kasus klik bersamaan masih membutuhkan tes HTTP terpisah pada bagian berikutnya.

Tes persetujuan independen berisi potongan berikut:

python

```
def test_approval_needs_an_independent_reviewer(self):
    own = self._request(user=self.approver)
    own.action_submit()
    with self.assertRaises(AccessError):
        own.action_approve()
    with self.assertRaises(AccessError):
        own.write({"decision_reason": "I approve my own request"})
```

`assertRaises(AccessError)` mengharapkan operasi ditolak. Jika operasi tiba-tiba berhasil, tes gagal. Bagian lanjutan pada sumber memeriksa requester/peer yang tidak boleh mengambil keputusan, lalu approver independen yang berhasil.

Odoo membungkus `assertRaises` miliknya dengan pengelolaan environment/savepoint. Pada tes addon ini gunakan satu kelas exception yang diharapkan pada setiap context. Polanya dapat dibaca pada BaseCase Odoo 19. Validator arsip memakai `unittest.TestCase` standar sehingga perilaku helper-nya tidak harus disamakan dengan helper Odoo.

Metode lain yang perlu ditelusuri:

Metode tes	Mengapa diperlukan
`test_rfq_preserves_estimate_even_when_vendor_prices_are_configured`	Harga vendor native dapat menghitung ulang baris; estimasi yang disetujui, termasuk nol, perlu tetap dipindahkan sesuai rancangan

Metode tes	Mengapa diperlukan
`test_reject_and_revision_preserve_chatter_then_require_new_approval`	Revisi setelah penolakan harus ditinjau ulang dan riwayat lama tetap dapat ditelusuri
`test_direct_state_audit_and_link_changes_are_blocked`	Pengguna tidak boleh melompati workflow melalui `write()` pada state/audit/tautan
`test_context_defaults_cannot_forge_approval_or_link`	Context RPC tidak boleh menyisipkan persetujuan saat create
`test_submitted_parent_and_every_line_mutation_path_are_frozen`	Perlindungan harus menutup perubahan parent, baris, penghapusan, dan pemindahan relasi
`test_standard_purchase_rights_are_still_required`	Kelompok Approver addon tidak otomatis menjadi izin membuat dokumen Purchase
`test_company_record_rules_hide_requests_and_lines`	Isolasi perusahaan berlaku pada pencarian, pembacaan ID, perubahan, dan baris
`test_numeric_edits_are_validated_before_database_conversion`	Input non-finite atau terlalu besar perlu ditolak sebelum konversi field/SQL merusak alur error
`test_workflow_actions_have_no_state_shortcuts`	Metode harus menolak urutan tindakan yang tidak sesuai status
`test_approved_request_can_only_be_cancelled_by_an_independent_approver`	Pembatalan setelah persetujuan juga memiliki batas pelaku

Sumber lengkap juga memeriksa kepemilikan draft, perusahaan vendor/produk/gudang, baris kosong/invalid/non-stok, dan akun tanpa kelompok SupplyFlow. Baca alasan kegagalan assertion sebelum mengubah implementasi atau melemahkan tes.

4. Uji kasus receipt dan perubahan stok sebenarnya

test_supplier_issue.py mempunyai 20 metode. Helper `_make_receipt()` membuat incoming picking native, mengonfirmasi, mengisi jumlah aktual, dan memvalidasi. Fixture menolak melanjutkan jika receipt belum done.

Pada Odoo 19, fixture ini menulis `quantity` dan `picked` pada move. Jangan mengganti contoh dengan nama field dari versi Odoo lain tanpa memeriksa sumber versi yang dipakai.

Di dalam `test_independent_approval_creates_native_return_without_moving_stock()`, bagian setelah pembuatan retur memeriksa:

python

```
self.assertNotEqual(returned.state, "done")
self.product.invalidate_recordset(["qty_available"])
self.assertEqual(self.product.qty_available, before)
returned.move_ids.write({"quantity": 3, "picked": True})
returned.button_validate()
self.assertEqual(returned.state, "done")
self.product.invalidate_recordset(["qty_available"])
self.assertEqual(self.product.qty_available, before - 3)
self.assertEqual(issue.return_state, "done")
```

Tes membedakan membuat rencana transfer dari memvalidasi perpindahan barang. `invalidate_recordset` meminta pembacaan nilai stok kembali ketika assertion memerlukan hasil terbaru. Pemeriksaan tidak sekadar membandingkan teks status kasus.

Contoh batas jumlah retur yang masih menunggu:

python

```
    def test_existing_pending_returns_reserve_their_quantity(self):
        first = self._issue(7)
        second = self._issue(4)
        self._approve(first)
        self._approve(second)
        first.with_user(self.approver).action_create_return()
        with self.assertRaises(UserError):
            second.with_user(self.approver).action_create_return()
        self.assertFalse(second.return_picking_id)
        self.assertEqual(second.state, "approved")
```

Fixture menerima 10 unit. Retur 7 yang sudah dibuat memakai kapasitas sumber itu walaupun belum done. Retur kedua sejumlah 4 harus ditolak, dan kasus kedua tetap approved tanpa tautan retur. Tes terpisah memeriksa retur native di luar addon dan retur yang dibatalkan.

Kelompok pemeriksaan lain pada file yang sama meliputi:

- Pembuatan retur idempoten, reviewer independen, larangan pengaju menyetujui kasusnya sendiri, dan hak pembatalan.
- Field state/audit/link, default context, detail receipt turunan, context wizard, serta revisi sumber yang tidak boleh dipalsukan.
- Parent/baris yang membeku sesudah pengajuan, pemilik draft, dan pemindahan baris ke parent lain.
- Receipt incoming selesai, kecocokan perusahaan, sumber stock move, angka positif/finite, serta batas kuantitas.
- Shortage yang harus diselesaikan dengan catatan tanpa retur fisik.

Checkpoint: ikuti satu tes negatif dari input sampai assertion. Catat kondisi awal, aktor yang memanggil metode, exception yang diharapkan, lalu status/tautan yang harus tetap utuh sesudah penolakan.

5. Uji integrasi seluruh distribusi dan angka workspace

test_distribution_flow.py mempunyai dua skenario. `_purchase_for()` menjalankan permintaan custom sampai RFQ, lalu mengonfirmasi PO melalui metode Purchase. `_finish_picking()` memproses picking melalui metode Inventory. `_stock_at()` membaca jumlah `stock.quant` pada lokasi gudang untuk assertion; helper itu tidak menulis quant untuk memalsukan hasil.

`test_approved_purchase_partial_receipt_return_and_customer_delivery()` memeriksa rangkaian berikut:

text

```
Permintaan 20 → persetujuan → RFQ/PO
  → receipt pertama 12 + wizard backorder 8
  → receipt backorder selesai: stok 20
  → rencana retur 2: stok masih 20
  → retur divalidasi: stok 18
  → sales order/delivery 7 divalidasi: stok 11
```

Tes memakai `Form.from_action(...).save()` untuk wizard keputusan backorder yang benar-benar dihasilkan Odoo. Artinya sisa 8 tidak dipalsukan dengan membuat baris stok secara terpisah. Purchase line, receipt move, return move, sale line, dan delivery diperiksa hubungannya.

`test_selected_receiving_warehouse_then_native_internal_transfer()` menerima 10 pada gudang B, memindahkan 4 ke A melalui internal picking, lalu memeriksa A=4, B=6, dan total produk tetap 10.

test_workspace.py memiliki tiga pemeriksaan:

Metode	Hasil yang diperiksa
`test_empty_workspace_uses_real_counts`	Workspace kosong mengembalikan nol dan daftar kosong
`test_unassigned_user_cannot_call_workspace`	Akun tanpa kelompok SupplyFlow ditolak
`test_counts_follow_request_workflow`	Permintaan submitted menambah pending menjadi 1; pembatalan mengembalikannya menjadi 0

Jumlah workspace berasal dari record yang dibuat pada tes. Tidak ada angka statistik contoh yang diperlakukan sebagai hasil database.

6. Jalankan ORM pada database terisolasi

Dari folder latihan:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Test.ps1
```

Test.ps1 melakukan urutan berikut:

1. Membaca/menyiapkan konfigurasi lokal melalui Common.ps1.
2. Pull image kecuali `-SkipPull`, lalu menyalakan PostgreSQL milik proyek.
3. Menghentikan layanan Odoo SupplyFlow yang biasa agar proses maintenance tidak berjalan bersamaan.
4. Membuat database unik `supplyflow_test_` diikuti GUID, dengan pemilik role aplikasi `supplyflow`.
5. Menjalankan container satu kali untuk instalasi addon serta tes.
6. Dalam `finally`, menghapus database/filestore uji, kecuali `-KeepDatabase` dipilih.

Container tes memakai compose.test.yaml, `--no-http`, `--max-cron-threads=0`, `--stop-after-init`, `--without-demo=True`, `-i supplyflow_office`, `--test-enable`, dan `--test-tags /supplyflow_office`. Overlay mematikan healthcheck untuk job yang memang tidak melayani HTTP.

`Test.ps1` meninggalkan aplikasi Odoo biasa dalam keadaan berhenti; script tidak menyalakannya kembali otomatis. PostgreSQL tetap berjalan. Sesudah tes, gunakan `Start.ps1` bila ingin membuka UI lagi:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Start.ps1 -WithTrainingData -SkipPull
```

Gunakan `-SkipPull` pada Test hanya jika image yang dipatok sudah tersedia. Untuk penyelidikan lokal:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Test.ps1 -SkipPull -KeepDatabase
```

Nama database yang ditahan dicetak ke terminal. Pilihan tersebut menahan database hasil instalasi/tes untuk inspeksi; transaksi fixture tes tetap mengikuti rollback `TransactionCase`. Hindari menjalankannya berkali-kali tanpa membersihkan hasil yang ditahan. Script clean_test_filestore.py memeriksa pola nama database uji sebelum membersihkan folder filestore.

7. Uji validator arsip tanpa PostgreSQL

test_backup_archive.py memakai `unittest.TestCase` standar, bukan environment Odoo. File mengimpor restore.py untuk memanggil `validate_archive()`. `setUp()` membuat folder sementara dan mendaftarkan pembersihannya.

`make_archive()` membangun ZIP kecil dengan dump sintetis berheader `PGDMP`, satu filestore sintetis, dan manifest. Data tersebut hanya cukup untuk menguji validator, bukan database PostgreSQL yang dapat direstore.

Contoh tes path traversal:

python

```
    def test_reject_parent_path(self):
        self.assert_invalid_path("../escape")
```

Helper `assert_invalid_path()` menyisipkan nama path itu ke ZIP; `assert_invalid()` mengharapkan validator menolak. Tes lain mengganti payload/hash/ukuran/versi, membuat symlink, nama duplikat, konflik direktori/berkas, atau header dump tidak valid.

Jalankan:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
python -m unittest discover -s tests -p test_backup_archive.py
```

Perintah ini tidak menyalakan server dan tidak memulihkan database. Keberhasilan 25 tes validator harus dilengkapi latihan backup/restore nyata. Latihan proyek yang sudah dilakukan membandingkan fingerprint 16 tabel serta lampiran filestore pada stack terpisah; rincian buktinya ada pada VERIFICATION.md.

8. Siapkan browser test

Node pada repo dipakai untuk verifikasi browser. Server ERP tetap Odoo. package.json mematok `@playwright/test` 1.64.1 dan menyediakan dua script:

json

```
  "scripts": {
    "test:browser": "playwright test --grep-invert @capture",
    "screenshots": "playwright test --grep @capture"
  },
```

playwright.config.js memakai satu worker, base URL `http://127.0.0.1:8070`, viewport desktop 1440×960, dan Microsoft Edge secara default. CI memilih Chromium melalui `PLAYWRIGHT_CHANNEL`. `SUPPLYFLOW_URL` dapat mengganti base URL untuk instance latihan yang sengaja dipilih.

Siapkan instance latihan dan dependensi:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Start.ps1 -WithTrainingData -SkipPull
npm ci
npm run test:browser
```

Data latihan menyediakan akun buyer, approver, warehouse, serta index ID dokumen. Password dibuat lokal dan dibaca helper dari `.secrets`; isi file kredensial tidak dicetak dalam tutorial. Suite workflow membuat dokumen verifikasi tambahan. Jalankan pada instance latihan proyek, bukan database operasional berisi transaksi pengguna.

Jika memilih Chromium pada komputer yang belum memilikinya, pasang runtime yang diperlukan dan set channel sebelum tes:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
npx playwright install chromium
$env:PLAYWRIGHT_CHANNEL = "chromium"
npm run test:browser
Remove-Item Env:PLAYWRIGHT_CHANNEL
```

9. Baca helper login, RPC, dan tombol

helpers.js menyediakan:

Fungsi	Tugas
`credentials(role)`	Membaca password admin atau akun latihan dari file lokal yang diabaikan Git
`login(page, role)`	Mengisi form login dan menunggu navbar ERP terlihat
`rpc(page, model, method, args, kwargs)`	Memanggil `call_kw`, memeriksa error JSON-RPC, lalu mengembalikan result
`clickWorkflow(...)`	Mengklik tombol, mengonfirmasi dialog bila diminta, dan menunggu respons metode bisnis yang sesuai
`authenticateApi(request, role)`	Login context HTTP tanpa membuka SPA browser
`trainingIndex(page)`	Membaca index ID fixture dari parameter latihan
`openAction(page, action)`	Membuka route action Odoo dan menunggu action manager
`openRecord(page, action, id)`	Membuka satu record dan menunggu form

`rpc()` memeriksa `body.error`, karena respons HTTP yang berhasil dikirim tetap dapat membawa error bisnis JSON-RPC. Assertion tidak cukup berhenti pada status HTTP 200.

Bagian awal `clickWorkflow()` memasang penunggu sebelum klik:

javascript

```
export async function clickWorkflow(page, model, method, buttonName, { confirm = false } = {}) {
    const [response] = await Promise.all([
        page.waitForResponse((candidate) => {
            const request = candidate.request();
            if (request.method() !== "POST" || !new URL(candidate.url()).pathname.startsWith("/web/dataset/call_butt
  >on")) return false;
            try {
```

```
                const params = request.postDataJSON()?.params;
                return params?.model === model && params?.method === method;
            } catch {
                return false;
            }
        }),
```

Bagian berikut pada fungsi lengkap mengklik tombol berdasarkan accessible name, menerima dialog jika perlu, memeriksa respons JSON-RPC, lalu menunggu field penanda pada dokumen native yang dibuka. Purchase order ditunggu melalui `order_line`, picking melalui `move_ids`.

Tujuannya agar assertion setelah klik tidak mendahului penyelesaian request bisnis. Jangan menggantinya dengan jeda tetap beberapa detik. Halaman yang terlihat cepat pada satu komputer belum menjamin transaksi sudah selesai saat komputer lain lebih lambat.

10. Telusuri sembilan workflow

workflows.spec.js berisi urutan berikut:

Tes	Pemeriksaan
`workspace counts, filters and keyboard navigation`	Angka sesuai RPC, empat filter, All requests, nomor terbaru, fokus/Tab/Enter/Escape, dan error JavaScript
`workspace recovers from RPC failure and fits a narrow screen`	Loading yang dikendalikan, Refresh disabled, respons kosong, error, retry, dan viewport 390 px
`purchase request QWeb produces a real PDF`	Endpoint laporan memberi MIME PDF, signature `%PDF-`, dan isi nonkosong
`requester creates a request through the form and withdraws it`	Pemilihan pemasok/gudang/produk, kuantitas/harga, submit, dan konfirmasi withdraw
`requester cannot approve, independent approver creates one native RFQ`	Sesi pengaju ditolak; sesi reviewer lain menyetujui serta membuka RFQ native tunggal
`supplier review creates return without completing warehouse transfer`	Tindakan review/retur dan pembukaan picking native tertaut
`rejection, revision and approval cancellation work through form buttons`	Alasan penolakan, revisi, pengajuan baru, persetujuan, dan pembatalan
`shortage resolution and issue cancellation work through form buttons`	Penyelesaian shortage tanpa picking dan varian pembatalan pelapor/Approver
`separate HTTP sessions cannot create duplicate RFQs or excess returns`	Panggilan bersamaan pada transaksi HTTP terpisah

Periksa judul dan isi tes bersama. Beberapa skenario menyiapkan data lewat RPC, kemudian memeriksa tindakan lewat UI. Itu berbeda cakupan dari mengetik semua input pada form. Tes permintaan keempat memang mengisi form dari New. Tes kondisi kosong/error sengaja mengganti respons RPC sementara; respons sintetis itu hanya fixture UI yang diberi label, bukan hasil statistik bisnis sebenarnya.

Bagian PDF pada tes memeriksa:

javascript

```
    const response = await page.request.get(`/report/pdf/supplyflow_office.report_request/${index.request_approved}`
>);
    expect(response.ok()).toBe(true);
    expect(response.headers()["content-type"]).toContain("application/pdf");
    const pdf = await response.body();
    expect(pdf.subarray(0, 5).toString()).toBe("%PDF-");
    expect(pdf.length).toBeGreaterThan(2000);
```

Endpoint ini membuktikan PDF benar-benar dihasilkan. Pemeriksaan itu belum menjadi audit tata letak seluruh panjang purpose, jumlah baris, atau variasi nama pemasok. PDF lokal juga disimpan di `.local/request-summary.pdf`, yang tidak masuk Git.

Mengapa perlu sesi HTTP terpisah untuk konkurensi?

Dua panggilan berurutan hanya membuktikan pengulangan. Uji konkurensi membuat empat context HTTP independen, login sesuai peran, lalu mengirim dua aksi pada waktu yang berdekatan:

javascript

```
        const results = await Promise.allSettled([
            rpc(reviewerOne, "supplyflow.request", "action_create_rfq", [[id]]),
            rpc(reviewerTwo, "supplyflow.request", "action_create_rfq", [[id]]),
        ]);
```

Setelah respons selesai, tes membaca dokumen hasil dan menghitung purchase order berdasarkan origin. Harus ada tepat satu RFQ. Setiap respons yang berhasil harus menunjuk ID RFQ tertaut yang sama. Tes tidak mensyaratkan kedua request selalu berhasil; penolakan lock/transaksi tertentu dapat terjadi, sedangkan invariant satu dokumen harus tetap terpenuhi.

Bagian retur membuat dua kasus dengan jumlah masing-masing lebih dari separuh sisa sumber, lalu memanggil `action_create_return` melalui reviewer context berbeda. Tepat satu keberhasilan dan satu kasus bertaut picking diperiksa. Picking verifikasi dibatalkan setelahnya agar reservasi uji tidak terus memakai jatah stok latihan.

11. Ambil screenshot dari aplikasi yang berjalan

screenshots.spec.js mempunyai fungsi `capture(page, name)`. Fungsi menunggu loading indicator hilang, lalu menyimpan screenshot `fullPage` pada `docs/screenshots`. Tes `@capture real ERP screens` mengunjungi delapan keadaan:

1. Operations desktop.
2. Daftar purchase requests.
3. Permintaan approved.
4. Purchase order/RFQ native tertaut.
5. Kasus supplier approved.
6. Receipt native selesai.
7. Sales order native.
8. Operations pada viewport 390×844.

Perintahnya:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
npm run screenshots
```

Perintah menulis ulang gambar pada `docs/screenshots`. Periksa setiap hasil sebelum commit: teks terbaca, data latihan sesuai, layar bukan loading/error, dan tidak ada kredensial/data pribadi yang ikut tampil. Screenshot membuktikan keadaan layar yang ditangkap; aturan bisnis tetap dibuktikan oleh assertion model dan workflow.

12. Cari error dari lapisan yang paling dekat

Dari folder latihan, perintah inspeksi awal:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
docker compose ps
docker compose logs --tail=150 odoo
docker compose logs --tail=100 postgres
```

`ps` menunjukkan layanan berjalan/sehat. Log Odoo memuat traceback Python, pemuatan XML, dan request laporan. Log PostgreSQL membantu ketika koneksi database gagal. Jangan menyalin isi `.secrets` atau `.runtime/odoo.conf` ke issue publik saat meminta bantuan.

Gejala	Tempat yang diperiksa	Langkah berikutnya
`Field ... does not exist` saat update	Model, import `models/__init__.py`, dan XML field	Pastikan model final sudah dimuat sebelum view final; periksa nama field
External ID tidak ditemukan	Manifest dan `ref` XML/action	Periksa nama addon, ID, serta urutan file `data`
Menu tidak tampil	`groups.xml`, kelompok akun, ACL, dan menu	Pastikan akun mendapat kelompok addon; jangan mengatasinya dengan `sudo()` menyeluruh
Tombol disembunyikan	`invisible`, `groups`, status, dan pengaju	Bandingkan aktor/status dengan rancangan, lalu periksa guard Python
Tombol ada tetapi AccessError/UserError	Metode model dan respons RPC	Baca pesan bisnis; periksa hak native Purchase/Inventory dan perusahaan aktif
Workspace kosong padahal ada data	`get_overview`, context perusahaan, state dokumen	Pahami bahwa metrik hanya menghitung status tertentu dan record yang boleh dibaca
Workspace menampilkan error	Network `get_overview`, Console, log Odoo	Cari error asli server; gunakan Try again setelah penyebabnya selesai
`Missing template` atau action tidak ditemukan	Manifest assets, `t-name`, `static template`, key registry	Cocokkan nama, update addon, lalu muat ulang asset
Perubahan XML tidak terlihat	Hasil update addon dan cache browser	Hentikan Odoo, jalankan `manage.py update`, nyalakan ulang, lalu refresh
PDF 500	Traceback QWeb, `report_name`, dan template	Periksa field yang dirujuk serta pola span di dalam td
Playwright timeout setelah klik	Network `call_button`, hasil bisnis, locator form native	Pastikan respons metode selesai; gunakan locator field yang sesuai mode readonly/edit

Gejala	Tempat yang diperiksa	Langkah berikutnya
Tes ORM melaporkan nol tes	Import tes, tag, dan parameter runner	Cocokkan `tests/__init__.py` serta `--test-tags /supplyflow_office`
Browser tidak dapat login setelah ORM	Status container Odoo	`Test.ps1` menghentikan Odoo; jalankan kembali Start dengan data latihan
Halaman pertama lama memuat	Log pemuatan asset dan sumber daya komputer	Tunggu bukti pemuatan/health; jangan langsung menghapus volume data

Field relasi yang sedang editable dapat menaruh nama pemasok pada nilai `<input>`, sedangkan field readonly dapat menampilkannya sebagai teks/link. Assertion browser harus membaca elemen yang sesuai. Mengubah assertion dari nilai ke teks tanpa memeriksa DOM dapat menyebabkan tes gagal meskipun tindakan bisnis berhasil.

Untuk menjalankan satu kelompok browser saat debugging, panggil CLI langsung dengan filter judul:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office'
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
node .\node_modules\@playwright\test\cli.js test --grep 'requester cannot approve'
```

Sesudah error diperbaiki, jalankan kembali skenario yang terdampak beserta pemeriksaan lain yang relevan. Jangan mengubah assertion menjadi lebih longgar hanya agar hasilnya hijau.

13. Susun urutan CI yang dapat diulang

.github/workflows/verify.yml berjalan pada push dan pull request dengan izin repository `contents: read`. Job pada runner Ubuntu baru melakukan:

1. Checkout sumber dan setup Node 22 dengan cache npm.
2. `npm ci` dari lockfile.
3. Jalankan 25 validator arsip melalui unittest.
4. Pasang Chromium beserta dependensi sistem Playwright.
5. Jalankan `Test.ps1` untuk instalasi/database unik serta tes ORM.
6. Jalankan `Start.ps1 -WithTrainingData -SkipPull` untuk instance browser.
7. Jalankan `npm run test:browser` dengan channel Chromium.
8. Jika gagal, cetak log container dan unggah `playwright-report/test-results` dengan retensi tiga hari.

Trace browser disimpan pada kegagalan di CI. Lokal memakai trace off secara default untuk mengurangi beban komputer. Kedua lingkungan memakai satu worker. Runtime lokal Windows/Edge dan CI Ubuntu/Chromium berbeda, sehingga keberhasilan keduanya memberi bukti pada dua lingkungan yang benar-benar dicoba, bukan jaminan seluruh browser/platform.

Run verifikasi implementasi telah lulus untuk 42 tes ORM, 25 tes arsip, dan sembilan workflow Chromium. Saat mengikuti tutorial pada commit baru, baca hasil run commit tersebut. Hasil lama tidak otomatis membuktikan perubahan baru.

Checkpoint akhir: jelaskan satu tes positif, satu penolakan izin, satu assertion stok, dan satu tes konkurensi dengan kata-kata sendiri. Untuk setiap tes, sebutkan kondisi awal, tindakan, hasil yang wajib terjadi, serta hasil yang wajib tidak terjadi. Lanjutkan ke 07. Operasi dan latihan.

07. Mengisi data latihan, menjaga data, dan mencoba satu alur lengkap

Bab ini memakai addon lengkap setelah bab 05. Tujuannya memahami mengapa data contoh, backup, dan restore mempunyai jalur sendiri, kemudian mencoba satu transaksi dari permintaan sampai stok benar-benar berpindah. Proses startup dasar sudah dibahas pada bab 01.

Untuk menulis ulang, gunakan terminal latihan yang sama dengan bab sebelumnya:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
```

Nama proyek Compose memisahkan container dan volume latihan dari `supplyflow-office` utama, meskipun nama database di dalamnya sama-sama `supplyflow`. Port latihan masih 8070; proyek utama harus berhenti agar tidak berebut port. Pada terminal baru, set `COMPOSE_PROJECT_NAME` lagi sebelum menjalankan script. Semua contoh operasi di bab ini ditujukan ke folder latihan tersebut.

1. Mengapa seed dijalankan melalui shell Odoo?

Buka manage.py. Fungsi `run_shell(filename)` membaca script dari `/opt/supplyflow`, lalu mengirim isinya ke proses odoo `shell`. **Fungsi lengkap dari sumber**:

python

```
def run_shell(filename):
    code = pathlib.Path("/opt/supplyflow", filename).read_text()
    subprocess.run(["odoo", "shell", "--config", CONFIG, "--no-http"], input=code, text=True, check=True)
```

Shell Odoo menyediakan env: environment berisi cursor database, pengguna, context, dan registry model yang sudah dimuat. Karena itu `env["supplyflow.request"]` mengenali model custom. Menjalankan `training_data.py` dengan Python Windows biasa tidak menyediakan environment tersebut.

`main()` pada `manage.py` memilih tiga mode. `initialize` memeriksa apakah addon sudah terpasang, memasangnya bila perlu, lalu menjalankan konfigurasi instance. `update` memperbarui addon dari source. `seed` memanggil `run_shell("training_data.py")`. Ketiganya tidak saling menggantikan: seed memerlukan model final yang sudah terpasang.

Pada Load-TrainingData.ps1, script menyiapkan konfigurasi, menyalakan PostgreSQL, membaca apakah Odoo sedang berjalan, lalu menghentikannya sebelum seed. Blok `finally` menyalakan Odoo lagi hanya jika sebelumnya berjalan. Jika sebelumnya berhenti, script seed tidak mengubahnya menjadi layanan yang terus berjalan.

Setelah pemasangan lengkap bab 05, pilih salah satu:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
# Memuat latihan sambil menyalakan aplikasi yang lengkap.
.\scripts\Start.ps1 -WithTrainingData -SkipPull
```

Atau, jika instance lengkap sudah ada dan hanya ingin memuat dataset opsional:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Load-TrainingData.ps1
```

`-SkipPull` hanya dipakai jika image yang dipatok sudah tersedia lokal. Seed tidak dipilih oleh instalasi biasa. Jangan memanggilnya pada checkpoint addon minimal bab 02.

2. Ikuti isi `training_data.py`

Buka file lengkap. `seed_training(environment)` menerima environment Odoo dan tidak mengembalikan dataset ke browser. Hasilnya berupa record database, external ID, parameter penanda, dan pesan status terminal.

Langkah pertama memeriksa `supplyflow.training_completed`. Jika nilainya `1`, fungsi selesai tanpa memuat ulang. Selanjutnya fungsi mensyaratkan `supplyflow.initialized`, mengunci perusahaan, dan memeriksa mata uang IDR. Tiga password latihan dibaca dari secret lokal, harus berbeda dan masing-masing sedikitnya 16 karakter. Password tidak ditulis tetap pada source.

Environment lalu memakai akun admin untuk persiapan dataset dengan context tanpa undangan/reset password melalui email. Record sintetis memakai penanda `[Training]` dan partner tidak diberi alamat email. Saat membuat dokumen custom, script berganti ke akun buyer atau warehouse; saat menyetujui, ia berganti ke approver. Jadi contoh tetap melalui metode bisnis pengguna yang berbeda.

External ID dan tiga helper kecil

External ID adalah nama stabil yang menunjuk ID database, misalnya `supplyflow_training.product`. Angka ID database dapat berbeda antarinstalasi. Namespace `supplyflow_training` di sini dibuat lewat `ir.model.data`; ia bukan addon kedua yang harus dipasang.

Helper di dalam `seed_training()`	Masukan	Hasil dan alasan
`existing(name, model)`	Nama latihan dan model yang diharapkan	Mengambil record dari external ID; menolak jika ID menunjuk model lain
`bind(name, record)`	Nama stabil dan satu record	Menyimpan/memeriksa `ir.model.data`, mencatat ID pada dictionary `identifiers`, lalu mengembalikan record
`ensure(name, model, create)`	Identitas, model, fungsi pembuat	Menggunakan record yang ada atau memanggil fungsi pembuat, kemudian mengikat identitasnya

Fungsi lengkap kecil dari sumber:

python

```
def ensure(name, model, create):
    record = existing(name, model)
    return bind(name, record or create())
```

`create` pada parameter adalah fungsi yang belum dijalankan. Contoh `lambda: make_product(...)` baru dipanggil jika record belum ditemukan. Helper pembuat produk, gudang, dan akun juga memeriksa tabrakan kode/login dengan data di luar dataset; ia berhenti daripada mengambil alih akun yang kebetulan bernama `buyer`.

Pada loop pembuat akun, `def create_user(role=role)` menangkap nilai peran untuk iterasi tersebut. Grup buyer mencakup Purchase dan Inventory; approver menambah kelompok Approver; warehouse mempunyai

Inventory dan peran Sales untuk latihan penjualan. Hak native tersebut berbeda dari hak addon.

Idempotensi berarti menjalankan loader lagi tidak menggandakan dataset yang sudah selesai atau mereset password akun latihan. Loader ini bukan tombol reset: setelah penanda selesai ada, ia juga tidak mengembalikan transaksi yang telah kamu ubah atau memperbaiki record latihan yang dihapus. Jangan menghapus parameter penanda untuk memaksa reset instance yang sedang dipakai.

Data dibuat lewat transaksi bisnis

Helper `make_warehouse()` membuat dua gudang satu langkah. `make_product()` membuat produk storable dengan kode latihan. `create_request()`, `submit()`, dan `approve()` membuat permintaan, mengajukan sebagai buyer, lalu menyetujui sebagai akun lain. Script mengonfirmasi PO native dan menyelesaikan receipt dengan jumlah aktual. Ia tidak mengisi saldo stok memakai angka langsung pada `stock.quant`.

Potongan kode asli saat menyelesaikan receipt:

python

```
if receipt.state != "done":
    for move in receipt.move_ids.filtered(lambda move: move.state not in ("done", "cancel")):
        move.write({"quantity": move.product_uom_qty, "picked": True})
    receipt.with_user(admin).button_validate()
```

Pada dataset baru, receipt awal berisi kertas 120, folder 80, dan pena 200. Transfer internal memindahkan 10 folder ke gudang cabang. Sales order kertas 10 dan pena 30 dikonfirmasi, tetapi delivery belum divalidasi. Pemesanan/reservasi tidak sama dengan pengurangan on-hand. Ada juga permintaan submitted/approved serta kasus masalah penerimaan submitted/approved untuk bahan penjelasan.

`create_issue()` menghubungkan kasus ke move kertas pada receipt selesai. `create_branch_transfer()` memakai transfer native antargudang. Pada akhir proses, indeks ID disimpan di `supplyflow.training_json`, penanda selesai ditulis, lalu `environment.cr.commit()` menyimpan transaksi. Blok paling bawah memanggil `env.cr.rollback()` jika seed melempar exception. Nama nomor dokumen tidak perlu berurutan tanpa celah setelah percobaan yang gagal.

3. Buat satu transaksi baru dari awal sampai selesai

Gunakan label unik, misalnya `[Latihan] Pembelian kertas sesi 1`, agar tidak bergantung pada keadaan fixture lama. Pada proyek utama, sebagian data latihan mungkin sudah berubah saat pengujian atau demonstrasi. Jalankan transaksi baru ini hanya pada instance latihan.

1. Login `warehouse`. Catat on-hand kertas di Main warehouse sebagai nilai **B**. Periksa juga gudang tujuan yang dipakai, supaya stok cabang tidak tercampur dalam perbandingan.
2. Login `buyer`. Buka **SupplyFlow Office > Purchase requests > New**. Pilih `[Training] Paper supplier`, Main warehouse, tanggal kebutuhan, dan purpose unik tadi. Tambahkan A4 paper sebanyak 5 dengan estimasi 45.000. Simpan dan pilih **Submit for Approval**.
3. Login `approver`. Buka permintaan itu, tinjau datanya, lalu **Approve**. Buyer tidak menyetujui permintaannya sendiri. Pilih **Create RFQ**, catat nomor RFQ, lalu buka ulang untuk membuktikan tautannya menuju dokumen yang sama.
4. Sebagai petugas Purchase yang berwenang, periksa vendor, gudang incoming, harga, dan pajak RFQ. Konfirmasi PO. Jika aturan native meminta approval PO tambahan, gunakan pengguna berwenang; persetujuan SupplyFlow tidak melewati aturan Purchase.
5. Login `warehouse`, buka receipt dari PO, masukkan jumlah aktual 5 dan validasi. Untuk skenario penerimaan penuh ini, tidak ada sisa yang perlu dijadikan backorder. On-hand Main warehouse menjadi **B + 5** jika tidak ada

transaksi lain.

6. Masih sebagai `warehouse`, buat **Supplier issues** untuk receipt tersebut: alasan damaged goods, uraian, resolver approver, tanggal, move kertas, jumlah terdampak 1. Simpan lalu **Submit for review**.
7. Login `approver`, **Approve supplier return**, kemudian **Create return transfer**. Pastikan jumlah rencana 1 dan receipt sumber benar. On-hand masih **B + 5** sebelum validasi retur.
8. Login `warehouse`, buka retur tertaut, periksa jumlah aktual 1 lalu validasi. On-hand menjadi **B + 4**. Kasus menyimpan `state=return_created`; status transfer terkait menjadi done.
9. Dengan akun Sales yang berwenang, buat quotation pelanggan `[Training] Office customer`, gudang Main, kertas 2, dan harga yang ditinjau. Konfirmasi order. Pada akun latihan warehouse, hak Sales tersedia untuk latihan ini.
10. Buka delivery dari sales order, periksa ketersediaan serta jumlah 2, lalu validasi. On-hand menjadi **B + 2**: penerimaan 5, retur 1, delivery 2. Nomor permintaan, PO, receipt, kasus, retur, SO, dan delivery memberi jejak dokumen untuk menjelaskan perubahan itu.

Catat hasil aktual, bukan hanya menyalin rumus. Bila stok berbeda, periksa lokasi, status done, transaksi lain, dan satuan. Screenshot hasil latihan adalah gambar aplikasi nyata; jangan menampilkan password atau informasi pribadi pada gambar.

4. Mengapa backup berisi database dan filestore?

PostgreSQL menyimpan record seperti permintaan, baris, picking, pengguna, dan metadata lampiran. Filestore menyimpan isi berkas lampiran. Database yang dipulihkan tanpa filestore yang cocok dapat menampilkan metadata lampiran sementara isi berkasnya hilang.

Backup.ps1 menerima `-OutputDirectory` opsional; default hasilnya folder `.backups` pada root repo. Nama ZIP memakai waktu UTC dan suffix acak. PostgreSQL harus sudah berjalan. Script membaca apakah Odoo sedang berjalan, menghentikan penulisan aplikasi, lalu membuat container satu kali untuk menjalankan backup.py.

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Backup.ps1
```

`backup.py` melakukan pemeriksaan koneksi/database/versi, memeriksa tidak ada koneksi aplikasi lain dengan role yang sama, menginventarisasi filestore, menjalankan `pg_dump`, lalu menulis arsip. `add_file()` membaca berkas per chunk, menulisnya ke ZIP, dan menghitung ukuran serta SHA-256. Output baru dibuka memakai mode `xb`, sehingga berkas yang sudah ada tidak ditimpa.

Isi ZIP	Kegunaan
`database.dump`	Dump custom PostgreSQL 17, termasuk data bisnis dan hash password pengguna
`filestore/...`	Isi lampiran dari database yang sama
`manifest.json`	Format/version, nama database, seri Odoo, versi addon, versi PostgreSQL, waktu, serta path/ukuran/SHA-256 payload

Arsip dibuat di `/tmp` container lalu disalin ke Windows dengan `docker cp`. Script PowerShell menunggu exit code pekerjaan dan memeriksa kepemilikan sebelum membersihkan container sementara: ID lengkap, nama acak, label proyek Compose, service odoo, dan `oneoff=True` harus cocok. Pembersihan ini tidak mencari lalu menghapus sembarang container Odoo.

Blok `finally` mencoba menyalakan Odoo kembali jika sebelumnya berjalan, termasuk ketika backup gagal. Jika sebelumnya berhenti, tetap berhenti. Menulis backup yang berhasil belum membuktikan restore berhasil; uji pemulihan merupakan pemeriksaan lain.

Backup tidak memasukkan source, image, `.secrets`, atau konfigurasi host. Simpan versi source yang sesuai dan secret secara privat terpisah. ZIP berisi data sensitif, tidak dienkripsi, dan harus tetap di luar Git. Jeda penulisan diperlukan untuk konsistensi database/filestore; pemeriksaan koneksi bukan pengganti menghentikan seluruh proses penulis tambahan yang kamu jalankan sendiri.

5. Baca restore sebelum menjalankannya

Restore mengganti database dan filestore. Bagian ini menjelaskan prosedur; jangan mencobanya pada proyek utama atau menjalankannya otomatis untuk melanjutkan tutorial. Buat backup keadaan saat ini dan siapkan instance latihan terpisah yang siap diganti. Pastikan versi addon sama dengan arsip.

Restore.ps1 mewajibkan `-Archive` dan switch `-ReplaceDatabase`. Tanpa switch penggantian, script berhenti sebelum restore. Berikut **contoh sintaks**. `supplyflow-WAKTU-SUFFIX.zip` adalah placeholder yang tidak dibuat tutorial; ganti dengan arsip yang benar-benar ada, dibuat dari instance latihan milikmu sendiri, dan dipercaya. Ini bukan perintah yang perlu dijalankan untuk menyelesaikan latihan UI:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
$labRoot = (Resolve-Path -LiteralPath 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop).ProviderPath
if ((Get-Location).ProviderPath -ne $labRoot -or $env:COMPOSE_PROJECT_NAME -ne 'supplyflow-belajar') {
    throw 'Folder aktif dan nama proyek harus menunjuk instance latihan.'
}
$archivePath = '.\.backups\supplyflow-WAKTU-SUFFIX.zip'
if (-not (Test-Path -LiteralPath $archivePath -PathType Leaf)) {
    throw 'Ganti placeholder dengan berkas backup latihan milik sendiri yang dipercaya.'
}
.\scripts\Restore.ps1 -Archive $archivePath -ReplaceDatabase
```

Urutan restore.py membantu memahami titik kegagalannya:

1. `validate_archive(path, expected_addon_version)` memeriksa arsip lebih dahulu melalui mode `--validate-only`, sebelum aplikasi dihentikan. Path traversal, symlink, berkas khusus, ZIP terenkripsi, nama duplikat, benturan path, format/versi salah, dan payload yang tidak cocok ditolak. Batasnya 100.000 anggota, 32 GiB total sebelum kompresi, serta manifest 16 MiB.
2. Sesudah validasi awal, PowerShell menghentikan Odoo bila sedang berjalan. Proses restore memvalidasi arsip kembali, lalu memeriksa target tetap database `supplyflow`, role/host lokal yang sesuai, PostgreSQL 17, dan tidak ada koneksi aplikasi aktif.
3. `restore_archive()` mengekstrak dump ke lokasi sementara dan filestore ke direktori staging pada volume yang sama. Ukuran dan hash diperiksa saat ekstraksi, sebelum SQL restore. `pg_restore --list` memeriksa struktur dump.
4. `pg_restore` memakai `--clean --if-exists --no-owner --no-privileges --exit-on-error --single-transaction`. Kesalahan SQL membatalkan transaksi restore; flag `--clean` hanya membersihkan objek yang tercantum pada dump, bukan semua objek tambahan sembarang instalasi.
5. Setelah SQL berhasil, filestore lama di-rename ke direktori sementara, staging menggantikannya, lalu salinan lama dibersihkan. `remove_working_directory()` hanya menerima direktori kerja dengan prefix yang diizinkan langsung di parent filestore, tanpa symlink.
6. Odoo dinyalakan kembali hanya jika restore seluruhnya berhasil dan Odoo sebelumnya berjalan. Jika sebelumnya berhenti, tetap berhenti. Jika restore gagal setelah dihentikan, aplikasi dibiarkan berhenti untuk

pemeriksaan.

SHA-256 membuktikan payload sesuai dengan manifest yang dibaca, bukan membuktikan identitas pembuat. Seseorang dapat mengganti payload dan manifest bersamaan. Gunakan hanya arsip sendiri yang dipercaya; dump database dapat menjalankan SQL saat dipulihkan.

PostgreSQL dan filesystem tidak memiliki satu transaksi bersama. SQL dapat sudah berhasil ketika pergantian filestore gagal. Kode mencoba mengembalikan filestore lama jika rename pengganti belum terjadi, tetapi ini tidak membatalkan SQL yang sudah selesai. Pada kegagalan, simpan log dan direktori kerja yang masih diperlukan; jangan langsung menyalakan Odoo atau menghapus volume. OPERATIONS.md menjelaskan keadaan per titik kegagalan.

Setelah pemulihan latihan, periksa satu permintaan/RFQ, satu kasus/retur, stok penting, dan satu lampiran yang dapat dibuka. Login memakai password pengguna pada saat backup, yang mungkin berbeda dari teks kredensial lokal sekarang. Password role PostgreSQL dan master Odoo tetap mengikuti secret host saat ini.

6. Mengapa sisa tes dibersihkan dengan guard?

Test.ps1 membuat nama database baru `supplyflow_test_` diikuti GUID 32 karakter. Script menghentikan Odoo, menjalankan tes pada database tersebut, lalu menghapus database dan filestore tes pada blok `finally`, kecuali `-KeepDatabase` dipilih. Ia tidak otomatis menyalakan aplikasi kembali; setelah tes, nyalakan lewat `Start.ps1` saat ingin memakai UI.

clean_test_filestore.py menolak nama selain pola tersebut. **Potongan asli**, bukan isi lengkap file:

python

```
if not re.fullmatch(r"supplyflow_test_[0-9a-f]{32}", name):
    raise SystemExit("Only a generated SupplyFlow test database filestore may be removed.")
```

Selanjutnya script memastikan parent tepat `/var/lib/odoo/filestore`, target bukan symlink, dan resolved target tetap langsung di parent itu sebelum `shutil.rmtree()`. Guard ini melindungi penghapusan hasil tes dari path salah. Jangan mengganti pemeriksaan tersebut dengan penghapusan seluruh volume atau parent filestore.

7. Latihan kecil: tambah referensi internal permintaan

Kerjakan hanya pada source latihan. Tambahan ini **belum ada pada kode final referensi**; tujuannya mempraktikkan hubungan model, guard, view, update, dan tes. Tambah satu field opsional, tanpa mengubah status atau perhitungan stok.

Di kelas `SupplyFlowRequest` pada purchase_request.py, setelah `purpose`, tambahkan baris latihan baru:

python

```
    internal_reference = fields.Char(string="Internal reference", tracking=True)
```

Masukkan `"internal_reference"` ke `_DRAFT_FIELDS` pada kelas yang sama, tanpa menghapus anggota set yang sudah ada. Ini langkah server yang penting: `write()` menggunakan set itu untuk mewajibkan draft milik pengaju. Readonly pada form saja tidak membekukan field melalui RPC.

Di purchase_request_views.xml, tambahkan elemen latihan berikut di dalam group form dekat `requested_date`:

xml

```
<field name="internal_reference" readonly="state != 'draft' or requester_id != uid"/>
```

Di kelas `TestPurchaseRequest` pada test_purchase_request.py, tambahkan metode latihan berikut. Fixture `_request()` dan `UserError` sudah tersedia pada file tersebut:

python

```
    def test_internal_reference_freezes_after_submission(self):
        request = self._request(internal_reference="LATIHAN-001")
        request.write({"internal_reference": "LATIHAN-002"})
        self.assertEqual(request.internal_reference, "LATIHAN-002")
        request.action_submit()
        with self.assertRaises(UserError):
            request.write({"internal_reference": "DIUBAH-SETELAH-AJU"})
```

Ikuti prosedur update addon pada bab 05, lalu tes seperti bab 06. Periksa field tersimpan pada draft dan perubahan langsung ditolak setelah submitted. Field tambahan ini tidak otomatis masuk ke PDF, RFQ, atau workspace; bagian itu sengaja di luar latihan kecil ini.

Menambahkan method tes latihan ini membuat jumlah tes ORM menjadi 43 pada salinanmu. Angka 42 pada catatan verifikasi adalah baseline proyek referensi sebelum latihan tambahan.

8. Simpan pekerjaan dan hentikan layanan

Simpan source dan catatan hasil, lalu hentikan instance latihan:

powershell

```
cd 'D:\Latihan-Odoo\supplyflow-office' -ErrorAction Stop
$env:COMPOSE_PROJECT_NAME = 'supplyflow-belajar'
.\scripts\Stop.ps1
```

Container berhenti, volume database/filestore tetap ada. Tidak perlu menghapus volume setiap selesai belajar. Jika source latihan sudah memakai Git, tinjau `git status --short` serta `git diff` sebelum commit. Pilih file model, view, dan tes yang berubah; periksa isi staged diff sebelum push.

Jangan push `.secrets`, `.runtime`, `.backups`, `.local`, database dump, screenshot yang menampilkan password, CV/dokumen pribadi, `node_modules`, atau hasil tes. .gitignore menangani folder generated proyek, tetapi tetap periksa file yang akan dipublikasikan. Git menyimpan source; backup menyimpan keadaan data. Mengunggah repo tidak mencadangkan transaksi ERP.

Periksa pemahaman

Pertanyaan	Jawaban yang perlu dapat kamu jelaskan
Mengapa seed memakai `odoo shell`?	Agar script mendapat env, registry model, context pengguna, dan cursor transaksi Odoo
Apa yang terjadi jika seed dipanggil lagi setelah selesai?	Penanda selesai membuatnya berhenti; data dan password latihan dipertahankan, bukan direset
Mengapa stock quant tidak diisi langsung oleh seed?	Receipt/transfer native harus mencatat hubungan dokumen dan perpindahan stok melalui workflow
Mengapa backup menyimpan filestore?	Metadata lampiran di database memerlukan berkas fisik yang sesuai

Pertanyaan	Jawaban yang perlu dapat kamu jelaskan
Mengapa hash valid belum membuat arsip terpercaya?	Hash hanya cocok dengan manifest; pembuat arsip yang tidak dipercaya dapat mengganti keduanya
Apa arti `--single-transaction` pada restore?	SQL restore menjadi satu transaksi, tetapi pergantian filestore sesudahnya tetap operasi filesystem terpisah
Mengapa field latihan harus masuk `_DRAFT_FIELDS`?	Supaya aturan server membekukan perubahan, termasuk pemanggilan langsung di luar form
Apakah commit source menyimpan stok latihan?	Tidak; stok berada pada database, dan cadangan data memakai arsip database plus filestore

Gunakan indeks file dan fungsi ketika ingin mencari kembali helper yang menjalankan salah satu langkah tersebut.

08. Indeks file, class, dan fungsi

Gunakan bab ini ketika sudah tahu pertanyaannya tetapi belum tahu file yang perlu dibuka. Nomor bab mengarah ke penjelasan bertahap. Tautan fungsi memakai `#L` untuk baris definisi pada source final; jika kode diubah kemudian, cari nama fungsinya karena nomor baris dapat bergeser.

`Masukan` berarti argumen atau record yang dibaca. `Hasil/efek` membedakan nilai yang dikembalikan dari perubahan database, file, atau tampilan. Fungsi compute biasanya mengisi field dan tidak mengembalikan dokumen baru. Fungsi pemeriksaan umumnya melanjutkan operasi bila sah atau menaikkan exception bila tidak sah.

Peta semua berkas teks proyek

Path pada tabel relatif terhadap root repo. Berkas teks di bawah mencakup konfigurasi, source, pengujian, lisensi, dan dokumentasi. Secret lokal, arsip, dependensi, serta hasil tes memang tidak termasuk source yang dipublikasikan.

Inventaris fungsi ERP mengikuti source implementasi yang dijelaskan pada tutorial. Tool dokumentasi tambahan scripts/build_tutorial_pdf.py membaca Markdown dan menghasilkan buku PDF dengan ReportLab; ia bukan bagian dari server ERP. Daftar operasi file, validasi, dan alur fungsi di bawah merujuk aplikasi serta alat operasinya.

Akar dan CI

Berkas	Untuk apa	Bab
.gitattributes	Aturan akhir baris LF untuk source tertentu dan penanda berkas biner.	01
.github/workflows/verify.yml	GitHub Actions menjalankan tes arsip, ORM, startup latihan, dan browser; menyimpan diagnostik jika gagal.	06
.gitignore	Mengecualikan secret, konfigurasi lokal, cadangan, dependensi, dan hasil tes dari Git.	01 / 07
LICENSE	Teks lisensi proyek dan ketentuan LGPL/GPL yang disertakan; bukan kode aplikasi.	02
README.md	Ringkasan fitur, screenshot, startup, batas kebijakan, dan tautan dokumentasi.	01
compose.test.yaml	Overlay yang mematikan healthcheck Odoo saat proses tes sekali jalan.	06
compose.yaml	Dua layanan, image, secret, bind mount, volume, healthcheck, port localhost, dan batas memori Odoo.	01
package-lock.json	Versi dependensi npm terkunci untuk pemasangan ulang dengan npm ci.	06

Berkas	Untuk apa	Bab
package.json	Dependensi Playwright serta perintah browser test dan capture; bukan frontend ERP terpisah.	06
playwright.config.js	Browser, URL, satu worker, batas waktu, trace CI, dan direktori hasil pengujian.	06

Addon Python, keamanan, dan antarmuka

Berkas	Untuk apa	Bab
addons/supplyflow_office/__init__.py	Memuat paket models ketika addon dibaca Odoo.	02
addons/supplyflow_office/__manifest__.py	Identitas addon, dependensi, urutan data XML/CSV, dan daftar aset web.	02 / 05
addons/supplyflow_office/data/sequences.xml	Sequence nomor permintaan dan kasus pemasok.	02 / 03 / 04
addons/supplyflow_office/models/__init__.py	Memuat purchase_request, supplier_issue, dan workspace agar model terdaftar.	02 / 03 / 04 / 05
addons/supplyflow_office/models/purchase_request.py	Dua model permintaan dan baris: estimasi, CRUD terjaga, keputusan, revisi, konversi RFQ.	03
addons/supplyflow_office/models/supplier_issue.py	Kasus receipt, baris sumber, kuantitas retur, wizard native, dan revisi stock.move.	04
addons/supplyflow_office/models/workspace.py	Model RPC tanpa tabel bisnis baru untuk hitungan dan daftar permintaan terakhir.	05
addons/supplyflow_office/reports/purchase_request_report.xml	Action laporan dan template QWeb PDF ringkasan permintaan.	05
addons/supplyflow_office/security/company_rules.xml	Empat record rule yang membatasi dokumen dan baris pada perusahaan aktif.	02
addons/supplyflow_office/security/groups.xml	Peran SupplyFlow User dan Approver serta hubungan grupnya.	02
addons/supplyflow_office/security/ir.model.access.csv	Hak CRUD model untuk grup; pemeriksaan pemilik dan status dilanjutkan di Python.	02 / 03 / 04
addons/supplyflow_office/static/src/workspace.js	Komponen Owl memuat RPC, keadaan tampilan, dan navigasi ke dokumen native.	05

Berkas	Untuk apa	Bab
addons/supplyflow_office/static/src/workspace.scss	Gaya workspace dan penyesuaian form pada layar kecil.	05
addons/supplyflow_office/static/src/workspace.xml	Template Owl untuk loading, error, kosong, metrik, dan daftar terbaru.	05
addons/supplyflow_office/views/menus.xml	Menu SupplyFlow dan client action workspace.	05
addons/supplyflow_office/views/purchase_request_views.xml	List, form, search, tombol workflow, dan window action permintaan.	03 / 05
addons/supplyflow_office/views/supplier_issue_views.xml	List, form, search, tombol kasus, dan smart button pembuka retur.	04 / 05

Script operasional

Berkas	Untuk apa	Bab
scripts/Backup.ps1	Menghentikan penulis Odoo, menjalankan job backup, menyalin ZIP, dan membersihkan job terverifikasi.	07
scripts/Common.ps1	Fungsi Compose bersama, password acak, secret, konfigurasi, dan petunjuk login lokal.	01 / 07
scripts/Load-TrainingData.ps1	Memuat data latihan sambil menghentikan aplikasi dan mempertahankan keadaan start/stop sebelumnya.	07
scripts/Restore.ps1	Opsi penggantian wajib, validasi awal, penghentian Odoo, restore, dan kebijakan restart jika berhasil.	07
scripts/Start.ps1	Inisialisasi konfigurasi, pull opsional, start database, install addon, seed opsional, lalu start Odoo.	01
scripts/Stop.ps1	Menghentikan layanan proyek tanpa menghapus volume data.	01 / 07
scripts/Test.ps1	Database uji bernama unik, instalasi dan tes addon, lalu pembersihan kecuali KeepDatabase.	06
scripts/container/backup.py	Membuat ZIP privat berisi dump custom PostgreSQL, filestore, manifest, ukuran, dan SHA-256.	07
scripts/container/clean_test_filestore.py	Script tanpa fungsi: menghapus hanya filestore dengan pola nama database tes dan path yang diperiksa.	06

Berkas	Untuk apa	Bab
scripts/container/configure_instance.py	Script shell Odoo satu kali: admin, grup, perusahaan IDR, URL, dan penanda initialized; tidak mendefinisikan fungsi.	01 / 07
scripts/container/init-database.sh	Membuat role supplyflow yang tidak superuser dan mengatur pemilik database saat volume PostgreSQL baru diinisialisasi.	01
scripts/container/manage.py	Dispatcher initialize/update/seed dan pemanggilan shell Odoo.	01 / 07
scripts/container/restore.py	Memvalidasi arsip dan memulihkan database tetap supplyflow serta filestore staging.	07
scripts/container/training_data.py	Seed sintetis opsional melalui workflow nyata dengan XML ID untuk penggunaan ulang record.	07

Pengujian

Berkas	Untuk apa	Bab
addons/supplyflow_office/tests/__init__.py	Memuat empat file pengujian ORM saat tes addon dijalankan.	06
addons/supplyflow_office/tests/test_distribution_flow.py	Dua tes end-to-end ORM: receipt sebagian/backorder/retur/delivery dan transfer dua gudang.	04 / 06
addons/supplyflow_office/tests/test_purchase_request.py	17 tes permintaan: RFQ, harga nol, revisi, izin, perusahaan, angka, context, dan mutasi baris.	03 / 06
addons/supplyflow_office/tests/test_supplier_issue.py	20 tes kasus: retur, reservasi pending, shortage, konteks wizard, sumber, peran, dan angka.	04 / 06
addons/supplyflow_office/tests/test_workspace.py	Tiga tes hitungan berdasarkan record dan penolakan pengguna tanpa peran.	05 / 06
tests/browser/helpers.js	Delapan helper login, RPC, klik workflow tersinkron, indeks latihan, dan navigasi.	06
tests/browser/screenshots.spec.js	Satu skenario capture menghasilkan delapan screenshot dari aplikasi yang berjalan.	05 / 06
tests/browser/workflows.spec.js	Sembilan skenario browser: workspace, gagal/retry, PDF, form, reviewer, retur, revisi, shortage, dan konkurensi HTTP.	06

Berkas	Untuk apa	Bab
tests/test_backup_archive.py	25 unit test validator ZIP dengan arsip sintetis; tidak memerlukan Docker.	06 / 07

Dokumentasi pendamping

Berkas	Untuk apa	Bab
docs/ARCHITECTURE.md	Pemisahan proses Odoo standar, addon, database, server, dan batas kebijakan.	01 / 02
docs/CODE-GUIDE.md	Panduan source singkat sebagai pendamping tutorial bertahap ini.	03 / 04 / 05
docs/DELIVERY-GATE.md	Pemeriksaan tampilan, keyboard, kontras, error, dan layar kecil.	05 / 06
docs/DESIGN.md	Keputusan tampilan operasi yang mengikuti antarmuka native Odoo.	05
docs/FLOWS.md	Diagram alur pembelian, penerimaan, penjualan, transfer, dan penyelesaian pemasok.	03 / 04
docs/OPERATIONS.md	Perintah operasional, secret, backup, restore, diagnosis, dan batas pemulihan.	07
docs/PRD.md	Kebutuhan distributor, pelaku, ruang lingkup, dan kriteria penerimaan.	01
docs/VERIFICATION.md	Bukti tes, PDF, konkurensi, backup/restore, dan batas hasil verifikasi.	06 / 07

Folder screenshots berisi delapan PNG biner: `01-operations.png`, `02-purchase-requests.png`, `03-approved-request.png`, `04-native-purchase.png`, `05-supplier-resolution.png`, `06-native-receipt.png`, `07-native-sale.png`, dan `08-mobile-workspace.png`. Screenshot memperlihatkan data latihan/pemeriksaan lokal; kode pengambilannya ada pada `screenshots.spec.js`.

Tutorial ini sendiri terdiri dari README serta bab berikut. Ini juga berkas dokumentasi, bukan file yang perlu dimuat manifest addon.

Berkas tutorial	Isi
01-persiapan-dan-docker.md	Lingkungan lokal dan container
02-kerangka-addon-dan-akses.md	Checkpoint addon minimal dan akses
03-model-permintaan-dan-rfq.md	Permintaan, baris, persetujuan, RFQ
04-receipt-masalah-dan-retur.md	Receipt, kasus, dan retur
05-tampilan-workspace-dan-pdf.md	Penyatuan addon final, view, Owl, QWeb

Berkas tutorial	Isi
06-pengujian-dan-debugging.md	Pengujian dan diagnosis
07-operasi-dan-latihan.md	Seed, operasi, dan latihan utuh
08-indeks-file-dan-fungsi.md	Indeks yang sedang dibaca

Class Python: tempat menaruh field dan method

Nama class adalah nama pada source Python. Nama model Odoo adalah string `_name` atau model yang diperluas melalui `_inherit`. Ejaan `SupplyflowIssue` memang mengikuti source; jangan menggantinya dengan nama class lain saat mencari.

Class dan source	Peran
SupplyFlowRequest	models.Model; tabel dokumen supplyflow.request, bidang kebutuhan dan jejak keputusan.
SupplyFlowRequestLine	models.Model; tabel supplyflow.request.line, daftar barang pada satu permintaan.
SupplyflowIssue	models.Model; tabel supplyflow.issue, masalah pada receipt dan hasil review.
SupplyflowIssueLine	models.Model; tabel supplyflow.issue.line, barang sumber dan jumlah terdampak.
StockMove	Perluasan stock.move; menambah revisi internal, tidak membuat model bisnis pengganti.
SupplyFlowWorkspace	models.AbstractModel supplyflow.workspace; menyediakan RPC, bukan tabel transaksi baru.

Semua fungsi Python aplikasi dan script

Ada 86 definisi fungsi non-tes pada source ini, termasuk method class dan helper yang bersarang dalam `seed_training()`. Callback lambda tidak mempunyai nama `def` terpisah. Script `configure_instance.py` dan `clean_test_filestore.py` berjalan di tingkat file, sehingga fungsinya dijelaskan pada peta file.

Baris tabel berikut menyebut input serta hasil/efek utama; urutan pemanggilan dan cuplikan lengkap ada pada bab terkait. Pemeriksaan izin tambahan tetap dapat berlangsung di ORM atau model native yang dipanggil.

Permintaan dan RFQ

Source: addons/supplyflow_office/models/purchase_request.py. Pembahasan: bab 03.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyFlowRequest._compute_amount_estimated	Subtotal baris -> isi amount_estimated; tolak total di luar kemampuan mata uang.
SupplyFlowRequest._onchange_company_id	Perusahaan pada form -> pilih default gudang perusahaan itu untuk membantu pengisian.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyFlowRequest._check_request_details	Record permintaan -> validasi tujuan, perusahaan pemasok, dan gudang; naikkan ValidationError bila tidak sesuai.
SupplyFlowRequest._require_group	ID grup (default User) -> tolak pengguna tanpa peran SupplyFlow yang diminta.
SupplyFlowRequest._lock_for_change	Recordset -> periksa hak tulis, flush, lock, invalidasi cache, periksa ulang akses sebelum perubahan.
SupplyFlowRequest._require_requester	Recordset -> tolak bila pengguna saat ini bukan pengaju salah satu dokumen.
SupplyFlowRequest._require_independent_approver	Recordset -> wajib Approver yang berbeda dari requester dan pembuat record.
SupplyFlowRequest._require_draft	Recordset -> wajib draft milik pengaju; dipakai bersama jalur edit isi.
SupplyFlowRequest._transition	Dictionary field tervalidasi -> super().write internal untuk status/audit setelah guard action.
SupplyFlowRequest.create	Daftar dictionary -> record baru bernomor otomatis, draft milik pengguna; saring audit dan default context.
SupplyFlowRequest.write	Dictionary perubahan -> simpan isi draft/catatan reviewer yang sah; tolak perubahan audit langsung.
SupplyFlowRequest.unlink	Recordset -> hapus hanya draft milik pengaju setelah lock dan pemeriksaan.
SupplyFlowRequest.action_submit	Draft berisi barang -> submitted, pelaku/waktu pengajuan, pesan chatter; kembalikan True.
SupplyFlowRequest.action_approve	Submitted dan reviewer independen -> approved, pelaku/waktu, chatter; kembalikan True.
SupplyFlowRequest.action_reject	Submitted dengan alasan -> rejected, pelaku/waktu, chatter; penolakan kosong ditolak.
SupplyFlowRequest.action_reset_draft	Rejected milik pengaju -> draft dan audit keputusan aktif kosong; riwayat chatter dipertahankan.
SupplyFlowRequest.action_cancel	Record dan pelaku sesuai status -> cancelled sebelum RFQ tercipta, dengan catatan pelaku.
SupplyFlowRequest.action_create_rfq	Satu approved dan izin Purchase -> buat/tautkan satu RFQ atau buka yang sudah ada; kembalikan window action.
SupplyFlowRequest._rfq_action	purchase_order_id -> dictionary pembuka form purchase.order; tidak mengubah stok.
SupplyFlowRequestLine._compute_name	Produk -> isi awal deskripsi dengan display_name produk.
SupplyFlowRequestLine._compute_product_uom_id	Produk -> isi satuan baris dari satuan produk.
SupplyFlowRequestLine._compute_subtotal	Jumlah dan harga estimasi -> isi subtotal hasil perkalian.
SupplyFlowRequestLine._check_line_details	Baris tersimpan -> validasi angka, presisi satuan, produk yang dapat dibeli/dilacak, dan perusahaan.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyFlowRequestLine._validate_numeric_input	Jumlah, harga, satuan, mata uang -> pasangan angka float tervalidasi sebelum konversi field.
SupplyFlowRequestLine.create	Dictionary baris dan induk -> baris baru hanya pada draft sendiri; lindungi field turunan/default context.
SupplyFlowRequestLine.write	Dictionary perubahan -> edit baris draft sendiri setelah validasi kombinasi angka; larang pindah induk.
SupplyFlowRequestLine.unlink	Baris -> hapus hanya bila induk draft milik pengaju, setelah guard dan lock.

Receipt, kasus, retur, dan stock.move

Source: addons/supplyflow_office/models/supplier_issue.py. Pembahasan: bab 04.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyflowIssue._check_member	Pengguna saat ini -> wajib peran SupplyFlow User sebelum menangani kasus.
SupplyflowIssue._check_approver	Pengguna saat ini -> wajib User dan Approver untuk review kasus.
SupplyflowIssue._check_draft_owner	Recordset -> wajib akses tulis, status draft, dan pelapor yang sama untuk edit isi.
SupplyflowIssue._transition	Dictionary yang sudah diperiksa action -> tulis status/audit melalui implementasi induk.
SupplyflowIssue._lock	Recordset -> periksa akses tulis, lock, invalidasi cache, dan periksa akses kembali.
SupplyflowIssue._check_independent_reviewer	Kasus -> wajib Approver yang berbeda dari pelapor untuk mengambil keputusan.
SupplyflowIssue.create	Daftar dictionary -> kasus draft bernomor dan pelapor saat ini; audit/link serta default berbahaya disaring.
SupplyflowIssue.write	Perubahan isi/catatan -> simpan sesuai pemilik dan tahap review; field status/audit langsung ditolak.
SupplyflowIssue.unlink	Kasus -> hapus hanya draft milik pelapor setelah lock.
SupplyflowIssue._check_receipt	Kasus -> pastikan receipt incoming sudah Done, perusahaan cocok, resolver internal, dan deskripsi terisi.
SupplyflowIssue._check_lines	Kasus dan returning flag -> periksa baris/sumber; shortage tidak boleh retur dan jumlah tidak melampaui sisa.
SupplyflowIssue.action_submit	Satu draft sah -> submitted dengan pesan chatter; kembalikan True.
SupplyflowIssue.action_approve	Satu submitted dan reviewer independen -> approved untuk retur, isi audit; barang belum bergerak.
SupplyflowIssue.action_accept_resolution	Submitted dengan catatan reviewer -> accepted dan audit keputusan; tidak membuat retur stok.
SupplyflowIssue.action_cancel	Kasus aktif dan pelaku berizin -> cancelled; kasus yang sudah selesai/bereturan tidak dibatalkan di sini.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyflowIssue._return_action	Tautan retur yang dapat dibaca -> dictionary pembuka form stock.picking.
SupplyflowIssue.action_open_return	Satu kasus dan anggota berhak baca -> buka retur yang tertaut; tolak bila belum ada.
SupplyflowIssue.action_create_return	Approved dan Approver -> lock sumber, periksa sisa, jalankan wizard, tautkan satu retur; kembalikan action.
SupplyflowIssueLine._validated_quantity_input	Nilai jumlah -> float positif finite dalam kemampuan presisi; selain itu ValidationError.
SupplyflowIssueLine._compute_returnable_qty	Sumber move dan retur terkait -> isi returnable_qty yang dihitung saat dibaca.
SupplyflowIssueLine._remaining_returnable_quantity	Satu sumber move -> jumlah diterima dikurangi retur Done dan komitmen pending non-cancel, satuan disamakan.
SupplyflowIssueLine._check_source_move	Baris -> wajib move Done dari receipt kasus, perusahaan cocok, dan jumlah positif sesuai presisi.
SupplyflowIssueLine.create	Dictionary baris -> baris pada draft sendiri dengan jumlah tervalidasi dan field sumber terlindungi.
SupplyflowIssueLine.write	Dictionary perubahan -> edit draft sendiri; larang pindah kasus dan pemalsuan detail sumber.
SupplyflowIssueLine.unlink	Baris -> hapus hanya setelah induk dikunci dan kepemilikan draft diperiksa.
StockMove.write	Dictionary perubahan move -> lanjutkan penulisan native kecuali revisi retur dikirim secara langsung.
StockMove._supplyflow_advance_return_revision	Move sumber terkunci -> naikkan penghitung revisi internal agar transaksi bersaing dengan snapshot lama perlu retry.

RPC workspace

Source: addons/supplyflow_office/models/workspace.py. Pembahasan: bab 05.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
SupplyFlowWorkspace.get_overview	Lingkungan pengguna/perusahaan -> dictionary hitungan pekerjaan dan delapan permintaan terbaru, mengikuti ACL/rule; tidak membuat data.

Manajemen server container

Source: scripts/container/manage.py. Pembahasan: bab 01 / 07.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
manage.run_shell	Nama script container -> baca lalu jalankan lewat odoo shell yang menyediakan env; kegagalan subprocess diteruskan.
manage.main	Mode initialize/update/seed -> instal bila belum ada, update addon, atau jalankan seed; initialize juga konfigurasi satu kali.

Data latihan termasuk helper bersarang

Source: scripts/container/training_data.py. Pembahasan: bab 07.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
training_data.seed_training	Environment ORM -> data sintetis, indeks XML ID, penanda selesai, commit; bila sudah lengkap, pertahankan record dan password.
training_data.seed_training.existing	Nama XML ID latihan dan model -> record yang masih ada; tolak bila ID menunjuk model berbeda.
training_data.seed_training.bind	Nama ID dan satu record -> daftarkan/periksa XML ID, tambahkan ID ke indeks, kembalikan record.
training_data.seed_training.ensure	Nama ID, model, callback pembuat -> gunakan record lama atau buat satu, lalu bind; menghindari duplikasi ID latihan.
training_data.seed_training.create_user	Peran dari loop -> akun sintetis dengan grup/password peran; tolak login yang sudah dimiliki di luar dataset.
training_data.seed_training.make_warehouse	Nama dan kode -> gudang latihan baru; tolak konflik kode yang sudah terpakai.
training_data.seed_training.make_product	Nama, kode, biaya, harga -> produk stok yang bisa dibeli/dijual tanpa pajak latihan; tolak kode yang berbenturan.
training_data.seed_training.create_request	Tujuan dan daftar produk/jumlah/biaya -> draft milik buyer pada pemasok/gudang latihan.
training_data.seed_training.submit	Permintaan -> jalankan Submit sebagai buyer hanya jika masih draft; tidak mengembalikan record baru.
training_data.seed_training.approve	Permintaan -> ajukan bila perlu dan setujui sebagai approver bila submitted.
training_data.seed_training.create_issue	Deskripsi, alasan, jumlah, deadline -> draft kasus milik warehouse pada receipt latihan.
training_data.seed_training.create_branch_transfer	Lingkungan gudang/produk latihan -> buat transfer internal sepuluh folder; validasi dilakukan kode pemanggil setelahnya.

Pembuatan arsip

Source: scripts/container/backup.py. Pembahasan: bab 07.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
backup.addon_version	Manifest lokal -> string versi addon yang cocok seri Odoo 19.
backup.database_environment	Konfigurasi tetap lokal -> environment PG termasuk password, tanpa mencetak secret; tolak target lain.
backup.checked_run	Daftar argumen, environment, capture flag -> CompletedProcess; kegagalan proses menjadi exception.
backup.postgres_version	Environment koneksi -> angka versi server, hanya menerima mayor PostgreSQL 17.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
backup.check_quiet_database	Koneksi database -> tolak bila ada koneksi lain dengan user aplikasi yang sama; tidak menulis data.
backup.filestore_files	Lokasi filestore tetap -> daftar berkas biasa terurut; tolak symlink dan berkas khusus.
backup.add_file	ZIP, file sumber, nama anggota, kompresi -> tulis anggota sambil hitung path, ukuran, SHA-256; kembalikan metadata.
backup.create_backup	Path ZIP baru langsung di /tmp -> pg_dump + filestore + manifest; cetak ringkasan JSON dan bersihkan output gagal miliknya.
backup.main	Argumen --output -> jalankan backup; kembalikan kode 0 atau 1 untuk kegagalan yang ditangani.

Validasi dan pemulihan arsip

Source: scripts/container/restore.py. Pembahasan: bab 07.

Fungsi dan baris source	Masukan -> hasil/efek dan alasan
restore.addon_version	Manifest lokal -> string versi addon untuk dibandingkan dengan manifest backup.
restore.database_environment	Konfigurasi lokal -> environment PG untuk database supplyflow saja; password tetap privat.
restore.validate_member	ZipInfo -> tolak traversal, path asing, symlink, direktori, enkripsi, dan ukuran anggota di luar batas.
restore.validate_archive	Path ZIP dan versi addon -> manifest setelah pemeriksaan anggota, batas ukuran, nama, versi, hash, dan header dump; belum restore.
restore.check_filestore_location	Path filestore tetap -> tolak pengalihan symlink, target bukan direktori, atau berkas khusus.
restore.remove_working_directory	Path kerja restore -> hapus hanya staging/previous di bawah parent filestore yang diperiksa.
restore.restore_archive	ZIP dan manifest -> ekstraksi staging dengan hash ulang, pg_restore transaksi tunggal, lalu tukar filestore; cetak ringkasan.
restore.main	--archive dan satu mode wajib -> validasi saja atau restore pengganti; kembalikan kode 0/1 yang ditangani.

Semua fungsi PowerShell

Script startup, stop, test, seed, dan restore sebagian besar berupa langkah berurutan di tingkat file. Empat fungsi bernama berikut dipakai ulang atau memeriksa kepemilikan job backup. Pembahasan ada pada bab 01 dan 07.

Fungsi dan source	Masukan -> hasil/efek dan alasan
Backup.ps1::Assert-BackupContainerOwnership	ID container 64 heksadesimal -> cocokkan nama unik dan label project/service/oneoff; throw sebelum operasi pada container asing.

Fungsi dan source	Masukan -> hasil/efek dan alasan
Common.ps1::Invoke-Compose	Argumen native Docker -> jalankan Compose pada root repo, teruskan output, throw bila exit code bukan nol; memakai @args agar -d/-v tidak dicuri binder.
Common.ps1::New-LocalPassword	Tanpa argumen -> string Base64 dari 30 byte acak kriptografis; tidak menulis atau mencetak password sendiri.
Common.ps1::Initialize-LocalConfiguration	Keadaan file dan volume -> buat secret baru hanya bila belum ada serta tulis konfigurasi/kredensial lokal; tolak secret hilang pada volume lama.

Semua method komponen Owl

Source: static/src/workspace.js. Nama class sama dengan class RPC Python, tetapi objek ini berjalan di browser. Pembahasan: bab 05.

Method dan source	Masukan -> hasil/efek dan alasan
SupplyFlowWorkspace.setup	Pembuatan komponen -> ambil layanan ORM/action, siapkan state reaktif, jadwalkan load sebelum tampilan awal.
SupplyFlowWorkspace.load	Pemanggilan awal atau Retry/Refresh -> RPC get_overview, isi data atau pesan error, selalu akhiri loading.
SupplyFlowWorkspace.openRequests	pending boolean -> buka window action permintaan, opsional filter Awaiting Approval; kembalikan hasil doAction.
SupplyFlowWorkspace.openIssues	Nama filter atau kosong -> buka kasus dengan default filter yang sesuai metrik; kembalikan hasil doAction.
SupplyFlowWorkspace.openRequest	ID permintaan -> buka form satu supplyflow.request pada jendela saat ini.

Semua ekspor helper browser

Source: tests/browser/helpers.js. Helper ini mendukung pengujian, bukan endpoint atau bagian server ERP. Pembahasan: bab 06.

Ekspor dan source	Masukan -> hasil/efek dan alasan
helpers.credentials	Peran (default admin) -> objek login/password dari secret lokal; jangan menampilkan hasilnya di log atau README.
helpers.login	Page dan peran -> isi form login lalu tunggu navbar; menyiapkan sesi UI yang benar sebelum langkah tes.
helpers.rpc	Page, model, method, args, kwargs -> panggil call_kw dengan sesi itu, kembalikan result atau throw bila RPC error.
helpers.clickWorkflow	Page, model/method, nama tombol, opsi confirm -> klik dan tunggu respons call_button yang tepat; kembalikan action setelah form native siap bila relevan.
helpers.authenticateApi	APIRequestContext dan peran -> autentikasi sesi HTTP untuk database supplyflow; tolak bila tidak mendapat uid.

Ekspor dan source	Masukan -> hasil/efek dan alasan
helpers.trainingIndex	Page admin -> ambil parameter indeks latihan melalui RPC dan kembalikan objek hasil JSON.parse.
helpers.openAction	Page dan XML ID action -> navigasi ke /odoo/action-... dan tunggu action manager terlihat.
helpers.openRecord	Page, XML ID action, ID record -> navigasi ke form record dan tunggu form terlihat.

Jika mencari tes atau fungsi bawaan Odoo

Method test dan fixture dikelompokkan per file pada peta pengujian di atas. Untuk membaca urutan `setUpClass()`, helper pembuat record, assertion, serta sembilan skenario browser, gunakan bab 06. Angka 42 adalah jumlah method tes ORM, bukan jumlah seluruh fungsi aplikasi.

Nama seperti `purchase.order.create()`, `stock.picking.button_validate()`, dan `stock.return.picking.action_create_returns()` berasal dari modul native Odoo. Source-nya tidak disalin ke repo SupplyFlow. Indeks ini mencatat fungsi yang ditulis dalam proyek, sedangkan bab 03 dan 04 menunjukkan titik pemanggilan proses native tersebut.

Contoh pencarian di terminal dari root repo: `rg -n "def action_create_rfq|action_create_rfq" addons/supplyflow_office`. Hasilnya memperlihatkan definisi Python, tombol XML, dan tes yang memanggilnya. Dari situ, ikuti satu nama fungsi yang sama melalui tampilan, aturan bisnis, lalu pengujian.